From Propagators to Replicas

Conflict-Free Replicated Data Types in Lean 4

> *"A replica does not store a value; it stores everything it has heard so far about a value. The only thing that has changed since the last book is the postal service."*
>
> — this book, Chapter 2

The Sequel to *From Zero to Propagators*:
Distributed Convergence as the Algebra of Join

Lean 4 code requires no external libraries beyond the standard prelude.

Edition 1.0

Preface

The previous book in this series, *From Zero to Propagators*, ended with a machine: a network of cells, each holding partial information in a bounded join-semilattice, each write a merge, the whole thing running to a fixpoint inside one process. This book takes that machine apart and mails the pieces to different continents.

The thesis. A state-based CRDT — a *conflict-free replicated data type* — is a propagator cell with a network attached. Each replica is a cell; merging is the join; an update is a monotone step upward; and *strong eventual consistency is nothing but the algebra of join.* Because $\sqcup$ is commutative, associative, and idempotent, replicas that have received the same set of updates — in any order, any number of times — compute the same fold and therefore hold equal state. The network's three sins (reordering, duplication, redelivery) are absorbed by three algebraic laws. Nothing else is needed, and — this book proves by refutation — nothing less suffices.

Who this book is for. Readers of the predecessor. We assume everything it built: partial orders, lattices, bounded join-semilattices, monotone maps, Knaster–Tarski, the propagator scheduler, and working fluency with Lean 4 tactics (`induction`, `cases`, `simp`, `omega`, `decide`). We recap where memory needs jogging; we do not re-teach. Two pieces of Lean the predecessor never needed are taught here from scratch, deliberately and slowly: *quotient types* (Chapter 8, where multisets of delivered updates live) and *well-founded recursion* in anger (Chapter 9, where gossip provably terminates).

The Lean 4 philosophy here. Unchanged from the predecessor: every definition and theorem has a Lean 4 encoding in the companion file `Crdt.lean`; no Mathlib, no dependencies beyond the prelude; the file compiles with a bare `lean Crdt.lean` and contains zero `sorry`. Where a Mathlib or core-library equivalent exists we say so in a note. Every claimed counterexample is `decide`-checked, every printed output in a "Computation Interlude" is the actual output of `#eval`, and the axiom footprint of every headline theorem is audited at the end of the file (the entire main chain uses only `propext` and `Quot.sound`; classical choice appears nowhere).

Refutations are first-class. Distributed systems folklore is full of plausible designs that do not work. This book celebrates them: each chapter of Part II derives the right design by first refuting the obvious one, in a red box, with a machine-checked counterexample. When you finish, you will not merely believe that a G-Counter needs per-replica slots — you will have watched `decide` reject the alternative.

How to read this book. Part I motivates replication and restates the propagator

vocabulary as a *merge discipline*. Part II builds the classic zoo — G-Counter, PN-Counter, G-Set, 2P-Set, LWW-Register, LWW-Element-Set, OR-Set — each a Lean structure with a verified semilattice instance. Part III proves the main theorem: strong eventual consistency for any state-based CRDT. Part IV covers delivery semantics, version vectors, operation-based CRDTs, a capstone replicated store with a simulated adversarial network, and an honest closing chapter on what CRDTs cannot do.

A note on Lean Unicode. All conventions carry over: `⊑ ⊔ ⊥ ∀ →` and friends are single characters entered with backslash sequences. Two new orders join the cast: `≼`/`≺` for the *element* order inside sorted collections (Chapter 5), carefully distinct from the *information* order `⊑` on replica states.

Contents

Part I

From One Process to Many

Chapter 1

Replicas, Conflicts, and the Price of Coordination

> *"In an asynchronous network that can lose messages, no service can be both always available and always consistent. When the network fails — and it will — you must choose which promise to keep."*
> — *after Gilbert & Lynch (2002)*

The previous book ended inside one process. Cells lived in one address space, propagators ran on one scheduler, and when the network of constraints reached its fixpoint, it did so because *we* decided the order in which every step ran. This chapter is about what happens when that luxury disappears: when the same piece of state lives on several machines at once, and the machines can only talk to each other through a channel that delays, drops, duplicates, and reorders whatever it is given.

We will call each copy of the state a *replica*. The question of the whole book is a single one:

What discipline must replicas obey so that, no matter how badly the network behaves, they end up agreeing?

The answer, it will turn out, is a discipline you already know. But we must first watch things go wrong without it.

1.1 Three Places at Once

Replication is not an exotic requirement; it is the default condition of modern software.

- **Offline edits.** Your phone edits a note in a tunnel; your laptop edits the same note at home. Both devices hold a replica, and neither can ask the other for permission before accepting your keystrokes.
- **Multi-datacenter writes.** A service accepts writes in Frankfurt and in Oregon, because sending every Frankfurt write on a 250-millisecond round trip to Oregon is a product decision nobody wants to ship. Each datacenter holds a replica.
- **Collaborative editing.** Two people type into the same document. Waiting for a lock on every keystroke would make the experience indistinguishable from taking turns.

In each case the system keeps several copies of one logical value, lets each copy accept updates *locally*, and reconciles later. The reconciliation step is where designs live or die. Accepting updates locally is easy. Merging them afterwards — so that every replica ends up with the same state, and that state reflects every update — is the hard part, and doing it casually produces bugs of a particularly nasty kind: silent ones.

1.2 A Counter, Twice

Let us make the smallest possible distributed system and break it. The state is a counter; the update is "increment"; there are two replicas, A and B, both starting at 0.

Suppose A observes two increments and B observes one. The ground truth — the number of increment events that actually happened in the world — is three. Now the replicas synchronize. The obvious strategy, the one every ad-hoc system reaches for first, is *overwrite*: when you sync, take the other side's value.

Watch closely.

event	replica A	replica B	truth
start	0	0	0
A: increment	1	0	1
A: increment	2	0	2
B: increment	2	1	3
B syncs from A	2	2	3
A syncs from B	2	2	3

The replicas agree. They agree on the wrong answer.

B's increment was not rejected, logged, or reported as a conflict. It was simply erased, in the fourth row, by a synchronization step that looked perfectly innocent. Nobody will notice until an invoice is wrong. This failure mode — *the lost update* — is the original sin of replication, and any design that can exhibit it under ordinary operation is broken no matter how rarely the loss occurs.

It is tempting to respond with scheduling: "sync in the other direction first," or "always sync the replica with the newer timestamp." Hold that thought — Chapter 6 treats timestamps seriously, and Refutation 1.1 below disposes of the schedule-shuffling idea wholesale. The disease is not the order of the syncs. The disease is that overwrite *forgets*.

1.3 The Price of a Leader

There is a classical cure for divergence: forbid it. Nominate one replica the *leader*; all writes go to the leader; everyone else holds a read-only copy. No merge, no conflict, no lost update — because there is never more than one writable copy.

The cure works, and you should know its price list.

- **A round trip per write.** Every increment from Frankfurt now waits on Oregon. Physics sets a floor under your latency that no amount of engineering removes.
- **Unavailability equals leader failure.** When the leader dies, or merely becomes unreachable, writes stop — everywhere. Electing a new leader safely is a consensus problem, which is to say: slow, subtle, and itself unavailable during partitions.
- **No offline operation, ever.** The phone in the tunnel cannot reach the leader, so the phone in the tunnel cannot take your keystrokes. This is not an implementation detail to be optimized away; it is the design working as intended.

Coordination buys consistency and charges availability. Sometimes that is the right trade — Chapter 13 is honest about when. But this book explores the other branch: what is the *most* we can guarantee if we refuse to coordinate at all?

1.4 CAP in One Honest Page

The trade-off has a theorem attached: the *CAP theorem*, named for the three properties it juggles — *C*onsistency, *A*vailability, and *P*artition tolerance. It is worth stating correctly, because its folklore version has done real damage.

CAP (Brewer's conjecture; Gilbert–Lynch 2002)

In an asynchronous network in which messages between nodes may be lost (that is: partitions can occur), no read/write register implementation can guarantee both

- *availability* — every request received by a non-failed node eventually receives a response, and
- *atomic consistency* (linearizability) — all operations appear to execute in a single total order consistent with real time

in all executions, including those containing partitions.

The folklore version says "consistency, availability, partition tolerance: pick two of three," as if the three were interchangeable menu items. Two corrections:

First, partition tolerance is not optional. A partition is not a feature you enable; it is a thing the physical network does to you. Cables are cut, switches reboot, datacenters lose their uplinks. A system advertised as "CA" is simply a system whose behavior under partition is undefined — which is a specification bug, not a third corner of a design space. The real theorem offers a choice of *two*, made per request, at the moment a partition exists: refuse to answer (keeping consistency) or answer from local state (keeping availability).

Second, consistency is a spectrum, not a bit. The theorem's "C" is linearizability, the strongest practical consistency notion, and its "A" is a strong availability notion too. Between "linearizable" and "anything goes" lies a whole ladder of guarantees — causal

consistency, read-your-writes, monotonic reads, eventual convergence — most of which are *not* forbidden to an available system. The theorem closes one door. It does not close the corridor.

This book walks the corridor to its end. We choose availability — every replica accepts every update locally, always, partition or no partition — and then ask for the strongest guarantee still purchasable without coordination. That guarantee has a name.

1.5 Strong Eventual Consistency, Informally

Eventual consistency is the weak promise you have probably met: if updates stop arriving, replicas will *eventually* agree — where "eventually" may involve conflict tickets, rollbacks, or a human being. It is a liveness property with no opinion about what the agreed state is or how the agreement happens.

Strong eventual consistency (SEC) is sharper. It demands:

- **Eventual delivery** — an update delivered at one correct replica is eventually delivered at all correct replicas; and
- **Convergence** — replicas that have delivered the *same set* of updates are in the *same state*. Not eventually: immediately, deterministically, with no conflict resolution left to do and nothing to roll back.

Notice what the second clause quietly quantifies over. The same *set* of updates — in any order, delivered any positive number of times. Convergence must hold even though the network reorders and duplicates. That is a strong algebraic constraint on the merge operation, and pinning down exactly which algebra it forces is the work of Part III, where SEC stops being a slogan and becomes a Lean theorem (Chapter 8) with an operational companion (Chapter 9).

The data types that achieve SEC without coordination are the *conflict-free replicated data types* — CRDTs — and building a zoo of them, correctly and with proofs, is Part II.

First, the promised funeral for overwrite.

1.6 Computation Interlude: Watching an Update Disappear

Everything in this chapter's demonstration fits in a few lines of Lean. The state of a naive replicated counter is just its count, and the naive sync is just projection onto the other argument. This code lives in `Crdt.Intro` in the companion file.

```
/-- A naive replicated counter: the state is the count. -/
abbrev NaiveCounter := Nat

```

```
def incr (c : NaiveCounter) : NaiveCounter := c + 1

/-- The "obvious" sync strategy: take the other replica's value.
    (Overwrite; a.k.a. naive last-write-wins.) -/
def overwrite (_mine theirs : NaiveCounter) : NaiveCounter := theirs

/-- Two replicas, three increments, one sync. The truth is 3. -/
def divergenceDemo : List (String × Nat) :=
  let a0 : NaiveCounter := 0
  let b0 : NaiveCounter := 0
  let a1 := incr (incr a0)      -- A observes two increments  → 2
  let b1 := incr b0             -- B observes one increment   → 1
  let bSynced := overwrite b1 a1  -- B pulls from A: B's increment is gone
  let aSynced := overwrite a1 bSynced
  [("replica A", a1), ("replica B", b1),
   ("B after sync", bSynced), ("A after sync", aSynced),
   ("ground truth", 3)]

#eval divergenceDemo
```

```
[("replica A", 2), ("replica B", 1), ("B after sync", 2), ("A after sync", 2),
 ↪  ("ground truth", 3)]
```

Both replicas finish at 2; three increments happened. The lost update, reproduced on demand.

In this series a broken design does not merely get a cautionary paragraph; it gets a machine-checked counterexample in a red box. Here is the first.

Refuted 1.1 — Last write wins is a merge strategy for free

Claim. Overwriting local state with the incoming state is an acceptable way to reconcile replicas; at worst the replicas briefly disagree.

Counterexample. The interleaving above: after synchronization, both replicas hold 2 although three increments occurred — an update is silently destroyed, and no later sync can recover it. Worse, overwrite is not even commutative, so two replicas that sync *toward each other* at the same moment end up swapped, not agreed. Both facts are checked by `decide`:

```
/-- Refuted: "last write wins is a merge strategy for free".
    The interleaving above loses B's increment: both replicas end
    at 2 although three increments happened. -/
theorem overwrite_loses_an_update :
    (overwrite (incr 0) (incr (incr 0))) ≠ 3 := by decide

/-- Refuted (Exercise 1.3): overwrite-merge is not even commutative,
    so replicas that sync in different directions disagree. -/
```

```
theorem overwrite_not_comm :
    ¬ (∀ a b : NaiveCounter, overwrite a b = overwrite b a) :=
  fun h => absurd (h 1 2) (by decide)
```

Moral. A reconciliation strategy must *remember*, not choose. What "remembering" means, precisely, is Chapter 2.

Is real last-write-wins this naive? No — production LWW attaches timestamps to writes and keeps the newer one, which at least makes the outcome deterministic. It still discards one of two concurrent writes, a behavior we will engineer deliberately and lawfully in Chapter 6, and whose costs Chapter 13 tallies. What is refuted here is the free lunch: reconciliation-by-overwrite with no further structure.

1.7 Exercises

Exercise 1.1 — Interleavings

★ Two replicas each perform one increment, and each then syncs once from the other (by overwrite), in some order. Enumerate all interleavings of these four events. What final (A, B) value pairs can result? Which interleavings lose an update, and how many end with the replicas disagreeing?

Exercise 1.2 — CAP corners

★ For each of the following systems, decide which side of the CAP trade-off it takes during a partition, and what its reconciliation story is afterwards: (a) `git`, when two clones commit independently; (b) a phone calendar syncing with a server; (c) a bank ledger settling transfers between accounts. One of these genuinely cannot take the available branch — which, and why? (Revisit your answer after Chapter 13.)

Exercise 1.3 — Overwrite is not commutative

★★ Prove in Lean, by `decide` on a concrete counterexample, that `overwrite` on `Nat` is not commutative — both as an `example` for one specific pair and as the negated universal `theorem` above. Then explain in one sentence why a *commutative* sync function is necessary (though, as Chapter 8 will show, not yet sufficient) for replicas that sync concurrently to agree.

Chapter 2

The Merge Discipline

> *"A cell does not hold a value; it holds what is known so far — and it stands ready, at any moment, to be told more."*
> — *after Radul & Sussman,* The Art of the Propagator *(2009)*

Chapter 1 ended with a moral: reconciliation must remember, not choose. The predecessor book spent three hundred pages, in effect, on what "remembering" means inside one process. This chapter restates that vocabulary as a discipline for replicas — and discovers that nothing new is needed except a change of postal service.

2.1 The Cell, Remembered

A replica does not *store a value*; it stores *everything it has heard so far* about a value. That sentence should sound familiar — it is the propagator cell's manifesto from the previous book, word for word. The only thing that has changed since the last book is the postal service. Inside one process, a cell's inputs arrived on a scheduler we controlled; between replicas, updates arrive late, twice, or in the wrong order, and no scheduler in the world can promise otherwise. The remarkable fact — the fact this book exists to prove — is that the *same* algebraic discipline that made propagator networks converge makes replicas agree.

Recall the discipline's three commitments, exactly as the predecessor made them:

- state lives in a partially ordered set, where $s \sqsubseteq t$ reads "t knows at least as much as s";
- the order is a *bounded join-semilattice*: any two states a, b have a least upper bound $a \sqcup b$ (the most conservative state containing both), and there is a least state $\bot$ ("knows nothing");
- writing is merging: a cell receiving v moves from c to $c \sqcup v$. Information only ever increases.

Figure 2.1 is the predecessor's cell-evolution picture with one new row.

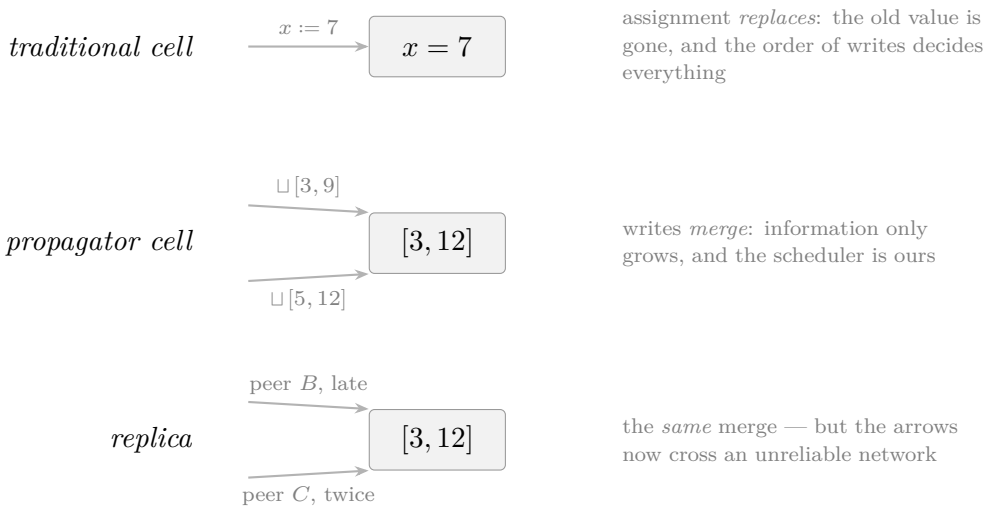

Figure 2.1: One discipline, three postal services. The traditional cell forgets. The propagator cell accumulates, fed by a scheduler under our control. The replica is the same accumulating cell, fed by a network under nobody's control.

The rest of this chapter re-derives the machinery — classes, notation, algebra — from scratch, as the series rules demand. If you worked through the predecessor, read the code as an old friend with a new job title.

2.2 The Classes, Re-derived

We re-derive rather than import: the companion file `Crdt.lean` depends on nothing but the Lean prelude. The classes are character-for-character the predecessor's, living in the namespace `Crdt.Order`, so your muscle memory transfers intact.

```
namespace Order

/-- A partial order: reflexive, antisymmetric, transitive.
    (Identical to the predecessor's class, so muscle memory transfers.) -/
class PartialOrder (α : Type) [LE α] where
  le_refl              : ∀ (a : α), a ≤ a
  le_antisymm {a b : α} : a ≤ b → b ≤ a → a = b
  le_trans {a b c : α}  : a ≤ b → b ≤ c → a ≤ c

/-- A bounded join-semilattice: every pair has a least upper bound `sup`,
    and there is a least element `bot`. The state space of every
    state-based CRDT in this book is one of these. -/
class BoundedJoinSemilattice (α : Type) extends LE α, PartialOrder α where
  sup : α → α → α
  bot : α
  le_sup_left   : ∀ (a b : α), a ≤ sup a b
  le_sup_right  : ∀ (a b : α), b ≤ sup a b
```

```
  sup_le        : ∀ (a b c : α), a ≤ c → b ≤ c → sup a b ≤ c
  bot_le        : ∀ (a : α), bot ≤ a

end Order
```

The notation is also the predecessor's, plus one addition: we write the order itself as `⊑` when we mean it as the *information* order, to keep it visually distinct from the arithmetic `≤` on numbers that will soon share the page.

```
infixl:65 " ⊔ " => Order.BoundedJoinSemilattice.sup
notation  "⊥"   => Order.BoundedJoinSemilattice.bot
/-- The information order. `a ⊑ b` is notation for `a ≤ b`, read
    "b knows at least as much as a". -/
infix:50  " ⊑ "  => LE.le
```

2.3 The ACI Toolkit

Three algebraic laws of $\sqcup$ do all the work in this book, so we prove them once, name them, and keep them within reach. They are *commutativity*, *associativity*, and *idempotence* — ACI for short — and each one is the antidote to exactly one thing a network does to messages.

network sin	what the replica computes	absorbed by
reordering	$b \sqcup a$ instead of $a \sqcup b$	commutativity (with associativity)
duplication	$a \sqcup a$ where a sufficed	idempotence
batching	$(a \sqcup b)$ delivered as one blob	associativity

Table 2.1: One law per network sin. This table is a promissory note; Chapter 8 pays it in full, including the proof that dropping any row breaks convergence.

Each law follows from the semilattice axioms by the same move the predecessor used everywhere: to prove two elements equal, squeeze them with antisymmetry.

```
theorem sup_comm (a b : α) : a ⊔ b = b ⊔ a := by
  apply le_antisymm
  · exact sup_le a b (b ⊔ a) (le_sup_right b a) (le_sup_left b a)
  · exact sup_le b a (a ⊔ b) (le_sup_right a b) (le_sup_left a b)

theorem sup_assoc (a b c : α) : a ⊔ b ⊔ c = a ⊔ (b ⊔ c) := by
  apply le_antisymm
  · apply sup_le
    · apply sup_le
      · exact le_sup_left a (b ⊔ c)
```

```
      · exact le_trans (le_sup_left b c) (le_sup_right a (b ⊔ c))
    · exact le_trans (le_sup_right b c) (le_sup_right a (b ⊔ c))
  · apply sup_le
    · exact le_trans (le_sup_left a b) (le_sup_left (a ⊔ b) c)
    · apply sup_le
      · exact le_trans (le_sup_right a b) (le_sup_left (a ⊔ b) c)
      · exact le_sup_right (a ⊔ b) c

theorem sup_idem (a : α) : a ⊔ a = a := by
  apply le_antisymm
  · exact sup_le a a a (le_refl a) (le_refl a)
  · exact le_sup_left a a
```

Two more small results round out the toolkit. Bottom is the identity of merge — a replica that knows nothing contributes nothing:

```
theorem sup_bot_left (a : α) : ⊥ ⊔ a = a := by
  apply le_antisymm
  · exact sup_le ⊥ a a (bot_le a) (le_refl a)
  · exact le_sup_right ⊥ a

theorem sup_bot_right (a : α) : a ⊔ ⊥ = a := by
  rw [sup_comm]; exact sup_bot_left a
```

And the order is recoverable from the join alone. This innocuous equivalence will be quietly load-bearing for the rest of the book: it says that "b already knows everything a knows" and "merging a into b changes nothing" are the same statement.

```
/-- The order is recoverable from the join: `a ⊑ b` exactly when
    merging `a` into `b` tells `b` nothing new. -/
theorem le_iff_sup_eq {a b : α} : a ⊑ b ↔ a ⊔ b = b := by
  constructor
  · intro h
    apply le_antisymm
    · exact sup_le a b b h (le_refl b)
    · exact le_sup_right a b
  · intro h
    rw [← h]
    exact le_sup_left a b

theorem sup_mono {a b c d : α} (h₁ : a ⊑ c) (h₂ : b ⊑ d) :
    a ⊔ b ⊑ c ⊔ d :=
  sup_le a b (c ⊔ d)
    (le_trans h₁ (le_sup_left c d))
    (le_trans h₂ (le_sup_right c d))
```

Proving `sup_idem` and `le_iff_sup_eq` yourself, from the axioms alone, is the warm-up this chapter owes you (Exercises 2.1 and 2.2).

2.4 Updates Are Inflationary

The predecessor demanded that *propagators* be monotone maps between lattices. Replicas need a related but simpler property of their *update* operations: an update may add information to the local state, never retract it. Applied to a state s, an update must land at or above s.

```
/-- An update function may only move a replica's state upward:
    it can add information, never retract it. -/
def Inflationary (f : α → α) : Prop := ∀ s, s ⊑ f s

/-- Merging any value in is an inflationary update — the propagator
    cell's `write` and the replica's `merge` in one lemma. -/
theorem sup_inflationary_left (v : α) : Inflationary (fun s => s ⊔ v) :=
  fun s => le_sup_left s v
```

The one-line theorem is worth a pause: it says *merge itself is an update*. Receiving a peer's state and receiving a locally generated update are the same kind of event, handled by the same $\sqcup$. This uniformity is why the zoo of Part II needs so few moving parts.

With the vocabulary assembled, the central definition of the book can be stated. Notice how little of it is new.

Definition — State-based CRDT

A *state-based conflict-free replicated data type* consists of

- a state space $(\sigma, \sqsubseteq, \sqcup, \bot)$ forming a bounded join-semilattice;
- *update* operations $u : \sigma \to \sigma$, each inflationary ($s \sqsubseteq u(s)$ for all s);
- *query* functions $q : \sigma \to \beta$, entirely unconstrained.

Each replica holds a state, applies updates locally, and reconciles with a peer by replacing its state with the join of the two states: $\mathit{merge}(s, t) = s \sqcup t$.

Read the third clause again: queries are *unconstrained*. Nothing requires a query to be monotone in the state, and Chapter 4 exhibits a respectable CRDT whose most important query goes *down* while the state goes up. The lattice disciplines what a replica *remembers*, not what it *reports* — keep the two ideas separate now and Part II will hold no confusions.

2.5 Composition Is Free: the Product Semilattice

One construction from the predecessor's exercises gets promoted to the main text, because four later chapters lean on it and exercises are not allowed to be load-bearing (the predecessor taught us that the hard way). If α and β are bounded join-semilattices, so is $\alpha \times \beta$, ordered and merged componentwise:

```
instance {α β : Type} [LE α] [LE β] : LE (α × β) where
  le p q := p.1 ≤ q.1 ∧ p.2 ≤ q.2

instance {α β : Type} [LE α] [LE β] [PartialOrder α] [PartialOrder β] :
    PartialOrder (α × β) where
  le_refl p := ⟨le_refl p.1, le_refl p.2⟩
  le_antisymm {p q} h₁ h₂ := by
    cases p; cases q
    have e₁ := le_antisymm h₁.1 h₂.1
    have e₂ := le_antisymm h₁.2 h₂.2
    simp_all
  le_trans h₁ h₂ := ⟨le_trans h₁.1 h₂.1, le_trans h₁.2 h₂.2⟩

instance {α β : Type}
    [BoundedJoinSemilattice α] [BoundedJoinSemilattice β] :
    BoundedJoinSemilattice (α × β) where
  sup p q := (p.1 ⊔ q.1, p.2 ⊔ q.2)
  bot := (⊥, ⊥)
  le_sup_left p q := ⟨le_sup_left p.1 q.1, le_sup_left p.2 q.2⟩
  le_sup_right p q := ⟨le_sup_right p.1 q.1, le_sup_right p.2 q.2⟩
  sup_le p q r h₁ h₂ :=
    ⟨sup_le p.1 q.1 r.1 h₁.1 h₂.1, sup_le p.2 q.2 r.2 h₁.2 h₂.2⟩
  bot_le p := ⟨bot_le p.1, bot_le p.2⟩
```

Every proof is the corresponding axiom applied to each coordinate; there is no idea here beyond "pairs of lattices are lattices of pairs." That very blandness is the point. When the PN-Counter (Chapter 4), the 2P-Set (Chapter 5), the OR-Set (Chapter 7) and the capstone store (Chapter 12) each need a composite state space, the semilattice instance — and with it, as Part III will show, the entire convergence guarantee — comes free of charge. Building the instance yourself, before peeking, is Exercise 2.3.

Where's Mathlib? Nowhere, as usual for this series. Mathlib has all of this — `SemilatticeSup`, `OrderBot`, `Prod` instances — under slightly different names and vastly greater generality. We rebuild it in about a page so that every axiom in the convergence theorem of Chapter 8 is one we wrote ourselves.

2.6 Computation Interlude: The Same Code, Twice

The predecessor's runtime pieces return unchanged: mutable `Cell`s whose `write` is a join, and the fuel-free fixpoint loop. (We will have sharp words about `runToFixpoint`'s termination story in Chapter 9 — one of this book's jobs is to pay off that IOU.)

```
structure Cell (α : Type) where
  ref : IO.Ref α

def Cell.new {α} [BoundedJoinSemilattice α] : IO (Cell α) := do
  let r ← IO.mkRef (⊥ : α)
  return { ref := r }

def Cell.read {α} (c : Cell α) : IO α :=
  c.ref.get

/-- Writing is merging: `c ← c ⊔ v`. Returns whether anything changed. -/
def Cell.write {α} [BoundedJoinSemilattice α] [DecidableEq α]
    (c : Cell α) (v : α) : IO Bool := do
  let old ← c.ref.get
  let merged := old ⊔ v
  if merged = old then
    return false
  else
    c.ref.set merged
    return true

abbrev PropStep := IO Bool

def runToFixpoint (steps : Array PropStep) : IO Unit := do
  let mut changed := true
  while changed do
    changed := false
    for step in steps do
      let c ← step
      if c then changed := true
```

For a working lattice we also re-derive the predecessor's flat lattice `FlatNat` — *unknown* $\sqsubseteq$ *known* n $\sqsubseteq$ *conflict* — whose merge agrees on matching observations and reports a conflict otherwise:

```
inductive FlatNat where
  | unknown         : FlatNat
  | known (n : Nat) : FlatNat
  | conflict        : FlatNat
  deriving Repr, DecidableEq
```

```

def FlatNat.le (a b : FlatNat) : Prop :=
  match a, b with
    | .unknown , _          => True
    | _        , .conflict => True
    | .known m , .known n  => m = n
    | .conflict, .unknown  => False
    | .conflict, .known _  => False
    | .known _ , .unknown  => False

instance : LE FlatNat := ⟨FlatNat.le⟩
```

The `PartialOrder` and `BoundedJoinSemilattice` instances (with `sup` matching on constructors and `bot := .unknown`) are the predecessor's proofs, reproduced verbatim in the companion file; we do not reprint the case bashes here.

Now the punchline of the chapter, in two runs. First, the code doing its *old* job — a propagator network computing a sum inside one process:

```
/-- The predecessor's example, replayed: a propagator network
    computing a sum inside one process. -/
def propagatorReplay : IO Unit := do
  let cx ← Cell.new (α := FlatNat)
  let cy ← Cell.new (α := FlatNat)
  let cs ← Cell.new (α := FlatNat)
  let _ ← cx.write (.known 3)
  let _ ← cy.write (.known 7)
  runToFixpoint #[addProp cx cy cs]
  IO.println s!"sum is {← cs.read}"

#eval propagatorReplay
```

```
sum is 10
```

And then, with not one new definition, the code doing its *new* job. Two cells, reread as two replicas of a single value. One replica hears an observation; then the two gossip — sloppily, with a duplicated delivery thrown in, because idempotence makes duplicates harmless:

```
/-- The same code, new reading: two *replicas* of one FlatNat cell.
    Each hears a (consistent) observation, then each merges the
    other's state — in different orders, twice. They agree. -/
def replicaReplay : IO Unit := do
  let a ← Cell.new (α := FlatNat)   -- replica A
  let b ← Cell.new (α := FlatNat)   -- replica B
  let _ ← a.write (.known 42)       -- A hears the value
```

```
  -- gossip, sloppily: B pulls from A, A pulls from B, B pulls again
  let _ ← b.write (← a.read)
  let _ ← a.write (← b.read)
  let _ ← b.write (← a.read)        -- duplicate delivery: harmless
  IO.println s!"replica A holds {← a.read}, replica B holds {← b.read}"

#eval replicaReplay
```

```
replica A holds 42, replica B holds 42
```

Same lattice, same `write`, same everything. The propagator cell was a replica all along; it just never had to leave the building. The next five chapters build the state spaces that make this trick do real work — counters, sets, registers — each one a bounded join-semilattice with a story, a wrong design, and a proof.

2.7 Exercises

Exercise 2.1 — Idempotence from the axioms

★ Prove `sup_idem : a ⊔ a = a` in Lean using only the six semilattice axioms and antisymmetry — without looking at the proof in this chapter. (It is three lemma applications and one `le_antisymm`.)

Exercise 2.2 — The order from the join

★ Prove `le_iff_sup_eq : a ⊑ b ↔ a ⊔ b = b`. State, in one English sentence each, what the two directions mean operationally for a replica receiving a peer's state.

Exercise 2.3 — The product semilattice

★★ Without looking at the instance in this chapter, define `BoundedJoinSemilattice (α × β)` from instances on the components, and prove all six laws. (The construction is restated in full in the main text precisely because later chapters depend on it — exercises are not allowed to be load-bearing — but you should build it once with your own hands. It is used by Chapters 4, 5, 7 and 12.)

Part II

The Replica Zoo

Part I ended with a contract: keep every replica's state in a bounded join-semilattice, make every update inflationary, and merge by join. Part II is where the contract earns its keep. We build the classic replicated data types — counters, sets, registers — one chapter each, and every one of them is a Lean structure with a verified `BoundedJoinSemilattice` instance. None of them is designed from inspiration. Each is *derived*, by watching the obvious design fail in a machine-checked red box and asking what the failure demands.

Chapter 3

Counting Without Coordination: the G-Counter

> *"The simplest question in distributed computing* — how many? — *already contains the whole subject."*

Every chapter in this part follows the same route, and it is worth stating the route once, out loud, so you know the drill:

1. an informal story — what the users want;
2. the wrong design, and a *refutation*: a red box with a machine-checked counterexample showing exactly how it fails;
3. the right design, forced by the failure;
4. the semilattice instance — this is the proof of convergence;
5. the proof that every update is inflationary;
6. a Computation Interlude, where `#eval` shows replicas agreeing under reordered and duplicated merges;
7. exercises.

The refutations are not warm-up entertainment. They are the design method. By the end of this chapter you will not merely believe that a distributed counter needs one slot per replica — you will have watched `decide` reject both alternatives.

3.1 Two Wrong Counters

The story: a like button. Users on different continents tap it; each tap lands on whichever replica is nearest; replicas exchange state when the network permits; everyone should eventually see the same total. No replica may wait for another before accepting a tap — that is the whole point of having replicas.

It is tempting to model the counter as a natural number and merge two copies by taking the larger. The temptation survives about one experiment. Let replicas A and B each start at 0 and each observe one increment; both now hold 1, and their merge holds $\max(1, 1) = 1$. Two events happened; the counter remembers one.

Refuted 3.1 — A distributed counter is a `Nat` merged by `max`

Both replicas increment once from 0; the merge undercounts.

```
theorem max_undercounts : Nat.max (0 + 1) (0 + 1) ≠ 2 := by decide
```

The failure is not a bug in `max` — `max` is a perfectly good join. The failure is that we joined the *wrong lattice*: two distinct events were stored in states that the order cannot tell apart.

Very well: if max forgets one of the two increments, add them. Merging by + makes the totals come out right in that experiment — and hands the network a new way to hurt us. Addition is not idempotent. The moment a state is delivered twice (and Chapter 1 was clear that it will be), the count jumps.

Refuted 3.2 — Then merge by addition

A replica holding 1 receives its own state again and "merges" it in.

```
theorem add_overcounts : 1 + 1 ≠ (1 : Nat) := by decide
```

Idempotence — $a \sqcup a = a$ — is precisely the law that makes duplicated delivery harmless, and + does not have it.

Read the two failures together and they dictate the design. The merge must be idempotent, so within any one coordinate it must behave like max, not like +. But max conflates concurrent increments, so concurrent increments must land in *different* coordinates. There is exactly one actor who can be trusted to own a coordinate without coordination: the replica itself.

Keep one tally per replica; merge tallies pointwise by max.

That is the G-Counter ("grow-only counter"), and we did not invent it — the refutations did.

3.2 An Indexed Family of Tallies

Fix a number of replicas R. A G-Counter state assigns a tally to each replica identifier, and replica identifiers are exactly the elements of `Fin R` — the type of natural numbers below `R`. So the carrier is a *function type indexed by* `Fin R`:

```
/-- The G-Counter: remember one tally *per replica*.
    An indexed family — your first dependent-indexed carrier. -/
def GCounter (R : Nat) : Type := Fin R → Nat
```

This is the first time in the series that a carrier is a function type rather than an inductive one, and it is worth pausing on what that buys and what it costs. It buys exactly the shape of the problem: a state *is* the assignment $i \mapsto$ tally of i, no encod-

ing, no constructor to unwrap. It costs one new proof move — to show two states equal we must show them equal *at every index*, and for that Lean gives us function extensionality.

The order is pointwise: one state is below another when every tally is.

```
namespace GCounter

open Order.PartialOrder Order.BoundedJoinSemilattice

variable {R : Nat}

instance : LE (GCounter R) := ⟨fun g h => ∀ i, g i ≤ h i⟩

instance : Order.PartialOrder (GCounter R) where
  le_refl g i        := Nat.le_refl (g i)
  le_antisymm h₁ h₂  := funext fun i => Nat.le_antisymm (h₁ i) (h₂ i)
  le_trans h₁ h₂ i   := Nat.le_trans (h₁ i) (h₂ i)
```

`funext`, first appearance. The axiom-backed theorem `funext : (∀ x, f x = g x) → f = g` converts pointwise equality of functions into equality of the functions themselves. Antisymmetry hands us `h₁ i : g i ≤ h i` and `h₂ i : h i ≤ g i` at every index; `Nat.le_antisymm` turns each pair into `g i = h i`; and `funext` assembles the indexed family of equalities into `g = h`. In Lean 4, `funext` is derived from quotient types — which is why, when Chapter 8 audits axioms, theorems that touch it report `Quot.sound`. We will meet quotients face to face there; for now, `funext` is simply the honest way to say "same tallies, same state."

The join is pointwise max, and the bottom — the state of a replica that has heard nothing — is zero everywhere. Every field of the semilattice instance reduces to a fact about `Nat` at one index:

```
instance : BoundedJoinSemilattice (GCounter R) where
  sup g h            := fun i => Nat.max (g i) (h i)
  bot                := fun _ => 0
  le_sup_left g h i  := Nat.le_max_left (g i) (h i)
  le_sup_right g h i := Nat.le_max_right (g i) (h i)
  sup_le _ _ _ h₁ h₂ := fun i => Nat.max_le.mpr ⟨h₁ i, h₂ i⟩
  bot_le g i         := Nat.zero_le (g i)
```

Notice how little there is here. Chapter 2 promised that a semilattice instance *is* the convergence proof for a state-based CRDT, and this one is six lines of pointwise delegation to the standard library. The refutations were the hard part; the mathematics, once the lattice is right, is bookkeeping.

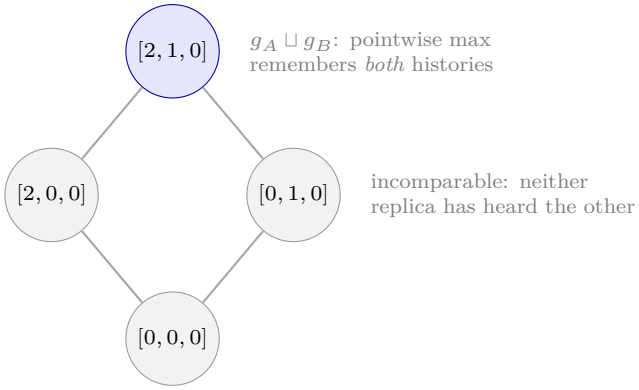

Figure 3.1: Two `GCounter 3` states after A increments twice and B increments once, drawn in the pointwise order. The states are incomparable — each knows something the other does not — and their join is the least state that knows both.

3.3 Updates and Queries

An increment at replica `i` bumps exactly slot `i`. The query — the number users actually see — sums the slots. And for the interludes we render a state as the list of its tallies:

```
/-- Replica `i` increments: bump exactly your own slot. -/
def increment (i : Fin R) (g : GCounter R) : GCounter R :=
  fun j => if j = i then g j + 1 else g j

/-- The query: total of all per-replica tallies. -/
def value (g : GCounter R) : Nat :=
  (List.finRange R).foldl (fun acc i => acc + g i) 0

/-- Render the state for the interludes. -/
def toList (g : GCounter R) : List Nat :=
  (List.finRange R).map g
```

Here `List.finRange R` is the standard library's list `[0, 1, ..., R-1]` of all elements of `Fin R` — the bridge between the indexed family and anything that wants to iterate over it.

The merge discipline demands that `increment` be inflationary, and it is, by a one-line case split: the bumped slot grows, the others stand still. It is also monotone, which we record because Chapter 11 will want it:

```
theorem increment_inflationary (i : Fin R) :
    Order.Inflationary (increment i) := by
  intro g j
  by_cases h : j = i <;> simp [increment, h]

theorem increment_monotone {g h : GCounter R} (i : Fin R) (hle : g ⊑ h) :
```

```
    increment i g ⊑ increment i h := by
  intro j
  by_cases hj : j = i <;> simp [increment, hj] <;> exact hle _

/-- Merging can only help: an increment survives any merge. -/
theorem increment_le_sup (i : Fin R) (g h : GCounter R) :
    increment i g ⊑ increment i g ⊔ h :=
  le_sup_left (increment i g) h
```

3.4 The Counter Counts

A counter that converges but reports nonsense would be no counter at all. The G-Counter's headline correctness property is exact: every increment, performed by any replica, raises the value by precisely one.

The proof is our first serious encounter with reasoning about folds, and it teaches a technique this book will use in Chapters 8, 9 and 11: *generalize the accumulator*. A statement about `foldl f 0 l` is usually unprovable by induction as stated, because after one step the accumulator is no longer `0`. State the lemma for an arbitrary seed `a` and the induction goes through. Three seed-generalized helpers do all the work — pulling a `+ 1` out of a fold, replacing the summand by a pointwise-equal one, and bounding a fold by a pointwise-larger one:

```
theorem foldl_add_shift {ι : Type} (f : ι → Nat) :
    ∀ (l : List ι) (a : Nat),
      l.foldl (fun acc j => acc + f j) (a + 1)
        = l.foldl (fun acc j => acc + f j) a + 1
  | [], _ => rfl
  | j :: t, a => by
      rw [List.foldl_cons, List.foldl_cons, Nat.add_right_comm]
      exact foldl_add_shift f t (a + f j)

theorem foldl_add_congr {ι : Type} {f g : ι → Nat} :
    ∀ (l : List ι), (∀ j ∈ l, f j = g j) → ∀ (a : Nat),
      l.foldl (fun acc j => acc + f j) a
        = l.foldl (fun acc j => acc + g j) a
  | [], _, _ => rfl
  | j :: t, h, a => by
      rw [List.foldl_cons, List.foldl_cons, h j List.mem_cons_self]
      exact foldl_add_congr t (fun k hk => h k (List.mem_cons_of_mem _ hk)) _

theorem foldl_add_le {ι : Type} {f g : ι → Nat} (h : ∀ j, f j ≤ g j) :
    ∀ (l : List ι) {a b : Nat}, a ≤ b →
      l.foldl (fun acc j => acc + f j) a ≤ l.foldl (fun acc j => acc + g j) b
  | [], _, _, hab => hab
```

```
  | j :: t, a, b, hab => by
      rw [List.foldl_cons, List.foldl_cons]
      exact foldl_add_le h t (Nat.add_le_add hab (h j))
```

The heart of the argument is the *bump lemma*: if two summands agree everywhere except at one index, where they differ by exactly one, then their sums over all of `Fin n` differ by exactly one. The natural temptation is to hunt for the index inside `finRange n` and split the list around it; the clean proof avoids list surgery entirely and inducts on `n` itself, using the library equation `finRange_succ`, which rewrites `finRange (n+1)` to `0 :: (finRange n).map Fin.succ`. Either the bumped index is `0` — then the seeds differ by one and `foldl_add_shift` carries the difference out — or it lives in the mapped tail, and the induction hypothesis applies one `Fin.succ` down:

```
/-- Bump one slot of the summand and the sum grows by exactly one.
    Proved by induction on `R` via `finRange_succ` — no Nodup needed. -/
theorem foldl_add_bump (n : Nat) :
    ∀ (f g : Fin n → Nat) (i : Fin n),
      (∀ j, j ≠ i → f j = g j) → f i = g i + 1 → ∀ (a : Nat),
      (List.finRange n).foldl (fun acc j => acc + f j) a
        = (List.finRange n).foldl (fun acc j => acc + g j) a + 1 := by
  induction n with
  | zero => intro _ _ i; exact i.elim0
  | succ n ih =>
      intro f g i hne hbump a
      rw [List.finRange_succ, List.foldl_cons, List.foldl_cons,
          List.foldl_map, List.foldl_map]
      cases i using Fin.cases with
      | zero =>
          rw [hbump, ← Nat.add_assoc]
          calc (List.finRange n).foldl (fun acc j => acc + f j.succ) (a + g 0
           ↪  + 1)
              = (List.finRange n).foldl (fun acc j => acc + f j.succ) (a + g
               ↪  0) + 1 :=
                foldl_add_shift _ _ _
            _ = (List.finRange n).foldl (fun acc j => acc + g j.succ) (a + g
             ↪  0) + 1 := by
                rw [foldl_add_congr (List.finRange n)
                      (fun j _ => hne j.succ (Fin.succ_ne_zero j))]
      | succ i' =>
          rw [hne 0 (Ne.symm (Fin.succ_ne_zero i'))]
          exact ih (fun (j : Fin n) => f j.succ) (fun (j : Fin n) => g j.succ)
           ↪  i'
            (fun j hj => hne j.succ (fun hc => hj (Fin.succ_inj.mp hc)))
            hbump (a + g 0)
```

The headline theorem is now an instantiation — `increment i g` and `g` agree away from

`i` and differ by one at it:

```
/-- The G-Counter counts: every increment raises the value by exactly one,
    no matter which replica performed it. -/
theorem value_increment (i : Fin R) (g : GCounter R) :
    value (increment i g) = value g + 1 :=
  foldl_add_bump R (increment i g) g i
    (fun j hj => by simp [increment, hj])
    (by simp [increment]) 0
```

One more fact, recorded with a warning label attached:

```
/-- The G-Counter's query happens to be monotone.
    (Chapter 4 shows this is a coincidence, not a law.) -/
theorem value_le_value_of_le {g h : GCounter R} (hle : g ⊑ h) :
    value g ≤ value h :=
  foldl_add_le hle (List.finRange R) (Nat.le_refl 0)
```

It is easy to read this and conclude that queries of CRDTs track the information order — more state, bigger answer. Hold that thought exactly one chapter.

3.5 Computation Interlude: Three Replicas, Any Order, Twice

Three replicas of a `GCounter 3`. Replica A increments twice, B and C once each. Then each replica merges what it has heard — in three different orders, with B's state delivered twice to A and A's twice to B:

```
/-- Interlude: three replicas, out-of-order and duplicated merges. -/
def interlude : List (String × List Nat × Nat) :=
  let gA := increment 0 (increment 0 (⊥ : GCounter 3))  -- A increments twice
  let gB := increment 1 (⊥ : GCounter 3)                 -- B increments once
  let gC := increment 2 (⊥ : GCounter 3)                 -- C increments once
  let atA := (gA ⊔ gB) ⊔ (gC ⊔ gB)   -- B delivered twice
  let atB := gC ⊔ (gA ⊔ (gB ⊔ gA))   -- another order, A delivered twice
  let atC := (gB ⊔ gC) ⊔ gA          -- a third order
  [ ("replica A", toList atA, value atA),
    ("replica B", toList atB, value atB),
    ("replica C", toList atC, value atC) ]

#eval interlude
```

```
[("replica A", [2, 1, 1], 4), ("replica B", [2, 1, 1], 4), ("replica C", [2,
 ↪  1, 1], 4)]
```

Same state, same value, everywhere — with nobody having agreed on an order of events, because no order was needed. Commutativity absorbed the reordering; idempotence absorbed the duplicates. Four increments happened; every replica reports four. Compare Refutations 3.1 and 3.2: this is what they were purchasing.

3.6 Exercises

Exercise 3.1 — Increments commute

★ Prove that increments commute as functions on states: `increment i (increment j g) = increment j (increment i g)` for *all* indices `i`, `j` — distinct or not. (`funext`, then a case split. Solution in Appendix B; it also falls out of a structure built in Chapter 11.)

Exercise 3.2 — An executable list-backed G-Counter

★★ Implementations usually store a plain list of tallies, not a function. Represent a counter as a `List Nat` of length `R` merged by `List.zipWith Nat.max`, and prove it agrees with the functional model: rendering the join is zipping the renders, `toList (g ⊔ h) = List.zipWith Nat.max (toList g) (toList h)`. (Generalize to a lemma about `zipWith` over two `map`s of the same list.)

Exercise 3.3 — The value of a merge

★★★ Prove `value (g ⊔ h) ≤ value g + value h`, and exhibit strictness: concrete states where the inequality is strict, checked by `decide`. (Hint: pointwise, $\max(a, b) \le a + b$; you will want a seed-generalized lemma splitting a fold of pointwise sums into two folds. Hints, not spoilers, in Appendix B.)

Chapter 4

The PN-Counter and the Query/State Split

> *"What a system remembers and what it reports are different budgets, and only one of them is required to grow."*

The like button from Chapter 3 acquired an unlike button, as like buttons do. Users can now decrement. A decrement makes the number smaller, the order on `GCounter` only goes up, and the merge discipline flatly requires updates to be inflationary. It appears the discipline forbids counting down.

It does not. It forbids *forgetting*.

The way out is to notice what a decrement is: not the removal of an increment, but a new event in its own right. Events are information; information accumulates. So we keep two grow-only counters — `P` for increments, `N` for decrements — and let the *query* subtract. The state never goes down. The report does.

4.1 A Pair of G-Counters

Chapter 2 built the product instance — if α and β are bounded join-semilattices then so is $\alpha \times \beta$, ordered and joined componentwise. It is doing real work now, so we restate it rather than lean on memory (in the predecessor this construction was an exercise; exercises cannot be load-bearing, so the companion file carries it in full):

```
instance {α β : Type}
    [BoundedJoinSemilattice α] [BoundedJoinSemilattice β] :
    BoundedJoinSemilattice (α × β) where
  sup p q := (p.1 ⊔ q.1, p.2 ⊔ q.2)
  bot := (⊥, ⊥)
  le_sup_left p q := ⟨le_sup_left p.1 q.1, le_sup_left p.2 q.2⟩
  le_sup_right p q := ⟨le_sup_right p.1 q.1, le_sup_right p.2 q.2⟩
  sup_le p q r h₁ h₂ :=
    ⟨sup_le p.1 q.1 r.1 h₁.1 h₂.1, sup_le p.2 q.2 r.2 h₁.2 h₂.2⟩
  bot_le p := ⟨bot_le p.1, bot_le p.2⟩
```

The PN-Counter is then one line of type definition and no new proofs at all — the

instance for the pair is found by instance search:

```
/-- Two G-Counters: increments land in `P` (first), decrements in `N`
    (second). State only ever grows; the *query* subtracts. -/
abbrev PNCounter (R : Nat) := GCounter R × GCounter R
```

Why `abbrev` and not `def`? An `abbrev` is a transparent alias: Lean unfolds it freely, so instance search sees `PNCounter R` *as* `GCounter R × GCounter R` and finds the product instance without our writing anything. The predecessor's advice — use `structure` for a nominally distinct type when you want to *own* the instances — still stands; here we want the opposite, to *inherit* them.

Updates route to the two components; the query is the difference, taken in `Int` because the truth may be negative:

```
def incr (i : Fin R) (p : PNCounter R) : PNCounter R :=
  (GCounter.increment i p.1, p.2)

def decr (i : Fin R) (p : PNCounter R) : PNCounter R :=
  (p.1, GCounter.increment i p.2)

def value (p : PNCounter R) : Int :=
  (GCounter.value p.1 : Int) - (GCounter.value p.2 : Int)
```

Both updates are inflationary — `decr` included, and that sentence should feel slightly scandalous until it does not:

```
theorem incr_inflationary (i : Fin R) : Order.Inflationary (incr (R := R) i)
  := 
  fun p => ⟨GCounter.increment_inflationary i p.1, le_refl p.2⟩

/-- Even a decrement moves the *state* upward. -/
theorem decr_inflationary (i : Fin R) : Order.Inflationary (decr (R := R) i)
  := 
  fun p => ⟨le_refl p.1, GCounter.increment_inflationary i p.2⟩

/-- A decrement lowers the value while the state strictly grows. -/
theorem value_decr (i : Fin R) (p : PNCounter R) :
    value (decr i p) = value p - 1 := by
  show (GCounter.value p.1 : Int) - (GCounter.value (GCounter.increment i p.2)
    : Int)
      = (GCounter.value p.1 : Int) - (GCounter.value p.2 : Int) - 1
  rw [GCounter.value_increment]
  omega
```

4.2 The Query/State Split

Chapter 3 ended on a loaded observation: the G-Counter's value is monotone in the state. Here is the same claim for the PN-Counter, in a red box, where it belongs.

Refuted 4.1 — The CRDT's value is monotone

Take the empty state and the state after one decrement. The second strictly dominates the first in the information order — it has heard one more event — yet its value is smaller: 0 versus −1.

```
theorem value_not_monotone :
    ¬ (∀ (p q : PNCounter 1), p ⊑ q → value p ≤ value q) := by
  intro h
  have hle : (⊥ : PNCounter 1) ⊑ (⊥, GCounter.increment 0 ⊥) :=
    ⟨bot_le _, bot_le _⟩
  exact absurd (h _ _ hle) (by decide)
```

This refutation dissolves a genuinely common confusion, so let us give its moral a line of its own.

The lattice disciplines what is remembered, not what is reported.

Convergence needs the *state* to live in a semilattice and the updates to climb it. The query is an ordinary function out of the state — a projection, a lens, an opinion. It owes the order nothing. The G-Counter's monotone `value` was a coincidence of its query pointing the same way as its growth; the PN-Counter's query looks at the same ever-growing memory and reports a number that wobbles freely up and down. Both converge, for the same reason, with the same instance supplying the proof.

The same point from another angle: value does not commute with merge in any simple way. It is not the max of the values, nor the min, nor either argument —

```
/-- Refuted: the value of a merge is not the max (nor min, nor either
    argument) of the values — value is a *projection*, not a lattice map. -/
theorem value_merge_ne_max :
    let p : PNCounter 1 := incr 0 ⊥
    let q : PNCounter 1 := decr 0 ⊥
    value (p ⊔ q) ≠ max (value p) (value q) := by decide
```

— because the merge combines *histories*, and the value of a combined history is not a function of the two values alone.

4.3 Computation Interlude: Counting Down, Converging Up

Replica A increments twice. Replica B increments once and then decrements once. The truth is $2 + 1 - 1 = 2$:

```
/-- Interlude: concurrent increments and decrements at two replicas,
    merged both ways. -/
def interlude : List (String × Int) :=
  let atA := incr 0 (incr 0 (⊥ : PNCounter 2))  -- A: +2
  let atB := decr 1 (incr 1 (⊥ : PNCounter 2))  -- B: +1 then −1
  [ ("A alone", value atA),
    ("B alone", value atB),
    ("A ⊔ B", value (atA ⊔ atB)),
    ("B ⊔ A", value (atB ⊔ atA)),
    ("(A ⊔ B) ⊔ B", value ((atA ⊔ atB) ⊔ atB)) ]

#eval interlude
```

```
[("A alone", 2), ("B alone", 0), ("A ⊔ B", 2), ("B ⊔ A", 2), ("(A ⊔ B) ⊔ B",
↪  2)]
```

B alone reports 0 — its increment and decrement cancel. Merged either way around, and with B's state delivered twice for good measure, every replica reports 2. The decrement was never unremembered anywhere; it was *propagated*, like everything else.

4.4 Exercises

Exercise 4.1 — The empty counter

★ Prove `value (⊥ : PNCounter R) = 0` for every `R`. (You will need that a fold of additions of zeros is the seed — a two-line induction. Solution in Appendix B.)

Exercise 4.2 — A clamped query

★★ Define `clampedValue : PNCounter R → Nat` reporting `(value p).toNat`, and exhibit (by `decide`) a state where `clampedValue` reports `0` while `value` is negative. This query *hides* debt rather than preventing it — keep this example in mind; Chapter 13 is about the difference.

Exercise 4.3 — A bounded counter?

★★ Try to design a PN-Counter variant whose value provably never goes below zero while remaining available to every replica. Write down where the attempt

breaks. *Finish after Chapter 13*, which proves a precise version of the obstruction and Exercise 13.2 which shows what coordination-free bounding *is* achievable.

Chapter 5

Sets That Only Grow: G-Set and 2P-Set

> *"A set that can only grow is barely a data structure; it is a ledger. The interesting question is what 'remove' can possibly mean in a world where nothing is ever crossed out."*

Counters aggregate; sets *name*. A replicated set of user identifiers, of seen-message identifiers, of items in a cart — this is the workload most of the replicated-data literature grew up on, and it is where the merge discipline meets its first genuinely awkward customer: removal. This chapter builds the two set designs that stay inside plain set-union merging — the grow-only G-Set and the two-phase 2P-Set — and ends on a refutation that no amount of union can fix. That cliffhanger is Chapter 7's to resolve.

5.1 The Representation Decision

Mathematically, a G-Set is the powerset lattice from the predecessor: states are finite sets, the order is $\subseteq$, the merge is $\cup$, bottom is $\emptyset$. But the predecessor's `Set α := α → Prop` is a type of *predicates* — unprintable, undecidable, no good for Computation Interludes or for `decide`-checked refutations. We need finite sets that *run*.

Our representation: *strictly sorted lists*. Strictness — each element strictly below the next — buys two invariants for the price of one predicate: the list is sorted *and* duplicate-free. And it buys the property everything downstream rests on: **canonicity**. A finite set of elements has exactly one strictly sorted listing, so "same members" will mean "same list" — literal, rewritable, Lean-level equality. Section 5.3 proves precisely that, and the semilattice instance stands on it.

Sorting needs an order on the *elements*, and here the companion file makes a design decision worth a note.

Which element type? The plan of record was to fix elements to `Nat` and sketch the generalization in a note. The companion file does the more honest thing: it hand-rolls a small typeclass, `TotalOrder α`, and builds sorted lists over any instance of it. The reason is concrete, not aesthetic: in Chapter 7 the OR-Set stores *tagged* elements — triples of element, replica, and sequence number — and needs them in sorted lists too. With a typeclass, that chapter writes one lexicographic-product instance and reuses every lemma below unchanged; with elements fixed to `Nat` it would need an injective pairing function and the injectivity proofs would tangle through its hardest theorem. This chapter still *runs* entirely over `Nat` — the one instance we install today.

```
/-- A decidable total order on the element type. `Nat` is the running
    example; lexicographic products arrive in Chapter 6 and pay off in
    Chapter 7. `≼`/`≺` are the *element* order — not to be confused
    with the information order `⊑` on states. -/
class TotalOrder (α : Type) where
  le : α → α → Prop
  deq : DecidableEq α
  decLe : (a b : α) → Decidable (le a b)
  le_refl : ∀ (a : α), le a a
  le_antisymm : ∀ {a b : α}, le a b → le b a → a = b
  le_trans : ∀ {a b c : α}, le a b → le b c → le a c
  le_total : ∀ (a b : α), le a b ∨ le b a

attribute [instance] TotalOrder.deq TotalOrder.decLe

infix:55 " ≼ " => TotalOrder.le

/-- Strictly below. -/
abbrev TotalOrder.lt {α : Type} [TotalOrder α] (a b : α) : Prop :=
  a ≼ b ∧ a ≠ b

infix:55 " ≺ " => TotalOrder.lt
```

Three things distinguish this from the `PartialOrder` of Chapter 2. It is *total* (`le_total`): any two elements compare, which is what makes "the sorted order" unique. It is *decidable* (`decLe`, plus decidable equality `deq`), which is what lets `insertS` below be a program and lets refutations run under `decide`. And it deliberately does *not* reuse the `≤` notation: elements compare with $\preccurlyeq$ and $\prec$, states with $\sqsubseteq$. Keeping the two orders typographically apart is not pedantry — in Chapter 6 a single definition will use both in one line, and in Chapter 10 confusing them would be a category error dressed as a type error.

The bundled `le_antisymm` and `le_total` give a small toolkit of strict-order facts — `lt_trans`, `lt_irrefl`, `lt_of_not_le` (totality means "not below" is "strictly above"),

`not_le_of_lt`, `lt_of_lt_of_le` — proved once in the companion, used silently below. The `Nat` instance is pure delegation:

```
instance : TotalOrder Nat where
  le          := Nat.le
  deq         := inferInstance
  decLe       := Nat.decLe
  le_refl     := Nat.le_refl
  le_antisymm := Nat.le_antisymm
  le_trans    := Nat.le_trans
  le_total    := Nat.le_total
```

5.2 Strictly Sorted Lists

The sortedness predicate is an inductive definition in the house style of the series — one constructor per way of being sorted:

```
/-- Strictly ascending — sorted *and* duplicate-free in one predicate. -/
inductive Sorted : List α → Prop where
  | nil : Sorted []
  | single (a : α) : Sorted [a]
  | cons {a b : α} {l : List α} (hab : a < b) (h : Sorted (b :: l)) :
      Sorted (a :: b :: l)
```

Note the $\prec$: adjacent elements are *strictly* increasing, so `[1, 1, 2]` is not sorted in our sense. Duplicate-freeness is not a second invariant to maintain; it is a corollary of this one.

Structural induction on `Sorted` yields the facts that everything in this chapter runs on: the tail of a sorted list is sorted (`sorted_tail`); the head is strictly below the whole tail (`sorted_head_lt`) and hence is the minimum (`sorted_head_le`); and conversely, putting a fresh minimum in front preserves sortedness (`sorted_cons_of_lt`). Here is the middle one, whose proof shape — induction on the list, generalizing the head — recurs throughout:

```
/-- In a sorted list the head is strictly below everything in the tail. -/
theorem sorted_head_lt {a : α} {l : List α} :
    Sorted (a :: l) → ∀ x ∈ l, a < x := by
  induction l generalizing a with
  | nil => intro _ x hx; cases hx
  | cons b t ih =>
      intro h x hx
      cases h with
      | cons hab hbt =>
          cases List.mem_cons.mp hx with
```

```
          | inl he => subst he; exact hab
          | inr ht => exact lt_trans hab (ih hbt x ht)
```

The workhorse operation is ordered insertion. It walks the list to the right place, and — because our lists are duplicate-free — inserting an element that is already present does nothing:

```
/-- Ordered insert, skipping duplicates. Structural recursion, so every
    counterexample in this chapter runs under `decide`. -/
def insertS (x : α) : List α → List α
  | [] => [x]
  | a :: l =>
      if x = a then a :: l
      else if x ≤ a then x :: a :: l
      else a :: insertS x l
```

Structural recursion is a feature, not a limitation. The kernel can *evaluate* structurally recursive definitions during typechecking, which is exactly what `by decide` does with our refutations: it normalizes both sides inside the proof checker. A well-founded recursion (Chapter 9 has one, on purpose) does not reduce that way. Every operation in this chapter is structural precisely so that claims like Refutation 5.2 can be checked by computation.

Two lemmas characterize `insertS`, and their statements are the whole interface — after them nobody ever unfolds `insertS` again:

```
theorem mem_insertS {x y : α} : ∀ {l : List α},
    x ∈ insertS y l ↔ x = y ∨ x ∈ l

theorem sorted_insertS {y : α} : ∀ {l : List α},
    Sorted l → Sorted (insertS y l)
```

(Both are by structural induction following `insertS`'s three-way split; the companion file has the details, and they are excellent finger exercise.) Union folds insertion, and filtering preserves sortedness because a sublist of a strictly sorted list is strictly sorted — three more statements whose proofs are the same inductions, spelled out in the companion:

```
/-- Union of raw lists: insert everything from `l` into `m`. -/
def unionL (l m : List α) : List α := l.foldr insertS m

theorem mem_unionL {x : α} : ∀ {l m : List α},
    x ∈ unionL l m ↔ x ∈ l ∨ x ∈ m

theorem sorted_unionL {l m : List α} (hm : Sorted m) : Sorted (unionL l m)
```

```
8
9  theorem sorted_filter (p : α → Bool) :
10     ∀ {l : List α}, Sorted l → Sorted (List.filter p l)
```

5.3 The Workhorse: Canonical Representations

Here is the hardest proof of the chapter, and the reason the chapter works at all.

Sorted extensionality

Two strictly sorted lists with the same members are *equal*: if `Sorted l₁`, `Sorted l₂`, and $x \in l_1 \iff x \in l_2$ for all x, then $l_1 = l_2$.

Why it matters: our merge will be `unionL`, and the ACI laws we owe the network are *equations* between lists. Union of sorted lists built by repeated insertion produces its result in an order that depends on the argument order — as computations. As *sets of members* they plainly agree. Canonicity is the bridge: prove the two sides have the same members — which is membership logic, all but automatic — and extensionality upgrades that to the equation we owe. Without this theorem, antisymmetry of the subset order fails and there is no partial order, no semilattice, no CRDT. It is not a lemma; it is the load-bearing wall.

The proof is a double structural induction, and each step uses each ingredient exactly once:

- *Empty against empty* is trivial; empty against `b :: t₂` is impossible, since b would have to be a member of the empty list.
- *Heads are equal.* Each head is a member of the other list (same members), and each head is its own list's *minimum* (`sorted_head_le`). So $a \preccurlyeq b$ and $b \preccurlyeq a$, and antisymmetry of the element order gives $a = b$. This is the only place totality's partner, antisymmetry, is irreplaceable.
- *Tails have the same members.* For x in the tail t_1, strictness gives $a \prec x$, so $x \neq a$; x is a member of $a :: t_2$, and being distinct from the head it is a member of t_2. Strictness is doing the duplicate-freeness work here: it is what lets us peel the head off the membership equivalence *cleanly*, with nothing left behind.
- The induction hypothesis finishes the tails.

```
1  /-- **The workhorse.** Two sorted, duplicate-free lists with the same
2      members are the same list: sortedness makes the representation
3      canonical, and canonicity is what the merge algebra stands on. -/
4  theorem sorted_ext : ∀ {l₁ l₂ : List α}, Sorted l₁ → Sorted l₂ →
5      (∀ x, x ∈ l₁ ↔ x ∈ l₂) → l₁ = l₂
6    | [], [], _, _, _ => rfl
7    | [], b :: _, _, _, h => by cases (h b).mpr List.mem_cons_self
```

```
  | a :: _, [], _, _, h => by cases (h a).mp List.mem_cons_self
  | a :: t₁, b :: t₂, h₁, h₂, h => by
      have hab : a = b := by
        have ha₂ : a ∈ b :: t₂ := (h a).mp List.mem_cons_self
        have hb₁ : b ∈ a :: t₁ := (h b).mpr List.mem_cons_self
        exact le_antisymm (sorted_head_le h₁ b hb₁) (sorted_head_le h₂ a ha₂)
      subst hab
      have htails : ∀ x, x ∈ t₁ ↔ x ∈ t₂ := by
        intro x
        constructor
        · intro hx
          have hne : x ≠ a :=
            fun he => lt_irrefl a (he ▸ sorted_head_lt h₁ x hx)
          cases List.mem_cons.mp ((h x).mp (List.mem_cons_of_mem a hx)) with
          | inl he => exact absurd he hne
          | inr hm => exact hm
        · intro hx
          have hne : x ≠ a :=
            fun he => lt_irrefl a (he ▸ sorted_head_lt h₂ x hx)
          cases List.mem_cons.mp ((h x).mpr (List.mem_cons_of_mem a hx)) with
          | inl he => exact absurd he hne
          | inr hm => exact hm
      rw [sorted_ext (sorted_tail h₁) (sorted_tail h₂) htails]
```

If you want one proof from this book under your fingers rather than under your eyes, make it this one. It is also the best structural- induction training the series has to offer: the recursion is on both lists at once, the interesting case does antisymmetry at the heads and an honest little membership argument at the tails, and every hypothesis earns its place.

5.4 The `SList` Type and Its Semilattice

Package a list with its sortedness proof and the set type is done:

```
/-- A finite set: a strictly sorted list plus the proof that it is one. -/
structure SList (α : Type) [TotalOrder α] where
  val : List α
  sorted : SList.Sorted val
```

Because `Sorted val` is a proposition, two `SLists` with equal `vals` are equal outright — proof irrelevance disposes of the certificates — and extensionality lifts to the packaged type:

```
instance : Membership α (SList α) := ⟨fun s x => x ∈ s.val⟩

```

```
/-- Same value, same set — the `Sorted` proof is irrelevant. -/
theorem val_inj {s t : SList α} (h : s.val = t.val) : s = t := by
  cases s; cases t; cases h; rfl

/-- Same members, same set. The `sorted_ext` payoff. -/
theorem ext_mem {s t : SList α} (h : ∀ x, x ∈ s ↔ x ∈ t) : s = t :=
  val_inj (sorted_ext s.sorted t.sorted h)
```

The operations wrap the raw ones, carrying their sortedness proofs, and each gets a `simp`-ready membership lemma — the interface through which all later reasoning flows:

```
def empty : SList α := ⟨[], .nil⟩

def insert (x : α) (s : SList α) : SList α :=
  ⟨insertS x s.val, sorted_insertS s.sorted⟩

def union (s t : SList α) : SList α :=
  ⟨unionL s.val t.val, sorted_unionL t.sorted⟩

def filter (p : α → Bool) (s : SList α) : SList α :=
  ⟨s.val.filter p, sorted_filter p s.sorted⟩

def size (s : SList α) : Nat := s.val.length

@[simp] theorem mem_insert {x y : α} {s : SList α} :
    x ∈ insert y s ↔ x = y ∨ x ∈ s := mem_insertS

@[simp] theorem mem_union {x : α} {s t : SList α} :
    x ∈ union s t ↔ x ∈ s ∨ x ∈ t := mem_unionL

@[simp] theorem mem_filter {x : α} {p : α → Bool} {s : SList α} :
    x ∈ filter p s ↔ x ∈ s ∧ p x := List.mem_filter

@[simp] theorem not_mem_empty (x : α) : ¬ x ∈ (empty : SList α) := by
  intro h
  cases h
```

Now the order. It is subset — and watch where extensionality slots in. Reflexivity and transitivity of $\subseteq$ are one-liners about implications; antisymmetry says mutual subsets are *equal*, and that is `ext_mem`, which is `sorted_ext`. Canonicity is load-bearing, exactly here:

```
instance : LE (SList α) := ⟨fun s t => ∀ x ∈ s, x ∈ t⟩

instance {s t : SList α} : Decidable (s ⊑ t) :=
  List.decidableBAll (· ∈ t.val) s.val
```

```
instance : Order.PartialOrder (SList α) where
  le_refl _ _ hx := hx
  le_antisymm h₁ h₂ := ext_mem fun x => ⟨h₁ x, h₂ x⟩
  le_trans h₁ h₂ x hx := h₂ x (h₁ x hx)

instance : BoundedJoinSemilattice (SList α) where
  sup := union
  bot := empty
  le_sup_left _ _ _ hx := mem_union.mpr (Or.inl hx)
  le_sup_right _ _ _ hx := mem_union.mpr (Or.inr hx)
  sup_le _ _ _ h₁ h₂ x hx :=
    match mem_union.mp hx with
    | Or.inl hs => h₁ x hs
    | Or.inr ht => h₂ x ht
  bot_le _ x hx := absurd hx (not_mem_empty x)
```

And now a small, satisfying reversal. The plan for this chapter was to prove `union_comm`, `union_assoc` and `union_idem` by hand from `sorted_ext` — membership logic on the left, membership logic on the right, extensionality in the middle. We can still do that. But we do not have to: Chapter 2 proved ACI *once, for every bounded join-semilattice, from the axioms alone*. The instance above is the hard work — and the route through it was `sorted_ext`, antisymmetry, instance — so the three laws arrive as corollaries:

```
theorem union_comm (s t : SList α) : union s t = union t s :=
  Order.sup_comm s t

theorem union_assoc (s t u : SList α) :
    union (union s t) u = union s (union t u) :=
  Order.sup_assoc s t u

theorem union_idem (s : SList α) : union s s = s :=
  Order.sup_idem s
```

Three routes so far to the same three laws: pointwise (the G-Counter, inherited from `Nat.max`), componentwise (the PN-Counter, inherited from the product), and now set-theoretic, through canonicity. One algebra. Chapter 6 adds a fourth route and Chapter 8 shows why the algebra, however reached, is the whole story.

5.5 The G-Set, and a Refutation of Deletion

The grow-only set is the `SList` read as a CRDT — add is insert, merge is union, query is membership:

```
/-- The grow-only set over `Nat`: an `SList` read as a CRDT.
```

```
    Update = insert (inflationary), merge = union, query = membership. -/
abbrev GSet := SList Nat

def add (x : Nat) (s : GSet) : GSet := SList.insert x s

theorem add_inflationary (x : Nat) : Order.Inflationary (add x) :=
  fun _ _ hy => SList.mem_insert.mpr (Or.inr hy)
```

Users, of course, want to remove things. The obvious removal is deletion — filter the element out. It is a perfectly good function on sets. It is a catastrophically bad *update*, because it moves the state *down* the lattice, and the lattice has an opinion about that: any peer still holding the element out-informs you, and the next merge puts it back.

Refuted 5.1 — Remove-by-deletion is monotone

Deletion is deflationary, and the merge resurrects what it deleted. Replica A deletes 1; a peer still holds $\{1\}$; one merge later, 1 is a member of A's state again.

```
def naiveRemove (x : Nat) (s : GSet) : GSet := SList.filter (· != x) s

theorem naiveRemove_not_inflationary :
    ¬ (add 1 (⊥ : GSet) ⊑ naiveRemove 1 (add 1 (⊥ : GSet))) := by decide

theorem naiveRemove_resurrects :
    1 ∈ (naiveRemove 1 (add 1 (⊥ : GSet)) ⊔ add 1 (⊥ : GSet)) := by decide
```

The element does not stay removed because, from the lattice's point of view, removal never happened: the deleted state is *indistinguishable from an earlier one*. Whatever "remove" is going to mean, it must be new information, not less information.

5.6 The 2P-Set: Removal as a Tombstone

If removal must add information, let it add exactly the assertion "this element was removed." Keep *two* grow-only sets: the elements ever added, and the elements ever removed — the second set's entries are traditionally called *tombstones*. Membership consults both.

```
/-- The two-phase set: a grow-only set of adds and a grow-only set of
    tombstones. Removal *adds* a tombstone — states only grow. -/
abbrev TwoPSet := GSet × GSet

def adds (s : TwoPSet) : GSet := s.1
def tombs (s : TwoPSet) : GSet := s.2

def add (x : Nat) (s : TwoPSet) : TwoPSet := (SList.insert x s.1, s.2)
```

```
def remove (x : Nat) (s : TwoPSet) : TwoPSet := (s.1, SList.insert x s.2)

/-- Membership: added and not tombstoned. -/
def mem (x : Nat) (s : TwoPSet) : Prop := x ∈ s.1 ∧ ¬ x ∈ s.2
```

The semilattice instance costs nothing — `TwoPSet` is a product of two `SList` lattices, and Chapter 2's product instance covers it, exactly as it covered the PN-Counter. The structural rhyme is worth hearing: *the 2P-Set is to the G-Set what the PN-Counter is to the G-Counter*. One grow-only component for the positive events, one for the negative, a query that reconciles them, and a state that never forgets either.

And, as with the decrement, the scandal-then-not sentence: removal is inflationary.

```
/-- Removal is inflationary *on state* — the tombstone is information. -/
theorem remove_inflationary (x : Nat) : Order.Inflationary (remove x) :=
  fun s => ⟨le_refl s.1, fun _ hy => SList.mem_insert.mpr (Or.inr hy)⟩
```

This design genuinely works — Refutation 5.1's resurrection cannot happen, because the tombstone travels through every merge and out-votes the stale add wherever it arrives. But the tombstone is *absorbing*, and that has a sharp edge:

Refuted 5.2 — The 2P-Set supports re-adding

Add 1, remove it, add it again. Membership stays false: the tombstone out-votes *every* add, including ones that come after the removal.

```
theorem no_readd :
    ¬ mem 1 (add 1 (remove 1 (add 1 (⊥ : TwoPSet)))) := by decide

/-- ... even though the element is in `adds` twice over. -/
theorem readd_recorded :
    1 ∈ adds (add 1 (remove 1 (add 1 (⊥ : TwoPSet)))) := by decide
```

The second `add` is not lost — `adds` records it — it is *outranked*. The state cannot express "added, removed, then added again," because neither component remembers order, and the query must resolve the standoff the same way forever. The 2P-Set resolves it for the tombstone. An element's life has exactly two phases — hence the name — and the second is terminal.

This is the cliffhanger of Part II. To let a *later* add defeat an *earlier* remove, the state must be able to tell later adds from earlier ones — some notion of "this add was not yet observed by that remove" must be reified into the data. Chapter 6 will reify *time*; Chapter 7 will reify *observation itself*, and win.

The Cost Ledger: Tombstones Are Forever

Before the interlude, an honest accounting. Tombstones solve removal by never letting go: `remove` inserts and nothing ever deletes, so the tombstone set grows monotonically for the lifetime of the system, and a removed element costs more space *after* removal than before — it occupies both components. Removed is not gone; removed is *remembered harder*. Exercise 7.2 in Chapter 7 makes the growth a theorem, and Chapter 13 closes the loop with the uncomfortable punchline: compacting tombstones away safely requires knowing that every replica has seen them — which is a coordination problem, the very thing CRDTs exist to avoid.

5.7 Computation Interlude: Convergence Is Not Consensus on What You Wanted

Replica A adds 1 and 3. Concurrently, replica B adds 1 and then removes it:

```
/-- Interlude: the 2P-Set converges regardless of merge order — it just
    converges to "removed wins, forever". -/
def interlude : List (String × Bool) :=
  let atA := add 3 (add 1 (⊥ : TwoPSet))       -- A adds 1, 3
  let atB := remove 1 (add 1 (⊥ : TwoPSet))    -- B adds 1 then removes it
  [ ("1 ∈ A ⊔ B", decide (mem 1 (atA ⊔ atB))),
    ("1 ∈ B ⊔ A", decide (mem 1 (atB ⊔ atA))),
    ("3 ∈ A ⊔ B", decide (mem 3 (atA ⊔ atB))),
    ("re-add at A ⊔ B", decide (mem 1 (add 1 (atA ⊔ atB)))) ]

#eval interlude
```

```
[("1 ∈ A ⊔ B", false), ("1 ∈ B ⊔ A", false), ("3 ∈ A ⊔ B", true), ("re-add at
 ↪  A ⊔ B", false)]
```

Both merge orders agree — 1 is out, 3 is in — and the attempted re-add on the merged state changes nothing. The 2P-Set is a perfectly lawful CRDT: it converges, deterministically, everywhere. Whether it converges to the semantics your users wanted is a different question, and the last line answers it for anyone who expected re-adding to work. Keep the distinction — *convergence versus intent* — close at hand; it returns in Chapters 6 and 13.

5.8 Exercises

Exercise 5.1 — Deciding membership

★ Exhibit the `Decidable` instances that make this chapter's `decide` refutations tick: membership `x ∈ s` for `s : SList α`, the subset order `s ⊑ t`, and 2P-Set membership `TwoPSet.mem x s`. (All three are in the companion; write them yourself first. The interesting one is `⊑` — find the bounded quantifier instance in the library.)

Exercise 5.2 — Size is monotone

★★ Prove that `SList.size` is monotone on G-Sets: if `s ⊑ t` then `size s ≤ size t`. This is a pigeonhole fact about strictly sorted lists — a sorted, duplicate-free list whose members all occur in another is no longer than it. Induct on both lists at once, in the style of `sorted_ext`. Solution in Appendix B.

Exercise 5.3 — Inflationary updates compose

★★ We proved `remove_inflationary` above. Prove that `TwoPSet.add x` is inflationary too, and then the general glue fact: the composition of two inflationary updates is inflationary. Conclude that any finite script of adds and removes moves a 2P-Set up its lattice — while, per Refutation 5.2, its *membership query* can flip from true to false along the way. State in one sentence why that is not a contradiction.

Exercise 5.4 — Intersection is a meet

★★★ Define intersection of `SList`s (filter one by membership in the other) and prove it is the *greatest lower bound* of the subset order, in the sense of the predecessor's `IsInfOf`: below both arguments, and above anything below both. Then explain — with a concrete two-replica scenario — why merging by intersection would violate the inflation discipline and lose updates, even though intersection is exactly as commutative, associative and idempotent as union. (Moral: ACI is necessary, but the merge must also sit on the *upper* side of the order. Hints in Appendix B.)

Chapter 6

Last Writer Wins: LWW-Register and LWW-Element-Set

> *"When two copies of the database disagree, let the most recent update win — and make sure everyone agrees on what 'most recent' means."*
> *— after Johnson & Thomas, RFC 677 (1975), where this idea first appeared*

Chapter 1 refuted the idea that overwriting is a merge strategy you get for free (Refutation 1.1): naive overwrite loses updates and is not even commutative. Yet overwrite semantics is often exactly what an application wants. A user profile has one display name; a thermostat has one set point; a configuration flag is on or off. For such data, "keep everything anyone ever said" — the G-Set's answer — is wrong. We want the *last* word to win.

This chapter recovers overwrite semantics *lawfully*. The move is the same one that powered every design in this part: if the merge needs information, put that information in the state. Overwrite needs to know which write came last, so we reify *time* into the state — each write carries a stamp, and merge keeps the write with the larger stamp. Done correctly, this yields a bounded join-semilattice like any other, and convergence follows from the same three laws. Done naively, it fails in a way that is instructive enough to open the chapter with.

6.1 The Tie That Breaks Everything

Model a naive timestamped register as a pair `(timestamp, value)`, and merge two of them by keeping the one with the larger timestamp:

```
/-- A naive timestamped write `(timestamp, value)` merged by
    timestamp alone. -/
def naiveMerge (a b : Nat × Nat) : Nat × Nat :=
  if a.1 < b.1 then b else a
```

When the timestamps differ this does what it promises. But nothing prevents two replicas from writing at the same logical instant — and then `naiveMerge` keeps whichever write happens to be its *left* argument.

Refuted 6.1 — LWW merge with bare timestamps is commutative

Two writes at timestamp 3, one of value 10 and one of value 20. Merging them in the two possible orders gives two different winners:

```
theorem naive_tie_breaks_comm :
    naiveMerge (3, 10) (3, 20) ≠ naiveMerge (3, 20) (3, 10) := by decide
```

Replica A computes `naiveMerge a b` and keeps 10; replica B computes `naiveMerge b a` and keeps 20. Both have received both writes. They disagree forever. Commutativity was not a nicety — it was the antidote to reordering, and we just lost it.

The failure is precise: `naiveMerge` is a max, but a max over a relation that is not *total* — two distinct writes can tie, and on a tie the function invents an answer from argument order, which is exactly the thing the network refuses to preserve.

The fix is equally precise.

Totalize the order.

Give each replica a distinct identity and stamp every write with the pair (timestamp, replica id), compared lexicographically: first by timestamp, then — only on a timestamp tie — by replica id. Two writes from the same replica never share a timestamp position in its own history without being the same write; two writes from different replicas differ in the second coordinate. Writes that can actually coexist never tie, so "the larger write" is well defined no matter who computes it.

6.2 More Total Orders, Once and For All

Chapter 5 introduced the `TotalOrder` class to keep the sorted-list machinery honest, and instantiated it at `Nat`. Stamps need two more instances — `Fin n` for replica ids and the *lexicographic product* for pairs — and the element sets later in this chapter want `Bool`. We prove each once; Chapter 7 will collect the rent a second time.

```
instance : TotalOrder Bool where
  le a b := (!a || b) = true
  deq := inferInstance
  decLe _ _ := inferInstanceAs (Decidable (_ = true))
  le_refl := by intro a; cases a <;> simp
  le_antisymm := by intro a b h₁ h₂; cases a <;> cases b <;> simp_all
  le_trans := by intro a b c h₁ h₂; cases a <;> cases b <;> cases c <;>
    ↪ simp_all
  le_total := by intro a b; cases a <;> cases b <;> simp

instance {n : Nat} : TotalOrder (Fin n) where
  le a b := a.val ≤ b.val
```

```
  deq := inferInstance
  decLe a b := Nat.decLe a.val b.val
  le_refl a := Nat.le_refl a.val
  le_antisymm h₁ h₂ := Fin.ext (Nat.le_antisymm h₁ h₂)
  le_trans := Nat.le_trans
  le_total a b := Nat.le_total a.val b.val
```

`Bool` is ordered `false` ≼ `true` (the definition `(!a || b) = true` is a Boolean encoding of implication); being finite, all four laws fall to case analysis. `Fin n` simply borrows the order of the underlying naturals, with `Fin.ext` converting value-equality back into `Fin`-equality for antisymmetry.

The interesting instance is the product. Compare first coordinates; only if they are *equal* consult the second:

```
/-- The lexicographic order on pairs: compare firsts, tie-break on
    seconds. Total because ties in the first coordinate are *equalities*
    (that is what antisymmetry buys). -/
instance instTotalOrderProd {α β : Type} [TotalOrder α] [TotalOrder β] :
    TotalOrder (α × β) where
  le p q := p.1 ≺ q.1 ∨ (p.1 = q.1 ∧ p.2 ≼ q.2)
  deq := inferInstance
  decLe _ _ := inferInstanceAs (Decidable (_ ∨ _))
  le_refl p := Or.inr ⟨rfl, le_refl p.2⟩
  le_antisymm {p q} h₁ h₂ := by
    rcases h₁ with h₁ | ⟨he₁, hle₁⟩
    · rcases h₂ with h₂ | ⟨he₂, _⟩
      · exact absurd h₂.1 (not_le_of_lt h₁)
      · rw [he₂] at h₁
        exact absurd h₁ (lt_irrefl _)
    · rcases h₂ with h₂ | ⟨_, hle₂⟩
      · rw [he₁] at h₂
        exact absurd h₂ (lt_irrefl _)
      · cases p; cases q
        simp_all
        exact le_antisymm hle₁ hle₂
  le_trans {p q r} h₁ h₂ := by
    rcases h₁ with h₁ | ⟨he₁, hle₁⟩
    · rcases h₂ with h₂ | ⟨he₂, _⟩
      · exact Or.inl (lt_trans h₁ h₂)
      · exact Or.inl (he₂ ▸ h₁)
    · rcases h₂ with h₂ | ⟨he₂, hle₂⟩
      · exact Or.inl (he₁ ▸ h₂)
      · exact Or.inr ⟨he₁.trans he₂, le_trans hle₁ hle₂⟩
  le_total p q := by
    by_cases he : p.1 = q.1
    · rcases le_total p.2 q.2 with h | h
```

```
      · exact Or.inl (Or.inr ⟨he, h⟩)
      · exact Or.inr (Or.inr ⟨he.symm, h⟩)
    · rcases le_total p.1 q.1 with h | h
      · exact Or.inl (Or.inl ⟨h, he⟩)
      · exact Or.inr (Or.inl ⟨h, fun hc => he hc.symm⟩)
```

Read the definition of `le` carefully: $p \preccurlyeq q$ when $p_1 \prec q_1$ *strictly*, or when $p_1 = q_1$ and $p_2 \preccurlyeq q_2$. The strictness in the first disjunct is what makes the case analysis in `le_antisymm` work: if each pair is below the other, the strict cases annihilate each other (a strict inequality in both directions is absurd; a strict inequality against an equality is `lt_irrefl` after rewriting), leaving only the case where the first coordinates are equal and antisymmetry of β finishes. Totality splits on whether the first coordinates are equal — if so, totality of β decides; if not, totality of α plus the disequality gives strictness on one side or the other.

Notice what we did *not* do: we did not prove these laws for "pairs of a natural and a replica id." We proved them for pairs over any two total orders. Chapter 7 needs triples (element, replica, counter); those are pairs nested in pairs, and this one instance covers them with zero new proof obligations.

6.3 The Abstract Maximum

Now the merge. Given a total order, "keep the larger" is a two-line function:

```
/-- The larger of two elements. -/
def maxo (a b : α) : α := if a ≤ b then b else a
```

We could now prove that merging stamped writes is commutative, associative, and idempotent by unfolding `maxo` inside the register type and case-bashing across three layers of `Option` and a trichotomy — a proof with dozens of cases, each trivial, the whole thing unreadable. The predecessor book taught a better move, and this chapter is where it pays off most visibly:

Prove the abstract lemma once.

`maxo` is a maximum over *any* decidable total order. Its algebra has nothing to do with stamps, registers, or options. So prove the algebra where it lives:

```
theorem le_maxo_left (a b : α) : a ≤ maxo a b := by
  unfold maxo; split
  · assumption
  · exact le_refl a

theorem le_maxo_right (a b : α) : b ≤ maxo a b := by
  unfold maxo; split
  · exact le_refl b
```

```
  · rename_i h
    exact le_of_lt (lt_of_not_le h)

theorem maxo_le {a b c : α} (h₁ : a ≤ c) (h₂ : b ≤ c) : maxo a b ≤ c := by
  unfold maxo; split <;> assumption

theorem maxo_idem (a : α) : maxo a a = a := by
  simp [maxo]

theorem maxo_comm (a b : α) : maxo a b = maxo b a := by
  unfold maxo
  by_cases h₁ : a ≤ b <;> by_cases h₂ : b ≤ a <;> simp [h₁, h₂]
  · exact le_antisymm h₂ h₁
  · exact absurd ((le_total a b).resolve_left h₁) h₂

theorem maxo_assoc (a b c : α) : maxo (maxo a b) c = maxo a (maxo b c) := by
  by_cases h₁ : a ≤ b <;> by_cases h₂ : b ≤ c
  · simp [maxo, h₁, h₂, le_trans h₁ h₂]
  · simp [maxo, h₁, h₂]
  · simp [maxo, h₁, h₂]
  · have hcb : c < b := lt_of_not_le h₂
    have hba : b < a := lt_of_not_le h₁
    have hca : ¬ a ≤ c := not_le_of_lt (lt_trans hcb hba)
    simp [maxo, h₁, h₂, hca]
```

Look at `maxo_comm`. The two interesting cases are exactly the two ways the naive design died and lived: if $a \preccurlyeq b$ and $b \preccurlyeq a$ simultaneously, then — in a total order with antisymmetry — a and b are *equal*, so it does not matter which one you keep. That is the theorem the pair $(\text{timestamp}, \text{replica id})$ was constructed to make true: naive timestamps could tie while the values differed; totalized stamps can only "tie" by being the same write. And if neither $a \preccurlyeq b$ nor $b \preccurlyeq a$, totality is violated — absurd. Associativity is a four-way case split where transitivity discharges the two coherent cases and produces the missing comparison facts in the others.

The first three lemmas say more than convenience: `le_maxo_left`, `le_maxo_right`, and `maxo_le` are precisely the three join-semilattice axioms for $\sqcup := \texttt{maxo}$. A total order is a semilattice where the join of two elements is always one of them.

6.4 The Option Lattice: "Never Written" Is Information Too

One ingredient is missing. A register that has never been written holds no stamped value at all, and a bounded join-semilattice needs a $\bot$. `Option` supplies it: `none` is "never written," below everything; two actual writes compare by the total order.

```
/-- "Never written" is below everything; two writes compare by order. -/
def leO {α : Type} [TotalOrder α] : Option α → Option α → Prop
  | none, _ => True
  | some _, none => False
  | some a, some b => a ≤ b

instance {α : Type} [TotalOrder α] : LE (Option α) := ⟨leO⟩

/-- Merging two optional writes: the larger one wins. -/
def mergeO {α : Type} [TotalOrder α] : Option α → Option α → Option α
  | none, y => y
  | x, none => x
  | some a, some b => some (maxo a b)
```

This should feel familiar: it is the `FlatNat` construction from the predecessor with the `conflict` ceiling removed and the `known`-versus-`known` clash resolved by `maxo` instead of escalated. Where a propagator cell treated two different values as a contradiction, a register treats them as a race with a lawful winner.

The instances are case analyses over the `Option` constructors, each case either trivial or delegated to a `TotalOrder` law or a `maxo` lemma:

```
instance {α : Type} [TotalOrder α] : Order.PartialOrder (Option α) where
  le_refl x := by
    cases x with
    | none => trivial
    | some a => exact TotalOrder.le_refl a
  le_antisymm {x y} h₁ h₂ := by
    cases x with
    | none =>
        cases y with
        | none => rfl
        | some b => exact False.elim h₂
    | some a =>
        cases y with
        | none => exact False.elim h₁
        | some b => exact congrArg some (TotalOrder.le_antisymm h₁ h₂)
  le_trans {x y z} h₁ h₂ := by
    cases x with
    | none => trivial
    | some a =>
        cases y with
        | none => exact False.elim h₁
        | some b =>
            cases z with
            | none => exact False.elim h₂
```

```
            | some c => exact TotalOrder.le_trans h₁ h₂

instance {α : Type} [TotalOrder α] : BoundedJoinSemilattice (Option α) where
  sup := mergeO
  bot := none
  le_sup_left x y := by
    cases x with
    | none => trivial
    | some a =>
        cases y with
        | none => exact TotalOrder.le_refl a
        | some b => exact le_maxo_left a b
  le_sup_right x y := by
    cases y with
    | none => cases x <;> trivial
    | some b =>
        cases x with
        | none => exact TotalOrder.le_refl b
        | some a => exact le_maxo_right a b
  sup_le x y z h₁ h₂ := by
    cases x with
    | none =>
        cases y with
        | none => trivial
        | some b => exact h₂
    | some a =>
        cases y with
        | none => exact h₁
        | some b =>
            cases z with
            | none => exact False.elim h₁
            | some c => exact maxo_le h₁ h₂
  bot_le x := by cases x <;> trivial
```

This too is proved once, for `Option` over *any* total order. Every LWW-shaped design you will ever build — registers of numbers, of strings, of Booleans, maps of registers — inherits these thirty lines.

6.5 The Register

Now the register itself is nothing but names:

```
/-- A stamp: `(timestamp, replicaId)`, ordered lexicographically.
    Replica ids are distinct, so stamps of concurrently coexisting
    writes never tie. -/
abbrev Stamp (R : Nat) := Nat × Fin R
```

```

/-- A write: a stamp plus the written value. The whole payload is
    ordered lexicographically — the value-level tiebreak is unreachable
    in real executions (stamps are unique per write) and exists to make
    the algebra total on the carrier. -/
abbrev Write (R : Nat) (α : Type) [TotalOrder α] := Stamp R × α

/-- The LWW register: an optional stamped write, merged by taking the
    lexicographically larger one. -/
abbrev LWWReg (R : Nat) (α : Type) [TotalOrder α] := Option (Write R α)
```

Honesty about the carrier. A `Write` is ordered as a *whole*: timestamp, then replica id, then — if both somehow agree — the value itself. In any real execution the third comparison is dead code: a replica never issues two writes with the same timestamp position, and two replicas never share an id, so coexisting writes always differ within the stamp. But the *type* `LWWReg` contains states no execution produces (two identical stamps on different values), and our theorems quantify over the type. Ordering the full payload makes the order total on the carrier and the ACI laws unconditional — no "well-formed register" side conditions to thread through every statement. This is a standing trade in formalization: widen the order, shrink the hypotheses. When the cheap trick is unavailable, you carry an invariant instead; Chapter 7 does exactly that, and you can compare the costs.

Updates and queries:

```
/-- Write at replica `i`, logical time `t`. Merging (not overwriting!)
    keeps the update inflationary even if `t` is stale. -/
def write {R : Nat} {α : Type} [TotalOrder α]
    (t : Nat) (i : Fin R) (v : α) (r : LWWReg R α) : LWWReg R α :=
  r ⊔ some ((t, i), v)

/-- The observable value. -/
def read {R : Nat} {α : Type} [TotalOrder α] (r : LWWReg R α) : Option α :=
  r.map (·.2)

theorem write_inflationary {R : Nat} {α : Type} [TotalOrder α]
    (t : Nat) (i : Fin R) (v : α) :
    Order.Inflationary (write (R := R) t i v) :=
  fun r => le_sup_left r (some ((t, i), v))
```

Note that `write` *merges* the new stamped value rather than storing it unconditionally: a write with a stale stamp (an old timestamp arriving late, a clock that jumped backward) is absorbed without regressing the state. That is what makes `write_inflationary` a one-line corollary of `le_sup_left` — the update is literally a join, the merge discipline of Chapter 2 applied to the replica's own hand.

And the three laws? They were never about registers:

```
theorem merge_comm {α : Type} [TotalOrder α] (x y : Option α) :
    x ⊔ y = y ⊔ x := Order.sup_comm x y

theorem merge_assoc {α : Type} [TotalOrder α] (x y z : Option α) :
    x ⊔ y ⊔ z = x ⊔ (y ⊔ z) := Order.sup_assoc x y z

theorem merge_idem {α : Type} [TotalOrder α] (x : Option α) :
    x ⊔ x = x := Order.sup_idem x
```

Each is the generic ACI theorem of Chapter 2 applied to the `Option` instance — which we earned through `maxo`'s lemmas — which we earned through the total order. Pause on the route, because it is the third one this book has taken to the same destination. The counters of Chapters 3 and 4 got their algebra *pointwise*, slot by slot from `Nat.max`. The sets of Chapter 5 got it *setwise*, from membership logic through canonical sorted representations. The register gets it from *linear-order max*.

Three routes, one algebra.

The tie that killed the naive design is now just a test case:

```
/-- The tie that broke `naiveMerge`, resolved: same timestamp, distinct
    replicas, both merge orders give the same winner. -/
theorem tie_resolved :
    (write 3 (0 : Fin 2) 10 ⊥ ⊔ write 3 1 20 ⊥)
      = (write 3 1 20 ⊥ ⊔ write 3 (0 : Fin 2) 10 ⊥) := by decide
```

Both writes carry timestamp 3; the replica ids 0 and 1 break the tie identically for every observer. (Replica 1's write wins — arbitrarily, but *consistently*.)

Physical clocks lie. Nothing in this chapter reads a wall clock, and that is deliberate. Physical timestamps skew between machines, jump during NTP corrections, repeat across leap seconds, and generally cannot promise that a later write carries a larger stamp. The `Nat` timestamps here are *logical tokens*: the theorems hold for whatever numbers you supply, and the *semantics you want* — later writes actually winning — holds exactly to the extent that your stamps respect causality. Chapter 10 builds logical time that earns that property instead of assuming it, and explains what LWW registers look like on top of honest clocks.

6.6 Maps of Registers: The LWW-Element-Set

The 2P-Set of Chapter 5 could not re-add (Refutation 5.2) because its tombstones were permanent: "removed" was a black hole. A register per element gives a set with *revisable* membership: the last word on each element — add or remove — wins.

First, a general fact worth its own box: states indexed by keys form a semilattice whenever the per-key state does, key by key.

```
instance instFunLE {ι α : Type} [BoundedJoinSemilattice α] : LE (ι → α) :=
  ⟨fun f g => ∀ i, f i ≤ g i⟩

instance instFunPartialOrder {ι α : Type} [BoundedJoinSemilattice α] :
    PartialOrder (ι → α) where
  le_refl f i := PartialOrder.le_refl (f i)
  le_antisymm h₁ h₂ := funext fun i => PartialOrder.le_antisymm (h₁ i) (h₂ i)
  le_trans h₁ h₂ i := PartialOrder.le_trans (h₁ i) (h₂ i)

instance instFunBJS {ι α : Type} [BoundedJoinSemilattice α] :
    BoundedJoinSemilattice (ι → α) where
  sup f g := fun i => f i ⊔ g i
  bot := fun _ => ⊥
  le_sup_left f g i := BoundedJoinSemilattice.le_sup_left (f i) (g i)
  le_sup_right f g i := BoundedJoinSemilattice.le_sup_right (f i) (g i)
  sup_le f g h h₁ h₂ i := BoundedJoinSemilattice.sup_le (f i) (g i) (h i) (h₁
    ↪ i) (h₂ i)
  bot_le f i := BoundedJoinSemilattice.bot_le (f i)
```

You proved the shape of this in Exercise 2.3 for pairs; functions are the same construction with infinitely many components, and `funext` — which entered the book with the G-Counter in Chapter 3 — carries antisymmetry. (The instance asks for a semilattice codomain, so it never collides with the hand-built `GCounter` instance: `Nat` by itself is not a `BoundedJoinSemilattice` in this book.)

The LWW-Element-Set is then a map from elements to Boolean registers, where the register's value records the last word: `true` for "added," `false` for "removed."

```
/-- The LWW-Element-Set: one Boolean LWW register per element.
    `true` = "last word was add", `false` = "last word was remove". -/
abbrev LWWElementSet (R : Nat) := Nat → LWWReg R Bool

def stampWrite (e : Nat) (t : Nat) (i : Fin R) (b : Bool)
    (s : LWWElementSet R) : LWWElementSet R :=
  fun e' => if e' = e then write t i b (s e') else s e'

def add (e t : Nat) (i : Fin R) (s : LWWElementSet R) : LWWElementSet R :=
  stampWrite e t i true s

def remove (e t : Nat) (i : Fin R) (s : LWWElementSet R) : LWWElementSet R :=
  stampWrite e t i false s

def mem (e : Nat) (s : LWWElementSet R) : Bool :=
```

```
  match s e with
  | some (_, b) => b
  | none => false

theorem stampWrite_inflationary (e t : Nat) (i : Fin R) (b : Bool) :
    Order.Inflationary (stampWrite e t i b) := by
  intro s e'
  by_cases h : e' = e <;> simp [stampWrite, h]
  · exact write_inflationary t i b (s e)
  · exact le_refl (s e')
```

Removal is a *write*, not a deletion — the same inflationary shape as the 2P-Set's tombstone, but revisable: a later `add` out-stamps it. And the characterization of membership under merge costs nothing at all:

```
/-- Merging maps merges each element's register — by definition. -/
theorem mem_merge (e : Nat) (s t : LWWElementSet R) :
    mem e (s ⊔ t) = mem e (fun e' => s e' ⊔ t e') := rfl
```

The proof is `rfl` because the pointwise instance *defines* $s \sqcup t$ as the function $e' \mapsto s\,e' \sqcup t\,e'$; asking about element e of the merged map just is asking about the merge of the two registers at e. When a theorem is definitional, the right proof is to notice that.

6.7 Computation Interlude: Ties, Re-adds, and Redeliveries

The register first. Three writes on two replicas, including a genuine timestamp tie:

```
/-- Interlude demo: reads after out-of-order, duplicated merges. -/
def interlude : List (String × Option Nat) :=
  let w₁ : LWWReg 2 Nat := write 1 0 100 ⊥   -- replica 0 writes 100 at t=1
  let w₂ : LWWReg 2 Nat := write 2 1 200 ⊥   -- replica 1 writes 200 at t=2
  let w₃ : LWWReg 2 Nat := write 2 0 150 ⊥   -- replica 0 writes 150 at t=2
   ↪ (tie!)
  [ ("read (w₁ ⊔ w₂)", read (w₁ ⊔ w₂)),
    ("read (w₂ ⊔ w₁)", read (w₂ ⊔ w₁)),
    ("read (w₂ ⊔ w₃)  -- t=2 tie, replica 1 wins", read (w₂ ⊔ w₃)),
    ("read (w₃ ⊔ w₂)", read (w₃ ⊔ w₂)),
    ("read ((w₁ ⊔ w₂) ⊔ w₂)", read ((w₁ ⊔ w₂) ⊔ w₂)) ]

#eval interlude
```

```
[("read (w₁ ⊔ w₂)", some 200),
 ("read (w₂ ⊔ w₁)", some 200),
```

```
("read (w₂ ⊔ w₃)  -- t=2 tie, replica 1 wins", some 200),
("read (w₃ ⊔ w₂)", some 200),
("read ((w₁ ⊔ w₂) ⊔ w₂)", some 200)]
```

One value, five delivery histories. The tie between w_2 and w_3 — both at $t = 2$ — resolves to replica 1's write in both merge orders, and the duplicated delivery in the last line is absorbed by idempotence.

Now the element set. Replica A adds elements 1 and 7 at $t = 1$; concurrently, replica B removes element 1 at $t = 2$:

```
/-- Interlude: concurrent add and remove of the same element resolve
    by stamp, identically at every replica. -/
def interlude : List (String × Bool) :=
  let s₀ : LWWElementSet 2 := ⊥
  let atA := add 1 1 0 (add 7 1 0 s₀)          -- A adds 1 and 7 at t=1
  let atB := remove 1 2 1 s₀                    -- B removes 1 at t=2
  [ ("1 ∈ A ⊔ B", mem 1 (atA ⊔ atB)),
    ("1 ∈ B ⊔ A", mem 1 (atB ⊔ atA)),
    ("7 ∈ A ⊔ B", mem 7 (atA ⊔ atB)),
    ("re-add 1 at t=3", mem 1 (add 1 3 0 (atA ⊔ atB))) ]

#eval interlude
```

```
[("1 ∈ A ⊔ B", false), ("1 ∈ B ⊔ A", false), ("7 ∈ A ⊔ B", true), ("re-add 1
 ↪  at t=3", true)]
```

Compare this line by line with the 2P-Set interlude of Chapter 5. Element 1 is out — B's remove at $t = 2$ out-stamps A's add at $t = 1$, identically in both merge orders — while the untouched element 7 survives. And the last line is the one the 2P-Set could not produce: a re-add at $t = 3$ out-stamps the remove, and 1 is back. Tombstones have become opinions with dates.

But look again at the first two lines and ask *why* the remove won. Not because B knew something about A's add — B removed 1 from an empty set, sight unseen. It won because $2 > 1$. The LWW-Element-Set resolves every add/remove race by clock arbitration, and whoever holds the later stamp wins an argument they may not have known they were having. When the desired semantics is "a removal should only cancel the adds it has actually *seen*," stamps are the wrong tool.

That sentence is a specification. The next chapter implements it.

6.8 Exercises

Exercise 6.1 — The empty register is the identity

★ Prove that merging with the never-written register changes nothing: `theorem lww_merge_bot (x : LWWReg R Nat) : ⊥ ⊔ x = x`. You have seen the generic fact this instantiates; find it.

Exercise 6.2 — Three-way ties

★★ Extend Refutation 6.1 to three replicas: find three naive writes, all carrying the same bare timestamp, such that merging them in two different arrangements yields different winners, and check it with `decide`. (Argument *grouping* is fair game as well as order — recall which two laws the network demands.) Then explain in a sentence why the totalized `Stamp` version cannot exhibit this for any arrangement.

Exercise 6.3 — Remove wins

★★★ The element set above breaks stamp ties in favor of whatever `maxo` keeps — with `Bool` ordered `false` $\preccurlyeq$ `true`, a tied add beats a tied remove. Some applications want the opposite bias: *remove wins* on equal stamps. Rebuild `LWWElementSet` so that a remove with the same stamp as an add defeats it, and prove your variant still converges (the ACI laws still hold). *Hint:* you should not need to touch `maxo` or the `Option` lattice at all — only the order on the payload. Observe what did and did not change: convergence survived, the *meaning* flipped. Convergence is not intent; sit with that until Chapter 13 makes it a theme.

Chapter 7

The OR-Set: Add Wins

> *"To delete a thing you must first point at it; and you can only point at what you have seen."*

Take stock of the sets you now own. The G-Set never forgets. The 2P-Set forgets exactly once per element, forever (Refutation 5.2). The LWW-Element-Set revises freely but settles every dispute by clock, so a remove can cancel an add its issuer never saw. None of them meets the specification the last chapter ended on:

A remove should cancel precisely the adds it has observed — past adds, not future ones, not concurrent ones.

Under that reading, when an add and a remove of the same element race, the add should survive: the remove cannot have observed it, so it has no authority over it. The literature calls the resulting design the *observed-remove set*, or OR-Set, and its slogan is *add wins*. This chapter builds it, and proves the slogan as a theorem with machine-checked content — the longest proof in the book, and the first one that runs on an *invariant* rather than on algebra alone.

7.1 Tags: Making Adds Distinguishable

The 2P-Set failed because all adds of element e were indistinguishable: one tombstone condemned them all, past and future. The fix is to make every add a distinct event. Each replica carries a counter; performing an add mints a fresh *tag* — the pair of the replica's id and its current count — and attaches it to the element:

```
/-- A tag: `(replica, sequence number)` — never minted twice. -/
abbrev Tag (R : Nat) := Fin R × Nat

/-- An element paired with the tag of the add that produced it.
    Lex-ordered, so the sorted-list machinery of Chapter 5 applies
    unchanged. -/
abbrev ETag (R : Nat) := Nat × Tag R
```

A remove then tombstones *tags*, not elements — and only the tags it can see. A later add of the same element carries a tag that did not exist when the remove was issued, so no tombstone covers it. Re-add works. Concurrent add survives. That is the whole

design; the rest of the chapter is making it precise and proving it.

Notice the type of `ETag`: a pair of a `Nat` and a pair of a `Fin R` and a `Nat`. Chapter 5 built its sorted-list sets over *any* decidable total order, and Chapter 6 proved the lexicographic product instance for pairs over any two total orders. Compose those instances twice and `ETag R` is totally ordered with zero new proofs; `SList (ETag R)` is a lawful set of tagged elements with all of Chapter 5's theorems attached. The generalization we invested in then pays its rent now.

7.2 The State and Its Lattice

An OR-Set replica remembers three things: which tagged adds it has heard of, which tags have been tombstoned, and how many tags each replica has minted (so it can mint fresh ones itself).

```
/-- Observed-remove set: tagged adds, tombstoned tags, and a per-replica
    tag counter (a G-Counter — Chapter 3 pays rent again). -/
structure ORSet (R : Nat) where
  adds  : SList (ORSet.ETag R)
  tombs : SList (ORSet.ETag R)
  ctr   : GCounter R
```

Two grow-only sets and a G-Counter. Every component is a bounded join-semilattice you have already verified, so the composite is one by gluing — the same componentwise pattern as the product instance of Chapter 2, spelled out for three named fields:

```
instance : LE (ORSet R) :=
  ⟨fun s t => s.adds ⊑ t.adds ∧ s.tombs ⊑ t.tombs ∧ s.ctr ⊑ t.ctr⟩

instance : Order.PartialOrder (ORSet R) where
  le_refl s := ⟨le_refl s.adds, le_refl s.tombs, le_refl s.ctr⟩
  le_antisymm {s t} h₁ h₂ := by
    have e₁ := le_antisymm h₁.1 h₂.1
    have e₂ := le_antisymm h₁.2.1 h₂.2.1
    have e₃ := le_antisymm h₁.2.2 h₂.2.2
    cases s; cases t
    simp_all
  le_trans h₁ h₂ :=
    ⟨le_trans h₁.1 h₂.1, le_trans h₁.2.1 h₂.2.1, le_trans h₁.2.2 h₂.2.2⟩

instance : BoundedJoinSemilattice (ORSet R) where
  sup s t := ⟨s.adds ⊔ t.adds, s.tombs ⊔ t.tombs, s.ctr ⊔ t.ctr⟩
  bot := ⟨⊥, ⊥, ⊥⟩
  le_sup_left s t :=
    ⟨le_sup_left s.adds t.adds, le_sup_left s.tombs t.tombs,
```

```
    le_sup_left s.ctr t.ctr⟩
  le_sup_right s t :=
    ⟨le_sup_right s.adds t.adds, le_sup_right s.tombs t.tombs,
     le_sup_right s.ctr t.ctr⟩
  sup_le s t u h₁ h₂ :=
    ⟨sup_le s.adds t.adds u.adds h₁.1 h₂.1,
     sup_le s.tombs t.tombs u.tombs h₁.2.1 h₂.2.1,
     sup_le s.ctr t.ctr u.ctr h₁.2.2 h₂.2.2⟩
  bot_le s := ⟨bot_le s.adds, bot_le s.tombs, bot_le s.ctr⟩
```

Not a single new idea in forty lines: three known lattices, glued. Composition is free, and Chapter 12 will lean on that freedom hard. Three `simp` lemmas make the components of a merge available to later proofs without unfolding the instance:

```
@[simp] theorem sup_adds (s t : ORSet R) : (s ⊔ t).adds = s.adds ⊔ t.adds :=
  ↪ rfl
@[simp] theorem sup_tombs (s t : ORSet R) : (s ⊔ t).tombs = s.tombs ⊔ t.tombs
  ↪ := rfl
@[simp] theorem sup_ctr (s t : ORSet R) : (s ⊔ t).ctr = s.ctr ⊔ t.ctr := rfl
```

7.3 The Operations

```
/-- Add at replica `i`: mint the fresh tag `(i, ctr i)`, record the
    tagged add, and burn the tag by bumping the counter. -/
def add (i : Fin R) (e : Nat) (s : ORSet R) : ORSet R :=
  { adds  := SList.insert (e, i, s.ctr i) s.adds
    tombs := s.tombs
    ctr   := GCounter.increment i s.ctr }

/-- Remove: tombstone every *observed* tag for `e` — and only those. -/
def remove (e : Nat) (s : ORSet R) : ORSet R :=
  { adds  := s.adds
    tombs := s.tombs ⊔ SList.filter (fun p => p.1 == e) s.adds
    ctr   := s.ctr }
```

Read `add` as a two-step ritual: use the tag $(i, \texttt{ctr}\ i)$, then burn it by incrementing your own counter slot so it can never be minted again. Read `remove` as a *quotation*: it copies into `tombs` exactly the tagged adds of e currently visible in `adds`. It does not tombstone the element; it tombstones its observations of the element.

Membership asks whether any tagged add has escaped tombstoning:

```
/-- Membership: some tagged add of `e` survives untombstoned. -/
def mem (e : Nat) (s : ORSet R) : Prop :=
```

```
  ∃ t : Tag R, (e, t) ∈ s.adds ∧ ¬ (e, t) ∈ s.tombs
```

The existential ranges over all of `Tag R`, which is infinite — but any witness must sit inside the finite `adds` list, so the proposition is equivalent to a bounded search, and hence decidable:

```
theorem mem_iff_bex {e : Nat} {s : ORSet R} :
    mem e s ↔ ∃ p ∈ s.adds.val, p.1 = e ∧ ¬ p ∈ s.tombs := by
  constructor
  · intro ⟨t, hin, hnt⟩
    exact ⟨(e, t), hin, rfl, hnt⟩
  · intro ⟨p, hin, he, hnt⟩
    obtain ⟨e', t⟩ := p
    cases he
    exact ⟨t, hin, hnt⟩

instance {e : Nat} {s : ORSet R} : Decidable (mem e s) :=
  decidable_of_iff _ mem_iff_bex.symm
```

That instance is what lets `decide` check every claim in this chapter's refutation and interlude. Both operations are inflationary, by inspection of each component — even `remove`, whose only change is to *grow* the tombstone set:

```
theorem add_inflationary (i : Fin R) (e : Nat) :
    Order.Inflationary (add i e) :=
  fun s => ⟨fun _ hx => SList.mem_insert.mpr (Or.inr hx),
            le_refl s.tombs,
            GCounter.increment_inflationary i s.ctr⟩

theorem remove_inflationary (e : Nat) :
    Order.Inflationary (remove (R := R) e) :=
  fun s => ⟨le_refl s.adds,
            le_sup_left s.tombs _,
            le_refl s.ctr⟩
```

Figure 7.1 replays Chapter 5's fatal trace in this vocabulary; keep it in view through the next two sections, because the theorem we are heading for is exactly the claim written in its margin.

7.4 The Freshness Invariant

Here is the claim we owe: in a race between an add and a remove of the same element, the add survives the merge. Try to prove it directly and you hit a wall — it is simply *false* for arbitrary values of type `ORSet R`. Nothing in the type stops a state from containing a tombstone for a tag that was never added, or an add whose tag the

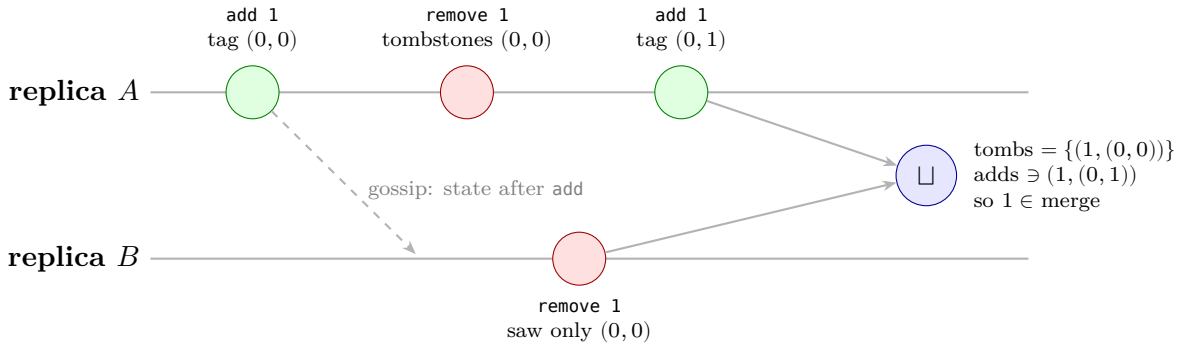

Figure 7.1: The trace that broke the 2P-Set, replayed with tags. B's remove quotes the only tag it has observed, $(0, 0)$. A's re-add mints the fresh tag $(0, 1)$, which no tombstone covers: after the merge, element 1 is present. Add wins.

counter has not yet reached; feed `addWins` such a state and the "fresh" tag it mints may already lie in someone's tombstone.

The type is too big. Real executions only ever visit a corner of it.

This is the situation the LWW chapter's honesty box foreshadowed: there we widened the order to make theorems unconditional; here no widening helps, and we take the other road. We name the corner — an *invariant* — prove that every operation preserves it, and prove the headline theorem *inside* it. This is the reader's first inductive-invariant proof, and it is worth slowing down for, because Chapters 8 and 9 run on exactly this style: establish at the initial state, preserve through every step, harvest at the end.

Definition — well-formed OR-Set

An OR-Set state is *well-formed* when

- every tombstone covers an *observed* add (`tombs` $\subseteq$ `adds`), and
- the counter dominates every recorded tag: a tag (i, k) in `adds` satisfies $k <$ `ctr` i.

```
structure Wf (s : ORSet R) : Prop where
  tombs_sub : ∀ p ∈ s.tombs, p ∈ s.adds
  ctr_dom   : ∀ p ∈ s.adds, p.2.2 < s.ctr p.2.1
```

The first clause is `remove`'s oath: it only quotes. The second is `add`'s: tags at or above your counter are *future* tags, untouched by any state you have produced so far. Together they say the next tag $(i,$ `ctr` $i)$ is genuinely fresh.

The empty state is well-formed vacuously, and each of the three ways a state can evolve preserves well-formedness:

```
theorem wf_bot : Wf (⊥ : ORSet R) where
  tombs_sub _ hp := absurd hp (SList.not_mem_empty _)
```

```
  ctr_dom _ hp := absurd hp (SList.not_mem_empty _)

theorem wf_add {s : ORSet R} (h : Wf s) (i : Fin R) (e : Nat) :
    Wf (add i e s) where
  tombs_sub p hp :=
    SList.mem_insert.mpr (Or.inr (h.tombs_sub p hp))
  ctr_dom p hp := by
    cases SList.mem_insert.mp hp with
    | inl he =>
        subst he
        simp [add, GCounter.increment]
    | inr hm =>
        have hd := h.ctr_dom p hm
        show p.2.2 < GCounter.increment i s.ctr p.2.1
        by_cases hpi : p.2.1 = i
        · rw [hpi] at hd ⊢
          simp [GCounter.increment]
          omega
        · simp [GCounter.increment, hpi]
          exact hd
```

`wf_add` is the only case with content: the freshly inserted tag $(i, \texttt{ctr}\ i)$ is dominated because the counter was bumped past it in the same operation, and every old tag stays dominated because `increment` never decreases any slot. Removal and merge are bookkeeping — a removal's new tombstones are quoted from `adds` by construction (that is what `SList.mem_filter` extracts), and a merge checks each clause componentwise, with `Nat.max` absorbing the two counter bounds:

```
theorem wf_remove {s : ORSet R} (h : Wf s) (e : Nat) :
    Wf (remove e s) where
  tombs_sub p hp := by
    cases SList.mem_union.mp hp with
    | inl ht => exact h.tombs_sub p ht
    | inr hf => exact (SList.mem_filter.mp hf).1
  ctr_dom := h.ctr_dom

theorem wf_merge {s t : ORSet R} (hs : Wf s) (ht : Wf t) :
    Wf (s ⊔ t) where
  tombs_sub p hp := by
    rw [sup_tombs] at hp
    rw [sup_adds]
    cases SList.mem_union.mp hp with
    | inl h => exact SList.mem_union.mpr (Or.inl (hs.tombs_sub p h))
    | inr h => exact SList.mem_union.mpr (Or.inr (ht.tombs_sub p h))
  ctr_dom p hp := by
    rw [sup_adds] at hp
```

```
    show p.2.2 < Nat.max (s.ctr p.2.1) (t.ctr p.2.1)
    cases SList.mem_union.mp hp with
    | inl h => exact Nat.lt_of_lt_of_le (hs.ctr_dom p h) (Nat.le_max_left _ _)
    | inr h => exact Nat.lt_of_lt_of_le (ht.ctr_dom p h) (Nat.le_max_right _
    ↪  _)
```

Every state reachable from $\bot$ by adds, removes, and merges is well-formed. That is what "inductive invariant" means: the three preservation lemmas are the induction step, one per constructor of the ways history can grow.

7.5 Add Wins

One more ingredient. The theorem will concern two replicas: A, about to add, and B, about to remove. What connects their states? In a real execution, only replica i ever increments counter slot i; everyone else learns slot i secondhand, through merges of states that ultimately descend from i's. So nobody's view of i's counter can *overtake* i's own — and merging preserves that, because the max of two numbers at most n is at most n:

```
/-- A replica's view of *someone else's* counter never overtakes the
    owner's own view — and merging peers preserves that. This is why
    the freshness hypothesis of `addWins` holds along real executions. -/
theorem ctr_view_merge {sA sB sC : ORSet R} (i : Fin R)
    (hB : sB.ctr i ≤ sA.ctr i) (hC : sC.ctr i ≤ sA.ctr i) :
    (sB ⊔ sC).ctr i ≤ sA.ctr i := by
  show Nat.max (sB.ctr i) (sC.ctr i) ≤ sA.ctr i
  exact Nat.max_le.mpr ⟨hB, hC⟩
```

We package that observation as a hypothesis: `sB.ctr` $i \le$ `sA.ctr` i, "B's view of i's counter is no fresher than i's own." With it, the headline:

```
/-- **Add wins.** Replica `i` adds `e` to its state `sA`; concurrently a
    peer removes `e` from its state `sB`. If `sB`'s view of `i`'s counter
    is no fresher than `i`'s own (true in every real execution), then
    after merging, `e` is a member: no tombstone can cover a tag that
    had not been minted when the remove happened. -/
theorem addWins {sA sB : ORSet R} (i : Fin R) (e : Nat)
    (hA : Wf sA) (hB : Wf sB)
    (hfresh : sB.ctr i ≤ sA.ctr i) :
    mem e (add i e sA ⊔ remove e sB) := by
  refine ⟨(i, sA.ctr i), ?_, ?_⟩
  · -- the fresh tagged add is in the merged adds
    rw [sup_adds]
    exact SList.mem_union.mpr (Or.inl (SList.mem_insert.mpr (Or.inl rfl)))
  · -- and no tombstone covers it
```

```
    intro htomb
    rw [sup_tombs] at htomb
    cases SList.mem_union.mp htomb with
    | inl hin =>
        -- a tombstone at A itself: then A had recorded tag (i, ctr i),
        -- contradicting counter dominance
        have := hA.ctr_dom _ (hA.tombs_sub _ hin)
        exact Nat.lt_irrefl _ this
    | inr hin =>
        -- a tombstone from B's remove: B had observed tag (i, ctr A i),
        -- contradicting B's view being no fresher than A's own
        have hadds : (e, i, sA.ctr i) ∈ sB.adds := by
          cases SList.mem_union.mp hin with
          | inl ht => exact hB.tombs_sub _ ht
          | inr hf => exact (SList.mem_filter.mp hf).1
        have hlt := hB.ctr_dom _ hadds
        simp at hlt
        omega
```

Walk it. The witness is the tag A mints: $(i, \texttt{sA.ctr}\ i)$. Its membership in the merged `adds` is immediate — A just inserted it. The work is showing no tombstone covers it, and the proof is an argument *about the past*. Suppose some tombstone does. It came from one of two places. If from A's own tombstones: well-formedness chains $\texttt{tombs} \subseteq \texttt{adds}$ into counter dominance, giving $\texttt{sA.ctr}\ i < \texttt{sA.ctr}\ i$ — a number strictly below itself. If from B's side — whether an old tombstone of B's or one freshly quoted by this very `remove` — then either way B had *observed* an add tagged $(i, \texttt{sA.ctr}\ i)$, so B's counter dominance gives $\texttt{sA.ctr}\ i < \texttt{sB.ctr}\ i$, colliding with the freshness hypothesis $\texttt{sB.ctr}\ i \leq \texttt{sA.ctr}\ i$. Both branches end in arithmetic absurdity, delivered by `omega`.

The tombstone would have to quote an observation of an event that had not yet happened. Well-formedness says states do not contain prophecies.

Add-wins, general form

In any execution over R replicas in which every state descends from $\bot$ by `add`, `remove`, and merge, if replica i performs `add i e` while any set of peers concurrently performs removes of e, then e is a member of every state that has merged i's add, until (at least) some replica *observes* that add and removes it.

Status of the proof. The two-replica instance is `ORSet.addWins` above, proved completely; its two hypotheses are discharged along real executions by the preservation lemmas (`wf_bot`, `wf_add`, `wf_remove`, `wf_merge`) and by `ctr_view_merge`. The n-replica statement requires threading "every peer's view of i's counter is at most i's own" through a full reachability relation over configurations; Chapter 9 builds exactly such a relation, and Exercise 7.3 points the way. We state the general claim in prose and do not pretend the Lean above proves more than it does.

The design has one load-bearing assumption left to stress: tags must be *fresh*. The counter discipline is not decoration.

Refuted 7.1 — Tags can be reused

Let a careless implementation mint tags without burning them:

```
def badAdd (i : Fin R) (k : Nat) (e : Nat) (s : ORSet R) : ORSet R :=
  { s with adds := SList.insert (e, i, k) s.adds }

theorem tag_reuse_bites :
    -- A adds 1 with tag (0,0); B (having seen it) removes 1;
    -- A "re-adds" 1 reusing tag (0,0); the merge has no member 1:
    -- B's old remove killed A's later add.
    let s₁ : ORSet 2 := badAdd 0 0 1 ⊥
    let atB := remove 1 s₁
    let atA := badAdd 0 0 1 s₁
    ¬ mem 1 (atA ⊔ atB) := by decide
```

A's second add is *later* than B's remove, yet the element is absent from the merge: the re-add reused tag $(0, 0)$, which B's tombstone already covered. A remove from the past reached forward and deleted an add from the future. Uniqueness of tags is exactly what `Wf`'s counter-dominance clause enforces, and this is the counterexample that shows why the invariant earns its keep.

7.6 Computation Interlude: The 2P Killer Trace, Replayed

Chapter 5 ended on a cliffhanger: add, remove, add again, and the 2P-Set answered `false`. The same trace, plus the concurrent-remove race and a duplicated delivery, against the OR-Set:

```
/-- Interlude: the add/remove/re-add trace that broke the 2P-Set,
    now passing, plus a duplicated-delivery run. -/
def interlude : List (String × Bool) :=
  let s₀ : ORSet 2 := ⊥
  let s₁ := add 0 1 s₀            -- A adds 1 (tag (0,0))
  let s₂ := remove 1 s₁           -- A removes 1
  let s₃ := add 0 1 s₂            -- A re-adds 1 (fresh tag (0,1))
  let atB := remove 1 s₁          -- concurrently, B (having seen s₁) removes
   ↪ 1
  [ ("1 ∈ s₁ (added)", decide (mem 1 s₁)),
    ("1 ∈ s₂ (removed)", decide (mem 1 s₂)),
    ("1 ∈ s₃ (re-added!)", decide (mem 1 s₃)),
    ("1 ∈ s₃ ⊔ B's remove (add wins)", decide (mem 1 (s₃ ⊔ atB))),
    ("1 ∈ (s₃ ⊔ atB) ⊔ atB (duplicate delivery)",
```

```
      decide (mem 1 ((s₃ ⊔ atB) ⊔ atB))) ]

#eval interlude
```

```
[("1 ∈ s₁ (added)", true),
 ("1 ∈ s₂ (removed)", false),
 ("1 ∈ s₃ (re-added!)", true),
 ("1 ∈ s₃ ⊔ B's remove (add wins)", true),
 ("1 ∈ (s₃ ⊔ atB) ⊔ atB (duplicate delivery)", true)]
```

Line three is the cliffhanger resolved: the re-add carries tag $(0, 1)$ and membership returns. Line four is `addWins` running concretely: B's remove was issued against s_1, where the only visible tag was $(0, 0)$; it has no authority over $(0, 1)$. Line five feeds the same remove in twice — idempotence of union shrugs.

7.7 The Bill: Tombstones, Again

The OR-Set's honesty about *which* adds a remove covers is paid for in the same currency as the 2P-Set's bluntness: state that never shrinks. Removed elements disappear from `mem` but their tagged adds and tombstones remain forever, and along any execution the tombstone set only grows:

```
/-- Tombstones never shrink: garbage is real (Exercise fodder; see
    Chapter 13 for the coordination irony). -/
theorem tombs_monotone {s t : ORSet R} (h : s ⊑ t) :
    s.tombs ⊑ t.tombs := h.2.1
```

A shopping list that has ever contained a million items carries a million ghosts. Exercise 7.2 makes the cost a theorem; Exercise 7.3 sketches the standard mitigation once Chapter 10 supplies the tooling; and Chapter 13 explains the sting in the tail — reclaiming tombstones safely needs the very coordination CRDTs exist to avoid.

7.8 Exercises

Exercise 7.1 — Deciding membership

★ The `Decidable (mem e s)` instance leans on `mem_iff_bex` and `decidable_of_iff`. Spell out why the raw definition of `mem` — an existential over the infinite type `Tag R` — is not directly decidable by instance search, and identify the core-library instance that decides the bounded form. Then use the instance: check by `decide` that `mem 1 (add 0 1 ⊥ : ORSet 2)` holds and that `mem 2 (add 0 1 ⊥ : ORSet 2)` does not.

Exercise 7.2 — The tombstone bill

★★ Prove that the tombstone *count* is monotone along the information order: `SList.size s.tombs ≤ SList.size t.tombs` whenever `s ⊑ t`. You will want a lemma of independent interest: a sorted, duplicate-free list that is a member-wise subset of another is no longer than it. (Compare Exercise 5.2; the proof is the same induction, now earning double rent.)

Exercise 7.3 — An optimized OR-Set

★★★ *Finish after Chapter 10.* Tombstones exist to remember which tags a remove has covered. But tags minted by replica i arrive in counter order, so "all of i's tags below k" compresses to the single number k — a version-vector-shaped summary. Sketch (in code; prove what you can) an OR-Set variant whose removal state is a `VV R` per element plus a set of *exceptions*, and show on an example that it answers the interlude's trace identically while storing asymptotically less. *Hint:* the summary is sound precisely because of the counter-dominance half of `Wf`; hints in Appendix B.

Part III

Convergence

Chapter 8

Multisets, Folds, and the Main Theorem

> *"Replicas of a conflict-free replicated data type converge without any synchronization: correct convergence is guaranteed by simple mathematical properties of the data type itself."*
> — *after Shapiro, Preguiça, Baquero & Zawirski (2011)*

Part II built five working replicated data types, and for each of them we proved the same three facts: the state space is a bounded join-semilattice, updates are inflationary, and merge is the join. Every chapter then ended with an interlude in which replicas received updates out of order, twice over, in scrambled groupings — and agreed anyway. Five times we watched it happen; not once did we prove it *must* happen.

This chapter pays that debt. We state and prove, once and for all state-based CRDTs, the theorem the whole book has been circling:

Replicas that have received the same set of updates hold equal state.

The proof needs a way to say "the same set of updates" when what a replica actually receives is a *sequence* — and the honest mathematical home for "a sequence where order and repetition are noise" is a new kind of type: a *quotient*. This is the chapter where you learn quotient types, deliberately and slowly, because this is the chapter where they stop being optional.

8.1 What a Replica Has Heard

Fix a bounded join-semilattice σ of states. In the model of this chapter, an execution delivers to each replica a *list* of update-states — each element is the full state some update produced somewhere, exactly as a gossiping peer would ship it. A replica's own state is then determined: it is everything it has heard, joined together.

```
namespace SEC

variable {σ : Type} [BoundedJoinSemilattice σ]

/-- A replica's state is the join of everything it has received,
```

```
    folded starting from ⊥. -/
def foldJoin (l : List σ) : σ := l.foldl (· ⊔ ·) ⊥
```

This is the propagator cell's `write` loop, replayed from the transcript: start at $\bot$ (no information), and for each arrival, merge. A cell that processed the deliveries `[u₁, u₂, u₃]` holds $((\bot \sqcup u_1) \sqcup u_2) \sqcup u_3$; that is exactly `foldJoin [u₁, u₂, u₃]`.

But the list is a lie — or rather, it says too much. A list records an *order* of arrival and a *count* of arrivals, and the network controls both. Replica A may receive `[u₁, u₂, u₃]` while replica B receives `[u₃, u₁, u₂, u₂, u₁]`: same news, different postal history. If the theorem is to say "same updates, same state," we need a mathematical object that remembers *which* updates arrived and forgets everything else the list accidentally recorded.

Forgetting order but keeping content is precisely what a *multiset* is. And a multiset is not a new inductive type — it is a new kind of type altogether.

8.2 Quotients: Types with Intent

Here is a situation you have met before, outside of Lean. You define integers as pairs of naturals, intending (p, n) to mean $p - n$. Then $(3, 5)$ and $(10, 12)$ both mean -2 — they are *different pairs* with the *same intent*. Every theorem you prove must respect the intent rather than the representation, and nothing in the type $\mathbb{N} \times \mathbb{N}$ enforces that. The type is too fine.

A *quotient type* coarsens a type by an equivalence relation: it manufactures a new type in which related elements are literally *equal*. Lean provides this as a primitive, in four moves. We demonstrate all four on the integer example first, because each move will be repeated, beat for beat, on multisets a few pages from now.

8.2.1 The setoid

A type paired with an equivalence relation on it is called a *setoid*. For pre-integers, the relation "same intended difference" can be stated without subtraction (which `Nat` would truncate): $(p_1, n_1) \approx (p_2, n_2)$ iff $p_1 + n_2 = p_2 + n_1$.

```
namespace IntPairs

/-- A "pre-integer": the pair `(p, n)` intends the difference `p - n`. -/
def PreInt := Nat × Nat

/-- Same intended difference (stated without subtraction). -/
def eqv (a b : PreInt) : Prop := a.1 + b.2 = b.1 + a.2

instance preIntSetoid : Setoid PreInt where
  r := eqv
```

```
  iseqv := {
    refl := fun _ => rfl
    symm := fun {a b} h => by
      have h' : a.1 + b.2 = b.1 + a.2 := h
      show b.1 + a.2 = a.1 + b.2
      omega
    trans := fun {a b c} h₁ h₂ => by
      have h₁' : a.1 + b.2 = b.1 + a.2 := h₁
      have h₂' : b.1 + c.2 = c.1 + b.2 := h₂
      show a.1 + c.2 = c.1 + a.2
      omega }
```

The `Setoid` instance is bookkeeping: the relation, plus proofs that it is reflexive, symmetric, and transitive. Once the instance exists, Lean writes `a ≈ b` for `eqv a b`.

8.2.2 The quotient, and `Quotient.mk`

```
/-- The integers: pre-integers, with intent made real by quotienting. -/
def MyInt := Quotient preIntSetoid

def mk (p n : Nat) : MyInt := Quotient.mk preIntSetoid (p, n)
```

`Quotient preIntSetoid` is a genuinely new type. Its elements are built by `Quotient.mk`, which takes a representative and returns its equivalence class. The word *class* is doing real work:

8.2.3 `Quotient.sound`: related representatives, equal classes

```
/-- `(3,5)` and `(10,12)` are *different pairs* but the *same* `MyInt`:
    equality of quotients is `Quotient.sound` of relatedness. -/
example : mk 3 5 = mk 10 12 := Quotient.sound (rfl : 3 + 12 = 10 + 5)
```

Read that twice. The equality sign is Lean's honest, definitional `Eq` — not a bespoke "equivalent-to" predicate we must remember to use. `Quotient.sound` converts a proof that two representatives are related into a proof that their classes are *equal*. Everything Lean knows about equality — rewriting, `congrArg`, substitution into any context — now applies to intent, not representation.

8.2.4 `Quotient.lift`: the toll

There is no free lunch, and here is where the bill arrives. To define a function *out of* a quotient, you define it on representatives — and prove it respects the relation. That proof is the toll `Quotient.lift` charges; refuse to pay and the definition is rejected.

```
/-- To define a function out of a quotient, define it on representatives
    and *prove it respects the relation* — that proof is the toll
    `Quotient.lift` charges. -/
def toInt : MyInt → Int :=
  Quotient.lift (fun a : PreInt => (a.1 : Int) - a.2)
    (fun a b h => by
      have h' : a.1 + b.2 = b.1 + a.2 := h
      show (a.1 : Int) - a.2 = (b.1 : Int) - b.2
      omega)

def neg : MyInt → MyInt :=
  Quotient.lift (fun a : PreInt => mk a.2 a.1)
    (fun a b h => Quotient.sound (by
      have h' : a.1 + b.2 = b.1 + a.2 := h
      show a.2 + b.1 = b.2 + a.1
      omega))
```

Try, as a thought experiment, to lift the function `fun a : PreInt => a.1` — "the first component." The obligation would demand $3 = 10$ from $(3,5) \approx (10,12)$. Unprovable, and rightly so: "the first component" is a fact about representation, not intent, and the quotient will not let you observe it. *The toll is not a formality; it is the entire point.*

8.2.5 `Quotient.ind`: proofs by representative

Finally, to prove a property of *all* elements of a quotient, it suffices to prove it for every `mk a` — `Quotient.ind` hands you a representative.

```
/-- To prove things about all quotient values, `Quotient.ind` hands you
    a representative. -/
theorem toInt_neg (q : MyInt) : toInt (neg q) = - toInt q := by
  induction q using Quotient.ind with
  | _ a =>
      show (a.2 : Int) - a.1 = -((a.1 : Int) - a.2)
      omega

#eval toInt (mk 3 5)              -- -2
#eval toInt (neg (mk 3 5))        -- 2
```

```
-2
2
```

Four moves: `mk` builds, `sound` equates, `lift` computes, `ind` proves. That is the whole API, and you have now used all of it.

8.3 Permutations, Hand-Rolled

The relation we actually need is "same elements, rearranged." Two lists are *permutations* of each other when one can be turned into the other by repeatedly swapping adjacent elements. As an inductive definition:

```
/-- Permutation of lists, hand-rolled (the stdlib has `List.Perm`;
    series rule: understand every axiom, so we build our own). -/
inductive Perm {α : Type} : List α → List α → Prop where
  | nil : Perm [] []
  | cons (x : α) {l₁ l₂ : List α} (h : Perm l₁ l₂) : Perm (x :: l₁) (x :: l₂)
  | swap (x y : α) (l : List α) : Perm (x :: y :: l) (y :: x :: l)
  | trans {l₁ l₂ l₃ : List α} (h₁ : Perm l₁ l₂) (h₂ : Perm l₂ l₃) :
      Perm l₁ l₃
```

The four constructors say: the empty list is a permutation of itself; a common head can be peeled off; adjacent elements may swap; and permutations chain. Transitivity is a *constructor* here, not a theorem — without it, `swap` could only ever disturb the first two positions.

> **The standard library already has this.** Core Lean ships `List.Perm` with the same four constructors and a lemma library around it. We hand-roll ours for the same reason the predecessor hand-rolled `PartialOrder`: this series' rule is to understand every axiom rather than import it. Nothing below would change if you swapped in the library's version.

Reflexivity and symmetry are proofs, each a one-screen induction:

```
theorem refl : ∀ (l : List α), Perm l l
  | [] => .nil
  | x :: t => .cons x (refl t)

theorem symm {l₁ l₂ : List α} (h : Perm l₁ l₂) : Perm l₂ l₁ := by
  induction h with
  | nil => exact .nil
  | cons x _ ih => exact .cons x ih
  | swap x y l => exact .swap y x l
  | trans _ _ ih₁ ih₂ => exact .trans ih₂ ih₁
```

Note the shape of `symm`: induction *on the derivation* of `Perm l₁ l₂`, one case per constructor. You will use this shape twice more before the chapter ends. One more fact, proved the same way, records that permutation forgets arrangement and nothing else:

```
/-- Permutation preserves membership (so it preserves the *set* of
    updates — only arrangement is forgotten). -/
```

```
theorem mem_iff {l₁ l₂ : List α} (h : Perm l₁ l₂) (x : α) :
    x ∈ l₁ ↔ x ∈ l₂
```

With reflexivity, symmetry, and the `trans` constructor in hand, lists under permutation form a setoid, and the multiset is one line:

```
/-- Lists-up-to-permutation form a setoid; `≈` now means `Perm`. -/
instance permSetoid (α : Type) : Setoid (List α) where
  r := Perm
  iseqv := ⟨Perm.refl, Perm.symm, Perm.trans⟩

/-- A finite multiset: lists up to permutation. -/
def MSet (σ : Type) : Type := Quotient (permSetoid σ)

namespace MSet

def ofList {σ : Type} (l : List σ) : MSet σ :=
  Quotient.mk (permSetoid σ) l

/-- Different lists, same multiset. -/
example : ofList [1, 2] = ofList [2, 1] :=
  Quotient.sound (.swap 1 2 [])

end MSet
```

This is the integer story, beat for beat: `PreInt` became `List σ`, `eqv` became `Perm`, `MyInt` became `MSet σ`, and `ofList [1,2] = ofList [2,1]` is the new `mk 3 5 = mk 10 12`.

8.4 The Fold and Its Laws

We want `foldJoin` to descend from lists to multisets. By the rules of Section 8.2, `Quotient.lift` will demand a proof that `foldJoin` respects permutation. This section earns that proof — and, along the way, every fold fact the rest of the book needs.

First, a confession about the definition. `foldJoin` starts its `foldl` at $\bot$, but no proof about a `foldl` ever succeeds by induction if the statement nails the accumulator to a constant: after one step the accumulator is $\bot \sqcup x$, the induction hypothesis no longer applies, and the proof dies. *The trap, announced in advance: state every helper over an arbitrary seed.*

```
/-- The same fold from an arbitrary seed. Generalizing the accumulator
    is what makes every lemma below go through by induction. -/
def foldJoinFrom (a : σ) (l : List σ) : σ := l.foldl (· ⊔ ·) a

theorem foldJoin_def (l : List σ) : foldJoin l = foldJoinFrom ⊥ l := rfl
```

8.4.1 The fold is a least upper bound

Three lemmas pin down exactly what the fold computes. The seed only grows; every member is below the fold; and anything above the seed and all members is above the fold.

```
/-- The seed only grows. -/
theorem le_foldJoinFrom (l : List σ) : ∀ (a : σ), a ⊑ foldJoinFrom a l := by
  induction l with
  | nil => intro a; exact le_refl a
  | cons x t ih => intro a; exact le_trans (le_sup_left a x) (ih (a ⊔ x))

/-- Every delivered update is below the fold: the fold is an upper
    bound of the list. -/
theorem le_foldJoinFrom_of_mem {x : σ} {l : List σ} (hx : x ∈ l) :
    ∀ (a : σ), x ⊑ foldJoinFrom a l := by
  induction l with
  | nil => cases hx
  | cons y t ih =>
      intro a
      cases List.mem_cons.mp hx with
      | inl he =>
          subst he
          exact le_trans (le_sup_right a x) (le_foldJoinFrom t (a ⊔ x))
      | inr hm => exact ih hm (a ⊔ y)

/-- ... and the *least* one over the seed: fold = lub of seed and list. -/
theorem foldJoinFrom_le {c : σ} {l : List σ} : ∀ {a : σ},
    a ⊑ c → (∀ x ∈ l, x ⊑ c) → foldJoinFrom a l ⊑ c := by
  induction l with
  | nil => intro a ha _; exact ha
  | cons y t ih =>
      intro a ha hl
      exact ih (sup_le a y c ha (hl y List.mem_cons_self))
        (fun x hx => hl x (List.mem_cons_of_mem y hx))
```

Watch the accumulator in each proof: it enters the induction hypothesis as `a ⊔ x`, not $\bot$ — exactly the generality the statements bought. Together the three lemmas say:

The fold of a list is the least upper bound of its seed and its members.

That single sentence is the engine of everything below, because "least upper bound of the members" manifestly does not depend on order or multiplicity.

8.4.2 Consequences

Two computation rules follow by antisymmetry from the lub characterization, with no further induction:

```
/-- The seed pulls out of the fold as one more join. -/
theorem foldJoinFrom_eq_sup (a : σ) (l : List σ) :
    foldJoinFrom a l = a ⊔ foldJoin l

/-- One more delivery is one more join. -/
theorem foldJoin_cons (x : σ) (l : List σ) :
    foldJoin (x :: l) = x ⊔ foldJoin l
```

and from those, the fact that makes gossip protocols cheap:

```
/-- Batching is free: delivering a peer's whole log at once is the same
    as joining with the peer's state. Gossip is join-propagation. -/
theorem foldJoin_append (l₁ l₂ : List σ) :
    foldJoin (l₁ ++ l₂) = foldJoin l₁ ⊔ foldJoin l₂ := by
  show (l₁ ++ l₂).foldl (· ⊔ ·) ⊥ = foldJoin l₁ ⊔ foldJoin l₂
  rw [List.foldl_append]
  exact foldJoinFrom_eq_sup (foldJoin l₁) l₂
```

A replica that merges a peer's whole state has, in one join, done the work of delivering the peer's entire log — because appending logs and joining states are the same operation. This is why Part IV's anti-entropy protocols ship states, not update-by-update transcripts: *batching is free.* (Exercise 8.1 asks you to reprove this with your own hands.)

8.4.3 The keystone

Now the lemma the theorem stands on. Suppose two lists have the same members — each element of one occurs in the other, with no constraint at all on positions or multiplicities. You might expect the proof to massage one list into the other: sort them, deduplicate them, induct on something clever. Resist the urge.

Both folds are least upper bounds of the same set of elements, and least upper bounds are unique — so antisymmetry finishes it in four lines.

```
/-- **The keystone.** Same members — any order, any multiplicities —
    same fold. Not proved by list surgery: both folds are least upper
    bounds of the same set, so antisymmetry finishes it. -/
theorem foldJoin_eq_of_same_mems {l₁ l₂ : List σ}
    (h : ∀ x, x ∈ l₁ ↔ x ∈ l₂) : foldJoin l₁ = foldJoin l₂ := by
  apply le_antisymm
  · exact foldJoinFrom_le (bot_le _)
```

```
      (fun x hx => le_foldJoinFrom_of_mem ((h x).mp hx) ⊥)
  · exact foldJoinFrom_le (bot_le _)
      (fun x hx => le_foldJoinFrom_of_mem ((h x).mpr hx) ⊥)
```

Each fold is $\sqsubseteq$ the other, because each is a least upper bound of members the other also bounds. This is the same antisymmetry move that proved `sup_unique` in the predecessor and `sorted_ext` in Chapter 5; by now it should feel like an old friend with a new coat.

8.4.4 Two roads to permutation-invariance

Since permutations preserve membership (`Perm.mem_iff`), the keystone immediately gives `foldJoin l₁ = foldJoin l₂` whenever `Perm l₁ l₂`. But it is worth also proving the fact *directly*, by induction on the permutation derivation, because that is the exact shape `Quotient.lift` asks about and the exercise is instructive:

```
/-- Reordering is absorbed (direct proof by induction on the
    permutation — the shape `Quotient.lift` will ask for). -/
theorem foldJoinFrom_perm {l₁ l₂ : List σ} (h : Perm l₁ l₂) :
    ∀ (a : σ), foldJoinFrom a l₁ = foldJoinFrom a l₂ := by
  induction h with
  | nil => intro _; rfl
  | cons x _ ih => intro a; exact ih (a ⊔ x)
  | swap x y l =>
      intro a
      show foldJoinFrom (a ⊔ x ⊔ y) l = foldJoinFrom (a ⊔ y ⊔ x) l
      rw [sup_assoc, sup_assoc, sup_comm x y]
  | trans _ _ ih₁ ih₂ => intro a; exact (ih₁ a).trans (ih₂ a)

theorem foldJoin_perm {l₁ l₂ : List σ} (h : Perm l₁ l₂) :
    foldJoin l₁ = foldJoin l₂ :=
  foldJoinFrom_perm h ⊥
```

Look at which algebraic law each case consumes. The `swap` case is commutativity and associativity, verbatim; `cons` is the seed generalization earning its keep once more; `trans` is transitivity of `Eq`. And where is idempotence? Nowhere — permutations never duplicate. Idempotence is a different sin's antidote:

```
/-- Duplication is absorbed: idempotence, on duty. -/
theorem foldJoin_dup (x : σ) (l : List σ) :
    foldJoin (x :: x :: l) = foldJoin (x :: l) := by
  show foldJoinFrom (⊥ ⊔ x ⊔ x) l = foldJoinFrom (⊥ ⊔ x) l
  rw [sup_assoc, sup_idem]
```

8.4.5 The fold descends

The toll is paid; the lift is free:

```
/-- The fold descends to multisets: `Quotient.lift` demands exactly
    `foldJoin_perm`, and Chapter 8 has already paid it. -/
def foldJoin {σ : Type} [BoundedJoinSemilattice σ] : MSet σ → σ :=
  Quotient.lift SEC.foldJoin (fun _ _ h => SEC.foldJoin_perm h)

theorem foldJoin_mk {σ : Type} [BoundedJoinSemilattice σ] (l : List σ) :
    foldJoin (ofList l) = SEC.foldJoin l := rfl
```

Notice what just happened, because it is the pedagogical heart of the chapter: your first `Quotient.lift` in production went through *trivially*, and it went through trivially *because* the hard work was a fold lemma you had independent reasons to want. That is what quotients feel like when they are used well — the proof obligation is not an obstacle, it is a checklist item you had already planned to prove.

An honest word about axioms. Run `#print axioms` on anything that touches `MSet` and Lean reports `Quot.sound`: the rule that related representatives give equal classes is an *axiom* of Lean's kernel, not a theorem. It joins `propext` (propositional extensionality, which our `funext`-style reasoning has used since the predecessor). The audit block at the end of `Crdt.lean` prints the axioms of every headline theorem in this book: the answer is `propext` and `Quot.sound` and nothing else. In particular `Classical.choice` appears nowhere — the entire chain, main theorem included, is constructive.

8.5 The Main Theorem

The pieces assemble in six lines. A replica, for the purposes of the theorem, *is* what it has received; "the same news" means the same delivered set — any order, any multiplicities at least one.

```
/-- A replica, abstracted to what it has received. -/
structure Replica (σ : Type) [BoundedJoinSemilattice σ] where
  delivered : List σ

def Replica.state (r : Replica σ) : σ := foldJoin r.delivered

/-- Two replicas have received the same *set* of updates — in any
    order, with any multiplicities ≥ 1. -/
def SameDeliveredSet (r₁ r₂ : Replica σ) : Prop :=
  ∀ x, x ∈ r₁.delivered ↔ x ∈ r₂.delivered
```

Strong eventual consistency, state-based

Let σ be a bounded join-semilattice and let r_1, r_2 be replicas whose delivered lists contain the same set of updates. Then r_1 and r_2 hold equal state.

```
/-- **Strong eventual consistency, state-based.** Replicas that have
    heard the same news hold the same state — no matter who told them,
    how often, or in what order. -/
theorem sec_state_based {r₁ r₂ : Replica σ}
    (h : SameDeliveredSet r₁ r₂) : r₁.state = r₂.state :=
  foldJoin_eq_of_same_mems h
```

The proof term is a single identifier. Every ounce of content lives in the keystone, which lives in the lub lemmas, which live in the semilattice axioms — the theorem is the algebra of join, and nothing else.

Notice what the theorem does *not* say. It does not say replicas agree at every moment — they demonstrably do not. It does not say the state they agree on is the one your users wanted — Chapter 13 is about exactly that gap. It says something narrower and, once you have operated a system without it, more precious: replicas that have heard the same news hold the same state, no matter who told them, how often, or in what order.

8.6 One Law per Sin

The proof consumed commutativity, associativity, and idempotence, and it is worth asking whether all three were needed. They were; here is the demonstration for idempotence. Take a structure with a commutative, associative merge that is *not* idempotent — $(\mathbb{N}, +)$ is the classic — and let the network deliver one update twice.

Refuted 8.1 — Commutativity and associativity alone suffice

Addition on `Nat` is commutative and associative. Fold the single update `1` once, then fold it delivered twice:

```
/-- Refuted: "commutativity and associativity alone suffice".
    Drop idempotence — `(Nat, +)` is commutative and associative —
    and one duplicated delivery double-counts. -/
theorem comm_assoc_not_enough :
    [1].foldl (· + ·) 0 ≠ [1, 1].foldl (· + ·) 0 := by decide
```

Same delivered *set*, different states: $1 \neq 2$. Without idempotence, "how often" leaks into the answer, and strong eventual consistency is false.

Each of the three laws is one network sin's antidote, and the proof of `foldJoinFrom_perm` together with `foldJoin_dup` and `foldJoin_append` shows the pairing exactly:

The network's sin	The law that absorbs it	Where it worked
reordering	commutativity + associativity	`foldJoinFrom_perm`
duplication	idempotence	`foldJoin_dup`
batching	associativity	`foldJoin_append`

Table 8.1: Three sins, three laws. Message *loss* is absent: no algebra can conjure an update a replica never received. Loss is repaired by delivery protocols (Chapter 9), not by the merge.

8.7 Computation Interlude: Same News, Any Postal Service

Three replicas of a G-Set receive the same three updates — as one, five, and six deliveries respectively, in three different orders.

```
/-- Interlude: three G-Set update streams folded in wildly different
    orders and multiplicities. -/
def interlude : List (String × List Nat) :=
  let u₁ : GSet := GSet.add 1 ⊥
  let u₂ : GSet := GSet.add 2 ⊥
  let u₃ : GSet := GSet.add 3 ⊥
  let r₁ : Replica GSet := ⟨[u₁, u₂, u₃]⟩
  let r₂ : Replica GSet := ⟨[u₃, u₁, u₂, u₂, u₁]⟩
  let r₃ : Replica GSet := ⟨[u₂, u₂, u₃, u₃, u₁, u₃]⟩
  [ ("replica 1", r₁.state.val),
    ("replica 2", r₂.state.val),
    ("replica 3", r₃.state.val) ]

#eval interlude
```

```
[("replica 1", [1, 2, 3]), ("replica 2", [1, 2, 3]), ("replica 3", [1, 2, 3])]
```

Five chapters of interludes showed this pattern experimentally. This time it is not an experiment: `sec_state_based` applies to all three pairs, because all three delivered lists have the same members. The output could not have been otherwise.

8.8 Exercises

Exercise 8.1 — Batching, by hand

★ Prove `foldJoin_append` yourself, without looking at the companion's proof: first show `(l₁ ++ l₂).foldl (· ⊔ ·) a = foldJoinFrom (foldJoinFrom a l₁) l₂` (or find the core lemma that says it), then finish with `foldJoinFrom_eq_sup`. Where exactly does associativity enter?

Exercise 8.2 — Multiset union is well-defined

★★ Define `MSet.union : MSet α → MSet α → MSet α` by lifting list append with `Quotient.lift₂`. The toll this time: if `Perm l₁ l₂` and `Perm m₁ m₂` then `Perm (l₁ ++ m₁) (l₂ ++ m₂)`. Prove it in two halves (append on the right of a fixed list, then on the left) and chain them. A full solution is in Appendix B.

Exercise 8.3 — The converse question

★★★ This chapter proved that ACI laws make folds delivery-insensitive. Is the converse true? Suppose `m : σ → σ → σ` and `e : σ` are such that `l.foldl m e` depends only on the set of members of `l`, for *all* lists `l`. Must `m` be commutative, associative, and idempotent on the reachable elements? Formulate the statement precisely (the phrase "on the reachable elements" is doing real work), then prove or refute it. Hints in Appendix B.

Chapter 9

Delivery: Gossip, Duplication, and Reordering

> *"The network is reliable."*
> *— the first of Peter Deutsch's eight fallacies of distributed computing (1994)*

Chapter 8 proved the main theorem over an abstraction: a replica *is* a list of delivered updates, and delivery itself happened offstage. That was the right altitude for the algebra, but a systems reader is entitled to ask where those lists come from, and whether a real network — one that reorders, duplicates, and drops — actually produces the hypothesis `SameDeliveredSet` the theorem consumes.

This chapter builds the network. Twice, in fact. First as an inductive *trace semantics* — a relation describing every execution an adversarial at-least-once network could produce — over which we prove that eventual delivery implies convergence. Then as an *executable driver* whose termination Lean will not accept on faith: the chapter where well-founded recursion stops being a curiosity from the predecessor's appendix and becomes the tool that pays off a debt.

9.1 An Honest Network

A configuration is a snapshot of the whole system: what each replica holds, what each replica has incorporated (a ghost log — present for the proof, invisible to the protocol), and the messages in flight. A message is addressed to a replica and carries a state together with the log that produced it.

```
namespace Delivery

variable {R : Nat} {σ : Type} [BoundedJoinSemilattice σ]

/-- A snapshot of the whole system: each replica's state, the (ghost)
    log of updates it has incorporated, and the messages in flight.
    A message carries a state and the log that produced it. -/
structure Config (R : Nat) (σ : Type) where
  states : Fin R → σ
  seen   : Fin R → List σ
  msgs   : List (Fin R × σ × List σ)
```

```
12
13 /-- The empty system. -/
14 def init (R : Nat) (σ : Type) [BoundedJoinSemilattice σ] : Config R σ :=
15   { states := fun _ => ⊥, seen := fun _ => [], msgs := [] }
```

Three kinds of event can occur, each a pure function on configurations:

```
1  def applyUpdate (c : Config R σ) (r : Fin R) (v : σ) : Config R σ :=
2    { states := fun j => if j = r then c.states j ⊔ v else c.states j
3      seen   := fun j => if j = r then v :: c.seen j else c.seen j
4      msgs   := c.msgs }
5
6  def applySend (c : Config R σ) (r r' : Fin R) : Config R σ :=
7    { c with msgs := (r', c.states r, c.seen r) :: c.msgs }
8
9  def applyDeliver (c : Config R σ) (r : Fin R) (p : σ × List σ) : Config R σ :=
10   { states := fun j => if j = r then c.states j ⊔ p.1 else c.states j
11     seen   := fun j => if j = r then p.2 ++ c.seen j else c.seen j
12     msgs   := c.msgs }
```

An `update` at replica r joins a new update-state v into r's state and records it in r's log. A `send` photographs r's current state-and-log and posts it to r'. A `deliver` merges an in-flight message into its addressee — the whole state in one join, the whole log in one append, which Chapter 8's `foldJoin_append` priced at zero.

The step relation is where the network's character lives:

```
1  /-- The network's honest contract. `deliver` picks *any* in-flight
2      message (reordering) and leaves it in flight (duplication); a
3      message that is never picked is lost. At-least-once, unordered. -/
4  inductive Step : Config R σ → Config R σ → Prop where
5    | update (c : Config R σ) (r : Fin R) (v : σ) :
6        Step c (applyUpdate c r v)
7    | send (c : Config R σ) (r r' : Fin R) :
8        Step c (applySend c r r')
9    | deliver (c : Config R σ) (r : Fin R) (p : σ × List σ)
10       (h : (r, p) ∈ c.msgs) :
11       Step c (applyDeliver c r p)
12
13 /-- Zero or more steps. -/
14 inductive Reachable : Config R σ → Config R σ → Prop where
15   | refl (c : Config R σ) : Reachable c c
16   | tail {c₁ c₂ c₃ : Config R σ} (h : Reachable c₁ c₂) (hs : Step c₂ c₃) :
17       Reachable c₁ c₃
```

Read the `deliver` constructor with suspicion, because its two omissions are the entire

model. Delivery requires only membership in `c.msgs` — any in-flight message, in any order relative to when it was sent: *reordering*. And the delivered message is not removed from `c.msgs` — it may be picked again, and again: *duplication, redelivery*. Loss needs no constructor at all: a message that no execution step ever picks is simply never heard from. This is an at-least-zero-times network dressed in at-least-once clothing, and it is the weakest delivery contract anyone ships software on.

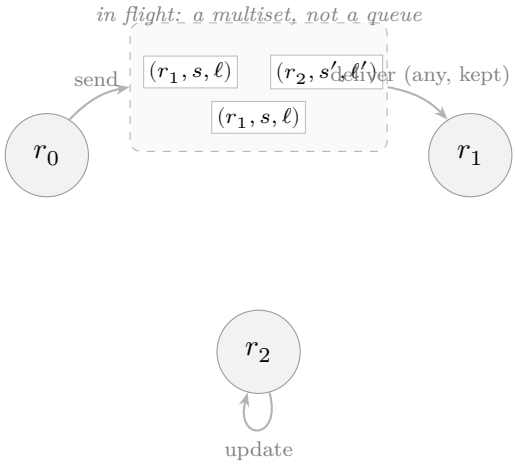

Figure 9.1: The delivery model. Messages in flight form a multiset: `deliver` may pick any element (reordering), the element stays in flight afterwards (duplication), and an element never picked is lost. The same message can appear twice because `send` can run twice.

9.2 The Invariant

To apply Chapter 8's theorem we must connect the *states* the protocol maintains to the *logs* the theorem speaks about. The connection is an invariant — every replica's state is the fold of its log, and every in-flight message tells the truth about itself — and the proof style is the one you met with `WfORSet` in Chapter 7: establish it at the initial configuration, show every step preserves it, conclude it holds along every execution.

```
/-- The inductive invariant: every replica's state is the fold of its
    log, and every in-flight message is honest about its own. -/
def Coherent (c : Config R σ) : Prop :=
  (∀ r, c.states r = SEC.foldJoin (c.seen r)) ∧
  (∀ m ∈ c.msgs, m.2.1 = SEC.foldJoin m.2.2)

theorem coherent_init : Coherent (init R σ) :=
  ⟨fun _ => rfl, fun _ hm => absurd hm (by intro h; cases h)⟩
```

Preservation is one case per constructor, and each case spends exactly one Chapter 8 lemma. An `update` conses onto the log and joins onto the state: `foldJoin_cons` says those agree. A `send` copies a replica's state and log into a message: the replica's own

invariant is precisely the message's honesty. A `deliver` appends a log and joins a state: `foldJoin_append` again.

```
/-- Every step preserves coherence — one case per event. -/
theorem coherent_step {c₁ c₂ : Config R σ} (h : Step c₁ c₂)
    (hc : Coherent c₁) : Coherent c₂ := by
  cases h with
  | update r v =>
      refine ⟨fun j => ?_, hc.2⟩
      show (if j = r then c₁.states j ⊔ v else c₁.states j)
          = SEC.foldJoin (if j = r then v :: c₁.seen j else c₁.seen j)
      by_cases hj : j = r <;> simp [hj]
      · rw [hc.1 r, SEC.foldJoin_cons, Order.sup_comm]
      · exact hc.1 j
  | send r r' =>
      refine ⟨hc.1, fun m hm => ?_⟩
      cases List.mem_cons.mp hm with
      | inl he => subst he; exact hc.1 r
      | inr hm' => exact hc.2 m hm'
  | deliver r p hp =>
      refine ⟨fun j => ?_, hc.2⟩
      show (if j = r then c₁.states j ⊔ p.1 else c₁.states j)
          = SEC.foldJoin (if j = r then p.2 ++ c₁.seen j else c₁.seen j)
      by_cases hj : j = r <;> simp [hj]
      · rw [hc.1 r, SEC.foldJoin_append, hc.2 (r, p) hp, Order.sup_comm]
      · exact hc.1 j

theorem coherent_reachable {c₁ c₂ : Config R σ} (h : Reachable c₁ c₂)
    (hc : Coherent c₁) : Coherent c₂ := by
  induction h with
  | refl => exact hc
  | tail _ hs ih => exact coherent_step hs ih
```

The mechanical texture of this proof is the point. An inductive invariant proof is long the way a phone book is long — many entries, no plot twists — and designing the `Config` so it stays that way is a skill. Here the design choice was to keep `seen` a plain list per replica and compare at the level of *folds*, never demanding equality of lists or multisets inside the invariant; the quotient stays where it belongs, in the statement of the final theorem's hypothesis.

9.3 Eventual Delivery Means Convergence

The final hypothesis is the operational reading of "the same news": every update anyone has incorporated, everyone has.

```
/-- "Everything anyone has incorporated, everyone has" — the quiescent,
    all-delivered endpoint of an execution. -/
def AllShared (c : Config R σ) : Prop :=
  ∀ (r r' : Fin R), ∀ x ∈ c.seen r, x ∈ c.seen r'
```

Eventual delivery ⇒ convergence

Along any execution from the empty system — updates, sends, and deliveries interleaved in any order, with any duplication and any loss — every configuration in which all incorporated updates are fully shared has all replicas in equal state.

```
theorem converged_of_allShared {c : Config R σ}
    (hr : Reachable (init R σ) c) (h : AllShared c) :
    ∀ (r r' : Fin R), c.states r = c.states r' := by
  intro r r'
  have hc := coherent_reachable hr coherent_init
  rw [hc.1 r, hc.1 r']
  exact SEC.foldJoin_eq_of_same_mems (fun x => ⟨h r r' x, h r' r x⟩)
```

Four lines: fetch the invariant, rewrite both states into folds of logs, and hand the shared-membership fact to Chapter 8's keystone. The trace semantics contributed the invariant; the algebra contributed everything else. That division of labor — operational reasoning establishes *what was received*, algebra establishes *what state that forces* — is the reusable pattern, and you will see it again in the capstone.

9.4 Paying Off the Fuel: A Terminating Gossip Driver

The theorem above is about executions someone else chooses. An *anti-entropy protocol* chooses its own: repeatedly find a replica that knows something a peer lacks, and make the peer pull. The predecessor's `runToFixpoint` did the analogous job for propagator networks with a `while changed` loop in `IO` — morally a fuel parameter, spent one unit per round, with termination taken on faith.

Fuel was an IOU. This section pays it.

The claim to be earned is that gossip *cannot go on forever*: each productive exchange strictly shrinks something finite. The something is a natural-number measure — how many (update, replica) pairs are still missing — and once Lean believes the measure strictly decreases, it accepts the recursion with no fuel in sight. First, the executable network view and the measure:

```
variable {R : Nat} {σ : Type} [DecidableEq σ]

```

```
/-- Executable network view: the set of updates ever issued, and each
    replica's log. -/
structure Net (R : Nat) (σ : Type) where
  issued : List σ
  logs : Fin R → List σ

/-- How many issued updates replica `r` still lacks. -/
def missing (n : Net R σ) (r : Fin R) : Nat :=
  (n.issued.filter (fun u => !decide (u ∈ n.logs r))).length

/-- The termination measure: total lacks, summed over replicas. -/
def cost (n : Net R σ) : Nat :=
  (List.finRange R).foldl (fun acc r => acc + missing n r) 0
```

The scheduler scans for a productive pair — some p knowing an issued update that some q lacks — and an exchange makes q merge p's whole log (batching is free; Chapter 8 said so):

```
/-- Does `p` know an issued update that `q` lacks? -/
def needsFrom (n : Net R σ) (p q : Fin R) : Bool :=
  n.issued.any (fun u => decide (u ∈ n.logs p) && !decide (u ∈ n.logs q))

/-- Scan for a productive exchange; `none` means quiescent. -/
def findPair (n : Net R σ) : Option (Fin R × Fin R) :=
  findPairAux n (allPairs R)

/-- One anti-entropy exchange: `q` merges `p`'s whole log.
    (Batching is free — Chapter 8's `foldJoin_append`.) -/
def exchange (p q : Fin R) (n : Net R σ) : Net R σ :=
  { n with logs := fun r => if r = q then n.logs p ++ n.logs r else n.logs r }
```

(`findPairAux` is a plain first-match scan over `allPairs R`, the list of all replica pairs; the companion proves the two facts you would expect: a returned pair really is productive, and a `none` means no pair is.)

9.4.1 The measure decreases

The decreasing argument splits into filter arithmetic and sum arithmetic. The filter half: shrinking what a predicate accepts can only shrink a filtered length, and if some list element flips from accepted to rejected, it strictly shrinks it. Both are inductions on the list (`filter_length_mono` and `filter_length_strict` in the companion). The sum half is our old friend the seed generalization: sums over `finRange` with pointwise-smaller summands are smaller, and strictly so if any single index is strictly smaller (`foldl_add_lt_of_seed`, `foldl_add_strict` — both stated over arbitrary seeds `a ≤ b`, or the induction would die exactly as Chapter 8 warned).

Stacked together:

```
/-- After an exchange, nobody lacks more than before. -/
theorem missing_exchange_le (n : Net R σ) (p q r : Fin R) :
    missing (exchange p q n) r ≤ missing n r

/-- ... and the receiver of a productive exchange lacks strictly less. -/
theorem missing_exchange_lt {n : Net R σ} {p q : Fin R} {u : σ}
    (hu : u ∈ n.issued) (hup : u ∈ n.logs p) (huq : ¬ u ∈ n.logs q) :
    missing (exchange p q n) q < missing n q

/-- A productive exchange strictly shrinks the measure. -/
theorem cost_exchange_lt {n : Net R σ} {p q : Fin R}
    (h : needsFrom n p q = true) : cost (exchange p q n) < cost n := by
  obtain ⟨u, hu, hup, huq⟩ := needsFrom_iff.mp h
  unfold cost
  exact foldl_add_strict R _ _ q (missing_exchange_le n p q)
    (missing_exchange_lt hu hup huq) (Nat.le_refl 0)
```

9.4.2 The driver

Now the recursion itself — the loop the predecessor could only run on fuel — accepted by Lean because the debt is paid:

```
/-- **The driver.** Exchange while a productive pair exists. Lean
    accepts the recursion because `cost` strictly decreases — the
    predecessor's fuel, paid off. -/
def gossipUntilQuiescent (n : Net R σ) : Net R σ :=
  match _h : findPair n with
  | none => n
  | some pq => gossipUntilQuiescent (exchange pq.1 pq.2 n)
termination_by cost n
decreasing_by
  exact cost_exchange_lt (findPairAux_spec _h)
```

Three things deserve a slow look. `termination_by cost n` names the measure. `decreasing_by` discharges the goal Lean generates at the recursive call — that the argument's measure is smaller — using the lemma chain above. And the `match _h : findPair n` syntax (a *discriminant equation*) captures the proof `_h : findPair n = some pq` that the decreasing argument needs: the recursion is only taken when the scan succeeded, and the scan succeeding is exactly the productivity fact `cost_exchange_lt` consumes.

The predecessor's `runToFixpoint` took fuel because we could not yet say *why* propagation stops. Here we can: the measure is the reason, stated as a number and spent like one.

Termination is not the only dividend. Quiescence — the `none` branch — now has a proved meaning:

```
/-- Quiescence means fully shared: every issued update known anywhere
    is known everywhere. -/
theorem quiescent_shares {n : Net R σ} (h : findPair n = none) :
    ∀ (p q : Fin R), ∀ u ∈ n.issued, u ∈ n.logs p → u ∈ n.logs q
```

which is precisely the `AllShared` hypothesis of the convergence theorem, delivered by construction rather than by assumption.

Why is the driver deterministic? The relational semantics earlier in the chapter let the *adversary* choose every step — that is where reordering, duplication, and loss live, and the convergence theorem quantifies over all of the adversary's choices. The driver, by contrast, is *our* code: it may as well scan deterministically, because a deterministic scan makes the measure argument local (one exchange, one strictly smaller number). The adversarial schedules return in the interlude below — as a seeded generator whose chaos runs *before* the verified driver mops up. The IO simulation there remains fuel-bounded like `runToFixpoint`, and honestly so: an adversary who never delivers can stall any protocol forever; the theorem covers those executions through the relational semantics, not by promising the impossible.

9.5 Computation Interlude: Chaos, then Quiescence

Four replicas, six updates, and a network simulated by a linear congruential generator that drops a third of the exchanges and duplicates a fifth of them:

```
/-- A linear congruential generator: cheap, deterministic chaos. -/
def lcgNext (s : Nat) : Nat := (s * 1103515245 + 12345) % 2147483648

/-- Four replicas; each initially knows only its own updates. -/
def demoNet : Net 4 GSet :=
  let u (k : Nat) : GSet := GSet.add k ⊥
  { issued := [u 1, u 2, u 3, u 4, u 10, u 20]
    logs := fun r =>
      if r = 0 then [u 1, u 10]
      else if r = 1 then [u 2, u 20]
      else if r = 2 then [u 3]
      else [u 4] }

/-- `fuel` random exchange attempts: some dropped, some duplicated,
    directions and targets scrambled by the seed. -/
def randomGossip : Nat → Nat → Net 4 GSet → Net 4 GSet
  | 0, _, n => n
  | fuel + 1, seed, n =>
```

```
      let s₁ := lcgNext seed
      let s₂ := lcgNext s₁
      let s₃ := lcgNext s₂
      let p : Fin 4 := ⟨s₁ % 4, Nat.mod_lt _ (by omega)⟩
      let q : Fin 4 := ⟨s₂ % 4, Nat.mod_lt _ (by omega)⟩
      let n₁ := if s₃ % 3 = 0 then n else exchange p q n      -- drop 1 in 3
      let n₂ := if s₃ % 5 = 0 then exchange p q n₁ else n₁    -- duplicate 1
       ↪ in 5
      randomGossip fuel s₃ n₂

def renderNet (n : Net 4 GSet) : List (List Nat) :=
  (List.finRange 4).map (fun r => (SEC.foldJoin (n.logs r)).val)

/-- Chaos first, verified anti-entropy after: for any seed, the driver
    quiesces and all replicas agree. -/
def simulate (seed : Nat) : List (List Nat) × List (List Nat) :=
  let chaotic := randomGossip 12 seed demoNet
  (renderNet chaotic, renderNet (gossipUntilQuiescent chaotic))

#eval simulate 1
#eval simulate 42
#eval simulate 2026
```

```
([[1, 2, 3, 4, 10, 20], [1, 2, 3, 4, 10, 20], [1, 2, 3, 10, 20], [1, 2, 3, 4,
 ↪ 10, 20]],
 [[1, 2, 3, 4, 10, 20], [1, 2, 3, 4, 10, 20], [1, 2, 3, 4, 10, 20], [1, 2, 3,
  ↪ 4, 10, 20]])
([[1, 2, 3, 4, 10, 20], [1, 2, 3, 4, 10, 20], [1, 2, 3, 4, 10, 20], [2, 3, 4,
 ↪ 20]],
 [[1, 2, 3, 4, 10, 20], [1, 2, 3, 4, 10, 20], [1, 2, 3, 4, 10, 20], [1, 2, 3,
  ↪ 4, 10, 20]])
([[1, 3, 4, 10], [1, 2, 4, 10, 20], [1, 2, 3, 4, 10, 20], [1, 2, 3, 4, 10,
 ↪ 20]],
 [[1, 2, 3, 4, 10, 20], [1, 2, 3, 4, 10, 20], [1, 2, 3, 4, 10, 20], [1, 2, 3,
  ↪ 4, 10, 20]])
```

Each pair shows the four replica states after the chaotic phase (left) and after the verified driver (right). The chaos differs with the seed — with seed 1, replica 2 is still missing update 4; with seed 42, replica 3 is missing two updates; with seed 2026, two replicas lag — but the right-hand column never varies. It cannot: the driver terminates (the measure said so), quiescence means fully shared (`quiescent_shares` said so), and fully shared means equal states (Chapter 8 said so).

9.6 Exercises

Exercise 9.1 — Disagreement before quiescence

★ Exhibit a concrete `Reachable (init 2 GSet) c` whose two replica states differ. (Two steps suffice. This is why the convergence theorem needs its `AllShared` hypothesis — replicas demonstrably do not agree at every moment.)

Exercise 9.2 — Replicas never forget

★★ Prove that `seen` is monotone along executions: if `Reachable c₁ c₂` then every element of `c₁.seen r` is an element of `c₂.seen r`. Induct on reachability, then case on the step; the `deliver` case needs nothing more than `List.mem_append`. Full solution in Appendix B.

Exercise 9.3 — Convergence under loss, with fairness

★★★ The convergence theorem's hypothesis `AllShared` is discharged by the driver in a lossless world. Formulate an *eventual delivery* fairness hypothesis for infinite executions — "every update incorporated somewhere is eventually incorporated everywhere" — as a property of infinite sequences `Nat → Config R σ` of `Step`-related configurations, and show it implies that states agree from some index on for any two replicas whose logs stop growing. Hints in Appendix B.

Chapter 10

Time Without Clocks: Version Vectors and Causality

> *"The concept of one event happening before another in a distributed system is examined, and is shown to define a partial ordering of the events."*
> — *Leslie Lamport,* Time, Clocks, and the Ordering of Events in a Distributed System *(1978)*

Chapter 6 bought overwrite semantics by reifying time into the state — and then admitted, in a note, that its timestamps were logical tokens with no honest relationship to wall clocks. This chapter builds the honest version. The remarkable thing is that we do not need any new machinery to do it: the data structure that tracks causality in a distributed system is one we verified in Chapter 3, wearing a different meaning.

10.1 The G-Counter, Read Again

A *version vector* records, for each replica, how many of that replica's events have been seen. Its carrier is $\mathrm{Fin}\, R \to \mathbb{N}$; its merge is pointwise maximum (having seen event 7 of replica i from one source and event 5 from another, you have seen seven); its bottom is the vector of zeroes. You have verified all of this already. Literally:

```
/-- A version vector: how many events of each replica have been seen. -/
abbrev VV (R : Nat) := GCounter R

namespace VV

/-- Replica `i` performs an event: tick its own entry.
    (`tick` *is* `increment`; the meaning changed, not the code.) -/
def tick (i : Fin R) (v : VV R) : VV R := GCounter.increment i v

theorem tick_inflationary (i : Fin R) : Order.Inflationary (tick (R := R) i)
  :=
  GCounter.increment_inflationary i
```

An `abbrev` and a renaming. The semilattice instance, the inflationarity of `tick`, the decidability of the order — all of it flows in from Chapter 3 without a single new proof.

This is the cleanest instance in the book of a phenomenon worth naming: a CRDT about CRDTs. The version vector is itself a state-based CRDT (it is a G-Counter), and its *job* is to describe the causal structure of other CRDTs' updates.

What changed is the reading of the order. For counters, $g \sqsubseteq h$ meant "h counts at least as much." For version vectors:

$v \sqsubseteq w$ means *w has seen everything v has seen.*

```
theorem vv_le_iff {v w : VV R} : v ⊑ w ↔ ∀ i, v i ≤ w i := Iff.rfl
```

(The proof is `Iff.rfl`: the information order on functions was already pointwise. The theorem exists to be cited, not to be hard.)

10.2 Happens-Before and Concurrency

With that reading, the causal vocabulary is a pair of definitions:

```
/-- Causal order: `v` happened before `w` when `w` has seen everything
    `v` has, and more. -/
def happensBefore (v w : VV R) : Prop := v ⊑ w ∧ v ≠ w

/-- Neither saw the other: genuinely concurrent. -/
def Concurrent (v w : VV R) : Prop := ¬ v ⊑ w ∧ ¬ w ⊑ v
```

`happensBefore` is Lamport's relation, computed from state: v happened before w when w's view strictly dominates v's. It is a strict partial order, and the proofs are the predecessor's order algebra on its feet:

```
theorem happensBefore_irrefl (v : VV R) : ¬ happensBefore v v :=
  fun h => h.2 rfl

theorem happensBefore_trans {u v w : VV R}
    (h₁ : happensBefore u v) (h₂ : happensBefore v w) :
    happensBefore u w := by
  refine ⟨le_trans h₁.1 h₂.1, fun he => ?_⟩
  subst he
  exact h₂.2 (le_antisymm h₂.1 h₁.1)

theorem concurrent_symm {v w : VV R} (h : Concurrent v w) :
    Concurrent w v := ⟨h.2, h.1⟩
```

Partial is the operative word, and it is not a defect to be engineered away:

Refuted 10.1 — Version vectors totally order events

Let two replicas each perform one event from the empty history. On `Fin 2`, the resulting vectors are $(1, 0)$ and $(0, 1)$: neither dominates the other, in either direction.

```
/-- Refuted: "version vectors totally order events". Two ticks at
    different replicas are comparable in neither direction. Concurrency
    is not a failure to know; it is a fact about the world. -/
theorem ticks_concurrent :
    Concurrent (tick 0 (⊥ : VV 2)) (tick 1 (⊥ : VV 2)) := by decide
```

No refinement of the data structure can repair this, because there is nothing to repair: the two events genuinely happened with neither knowing of the other. Chapter 6 *chose* to arbitrate such pairs (with replica-id tiebreaks); the OR-Set of Chapter 7 *chose* add-wins. The version vector's contribution is to identify exactly which pairs require a choice.

Concurrency is not a failure to know; it is a fact about the world.

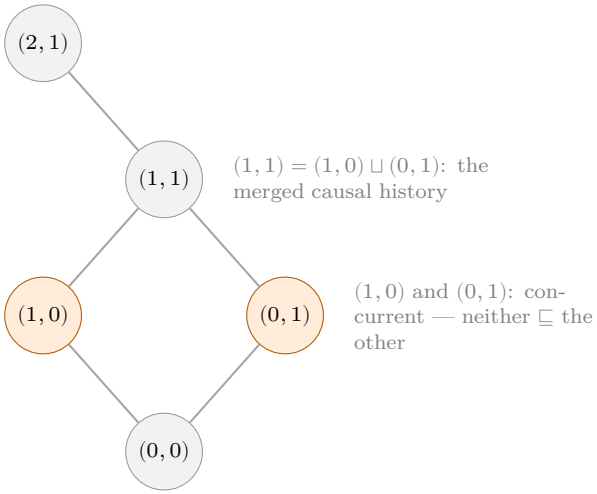

Figure 10.1: A fragment of the version-vector order for two replicas. Causal history grows upward; the highlighted pair is concurrent. The join of two histories is the least history containing both — merging version vectors is joining causal pasts.

The join has a causal reading too: $v \sqcup w$ is the least history that has seen everything v or w saw. Exercise 10.2 makes that precise; the companion records the easy half as `merge_seen_left`.

10.3 Causal Delivery

The payoff that Part IV will spend: version vectors let a replica decide *when a message is safe to apply*. Stamp each message from replica i with the sender's vector; the receiver applies it only when it is the very next event of i and every other dependency is already

seen.

```
/-- Causal delivery: replica `recv` may apply the message stamped `s`
    from replica `i` when it is the very next event of `i` and every
    other dependency is already seen. -/
def deliverable (i : Fin R) (s recv : VV R) : Prop :=
  s i = recv i + 1 ∧ ∀ j, j ≠ i → s j ≤ recv j
```

A buffer of undeliverable messages plus this predicate is the classic *causal broadcast* construction: messages may arrive in any order, but they are *applied* in an order consistent with happens-before. Three earlier threads now tie off. The logical clocks that Chapter 6 promised are these: replace wall-time stamps with vector stamps and ties become exactly the concurrent pairs, the ones a register must arbitrate by policy. The optimized OR-Set of Exercise 7.3 becomes approachable: version vectors summarize which tags a replica has observed, collapsing tombstone sets. And Chapter 11's operation-based CRDTs will state causal delivery as a standing assumption — the transport obligation that replaces the join algebra.

10.4 Computation Interlude: Causal Facts, Decided

Because the order on version vectors is decidable (a `Decidable` instance for $\sqsubseteq$ on `Fin R → Nat` lands in the companion beside this chapter), causal claims about concrete histories are `decide`-checkable — and evaluable:

```
/-- Interlude: causal facts, decided. -/
def interlude : List (String × Bool) :=
  let vA := tick 0 (tick 0 (⊥ : VV 3))        -- A ticked twice
  let vB := tick 1 (tick 0 (⊥ : VV 3))        -- B ticked after seeing A's
    ↪ first
  [ ("A's first tick ⊑ vB", decide ((GCounter.increment 0 (⊥ : VV 3)) ⊑ vB)),
    ("vA concurrent with vB", decide (Concurrent vA vB)),
    ("vA ⊔ vB dominates both", decide (vA ⊑ vA ⊔ vB ∧ vB ⊑ vA ⊔ vB)),
    ("deliverable: B's msg at A", decide (deliverable 1 vB (tick 0 (tick 0 (⊥
      ↪ : VV 3))))) ]

#eval interlude
```

```
[("A's first tick ⊑ vB", true),
 ("vA concurrent with vB", true),
 ("vA ⊔ vB dominates both", true),
 ("deliverable: B's msg at A", true)]
```

Replica B ticked after seeing A's first event, so A's first tick happened before B's state; yet A's *second* tick makes `vA` and `vB` concurrent — two truths that no single timeline could hold at once, and that the partial order holds without strain.

10.5 Exercises

Exercise 10.1 — Ticking is learning

★ Prove `tick_inflationary` from scratch — without citing `GCounter.increment_inflationary` — and observe that your proof is the same proof. What, if anything, about the *statement* changed between Chapter 3 and here?

Exercise 10.2 — The merge is the least common future

★★ State and prove: $v \sqcup w$ is the *least* version vector that dominates both v and w — that is, $v \sqsubseteq u$ and $w \sqsubseteq u$ imply $v \sqcup w \sqsubseteq u$. (You will find the proof is one lattice axiom. The exercise is noticing that "least upper bound of causal histories" was an axiom all along.) Solution in Appendix B.

Exercise 10.3 — Dotted version vectors

★★★ A version vector compresses a causal history into per-replica maxima, which assumes each replica's own events are seen contiguously. Systems that let servers assign ids on behalf of clients break the assumption and use *dotted* version vectors: a vector plus one extra "dot" (i, k) standing for event k of replica i, possibly beyond the contiguous prefix. Sketch the carrier, the order, and the merge; where does the semilattice structure survive, and where does it need care? Hints in Appendix B.

Part IV

Operations, Systems, and Limits

Chapter 11

Op-Based CRDTs and the Correspondence

> *"If you cannot afford to ship your whole state, ship your intentions — and pray the postal service reads them exactly once."*

Everything so far has been *state-based*: a replica ships its whole state, and the receiver joins it in. There is a second tradition. In an *operation-based* (or *op-based*) CRDT, a replica ships the *operation* it performed — "increment slot 3", "add element 7" — and every receiver applies it locally. The messages are small. The price is paid elsewhere, and this chapter is about locating exactly where.

The state-based designs of Part II asked nothing of the network beyond eventual delivery: reordering, duplication, and redelivery were all absorbed by the algebra of $\sqcup$. An op-based design has no join to hide behind. If operations arrive out of order, the replicas must hope the operations *commute*; if an operation arrives twice, it will be *applied* twice. The burden moves from the algebra to the transport.

We keep this chapter honest and light. A full formalization of op-based CRDTs — causal delivery preconditions, delivery-order semantics, the general equivalence with state-based designs — is a different book. What we can do precisely, and do, is: define the fully-commutative special case, prove that commutativity absorbs reordering, exhibit the op-based G-Counter, prove the state/op *correspondence* for it, and refute — twice, by machine — the hope that op-based designs tolerate the network sins that state-based ones shrug off.

11.1 Shipping Operations

Definition — Op-based CRDT, fully-commutative case

An op-based CRDT is a triple of a state type σ, an initial state, and an interpreter $\text{apply} : \text{Op} \to \sigma \to \sigma$, subject to the law that any two operations commute: $\text{apply}\, o_1 \circ \text{apply}\, o_2 = \text{apply}\, o_2 \circ \text{apply}\, o_1$.

```
/-- An operation-based CRDT (simplified to the fully-commutative case,
    which is all the counter needs): local state, an interpreter for
```

```
    operations, and the law that operations commute. -/
structure OpCRDT (Op σ : Type) where
  init  : σ
  apply : Op → σ → σ
  comm  : ∀ (o₁ o₂ : Op) (s : σ),
    apply o₁ (apply o₂ s) = apply o₂ (apply o₁ s)

/-- Replay a log of received operations. -/
def applyLog {Op σ : Type} (c : OpCRDT Op σ) (log : List Op) (s : σ) : σ :=
  log.foldl (fun s o => c.apply o s) s
```

The general op-based definition demands less: only *concurrent* operations must commute, and the transport must deliver operations in causal order (Chapter 10 built exactly the tool for saying "causal order"). Our simplification — *all* operations commute — is honest for counters and lets the chapter's theorems stay small. Where the general case differs, we say so in prose.

Commutativity is precisely the property that makes delivery *order* irrelevant. We can say that as a theorem, and the proof is a pleasant reprise of Chapter 8: induction over the permutation, with the accumulator generalized.

```
/-- Commutativity absorbs reordering (but *nothing* absorbs
    duplication — see the refutation below). -/
theorem applyLog_perm {Op σ : Type} (c : OpCRDT Op σ) {l₁ l₂ : List Op}
    (h : Perm l₁ l₂) : ∀ s, applyLog c l₁ s = applyLog c l₂ s := by
  induction h with
  | nil => intro _; rfl
  | cons x _ ih => intro s; exact ih (c.apply x s)
  | swap x y l =>
      intro s
      show applyLog c l (c.apply y (c.apply x s))
          = applyLog c l (c.apply x (c.apply y s))
      rw [c.comm]
  | trans _ _ ih₁ ih₂ => intro s; exact (ih₁ s).trans (ih₂ s)
```

Read the docstring's parenthesis again, because it is the whole chapter.

Permutations model *reordering*. They do not model *duplication*: a list and the same list with an element repeated are not permutations of each other. In Chapter 8 that gap was closed by idempotence — `foldJoin_dup` erased the duplicate. Here there is nothing to close it.

11.2 The Op-Based G-Counter

An operation is "increment at replica i" — an element of `Fin R`. The interpreter is the `increment` we have had since Chapter 3, and the commutativity law is a four-way case split on indices.

```
/-- The op-based G-Counter: an operation is "increment at replica i". -/
def opCounter (R : Nat) : OpCRDT (Fin R) (GCounter R) where
  init := ⊥
  apply := GCounter.increment
  comm o₁ o₂ s := by
    funext j
    by_cases h₁ : j = o₁ <;> by_cases h₂ : j = o₂
    · subst h₁; subst h₂; rfl
    · subst h₁; simp [GCounter.increment, h₂]
    · subst h₂; simp [GCounter.increment, h₁]
    · simp [GCounter.increment, h₁, h₂]
```

Note what this design does *not* need: no causal order (all operations commute outright), no per-replica slots visible to the network, no join. And note what it *does* need, which the state-based G-Counter never asked for: every operation delivered *exactly once*. The two designs make opposite trades:

	state-based	op-based
message size	whole state	one operation
reordering tolerated by	$\sqcup$ comm. + assoc.	op commutativity
duplication tolerated by	$\sqcup$ idempotence	*nothing* — *forbidden*
loss tolerated by	any later merge	*nothing* — *forbidden*
transport must provide	eventual delivery	exactly-once (+ causal order, in the general case)

11.3 The Correspondence, for One Instance

The two traditions are not rivals; they are two readings of one history. A replica that executes an op log also passes through a sequence of *states*, and in the state-based reading those states are exactly what it would have shipped. Fold the shipped states by join and you recover the op-log replay. For inflationary interpreters this is a theorem, and its proof is one lemma from Chapter 2 applied once per step: each state absorbs its predecessor (`le_iff_sup_eq`), so the running join is always just the latest state.

```
/-- The state each replica would *ship* after each operation. -/
def statesAlong {Op σ : Type} (c : OpCRDT Op σ) : List Op → σ → List σ
  | [], _ => []
  | o :: log, s =>
```

```
      let s' := c.apply o s
      s' :: statesAlong c log s'

/-- **The correspondence, for inflationary interpreters.** Folding the
    stream of shipped states by join replays the op log exactly: the
    state-based and op-based readings of the same history agree. -/
theorem foldJoin_statesAlong {Op σ : Type} [BoundedJoinSemilattice σ]
    (c : OpCRDT Op σ) (hinf : ∀ o, Order.Inflationary (c.apply o)) :
    ∀ (log : List Op) (s : σ),
      SEC.foldJoinFrom s (statesAlong c log s) = applyLog c log s
  | [], _ => rfl
  | o :: log, s => by
      show SEC.foldJoinFrom (s ⊔ c.apply o s) (statesAlong c log (c.apply o
        ↪ s))
          = applyLog c log (c.apply o s)
      rw [Order.le_iff_sup_eq.mp (hinf o s)]
      exact foldJoin_statesAlong c hinf log (c.apply o s)

/-- Instantiated at the G-Counter: op-log replay = join of state deltas. -/
theorem opGCounter_simulates (R : Nat) (log : List (Fin R)) :
    SEC.foldJoin (statesAlong (opCounter R) log ⊥)
      = applyLog (opCounter R) log ⊥ :=
  foldJoin_statesAlong (opCounter R) GCounter.increment_inflationary log ⊥
```

The general correspondence (stated, not proved here)

Every op-based CRDT whose transport provides causal, exactly-once delivery can be emulated by a state-based CRDT, and vice versa: a state-based CRDT is an op-based one whose single operation is "merge this state" — an operation that commutes with itself because $\sqcup$ is commutative and associative, and that tolerates redelivery because $\sqcup$ is idempotent. *This box is a claim, not a Lean theorem.* The emulation in the op-to-state direction requires formalizing causal delivery as a precondition, which we have built the vocabulary for (Chapter 10) but not the machinery. We proved the correspondence for the G-Counter above; the general statement is Shapiro et al. (2011), Theorems 2.2–2.3.

11.4 Two Refutations

First, the counter. The op-based counter under at-least-once delivery is simply a wrong counter:

Refuted 11.1 — The op-based counter tolerates duplicated delivery

A duplicated increment operation counts twice. There is no idempotence to hide behind: `applyLog` of `[0, 0]` and of `[0]` differ, and `decide` can watch them differ.

```
theorem op_dup_overcounts :
    GCounter.value (applyLog (opCounter 1) [0, 0] ⊥)
      ≠ GCounter.value (applyLog (opCounter 1) [0] ⊥) := by decide
```

Contrast Chapter 3: the state-based G-Counter received its own state twice and shrugged (`foldJoin_dup`). The op-based one cannot. The design did not get worse; the *contract* moved.

Second, sets — where duplication interacts with removal in a way that is easy to underestimate. Consider a naive op-based set whose operations are "add e" and "remove e":

```
/-- A tiny op-based set, to break in two ways. -/
inductive SetOp where
  | add (e : Nat)
  | remove (e : Nat)

def applySetOp : SetOp → GSet → GSet
  | .add e, s => SList.insert e s
  | .remove e, s => SList.filter (· != e) s
```

A removal operation is idempotent as a *function* — removing twice in a row is harmless. The damage needs a redelivery that arrives *late*, after a re-add:

```
theorem op_set_dup_kills_readd :
    let exactlyOnce := [SetOp.add 1, .remove 1, .add 1]
    let withDup := [SetOp.add 1, .remove 1, .add 1, .remove 1]
    (1 ∈ withDup.foldl (fun s o => applySetOp o s) (⊥ : GSet))
      ≠ (1 ∈ exactlyOnce.foldl (fun s o => applySetOp o s) (⊥ : GSet)) := by
  decide
```

The network redelivered an old remove after a new add, and the new add died — the same wound Chapter 7 healed with tags, torn open again because operations carry no tags to be selective with. And these operations do not even commute, so causal order is load-bearing too:

```
theorem setops_not_comm :
    applySetOp (.add 1) (applySetOp (.remove 1) (⊥ : GSet))
      ≠ applySetOp (.remove 1) (applySetOp (.add 1) (⊥ : GSet)) := by decide
```

The moral, once more, slowly:

Op-based CRDTs do not eliminate the three network sins. They re-assign them — from the algebra, which Part II taught you to verify, to the transport, which you must now trust.

Real op-based systems earn that trust with sequence numbers, acknowledgements, and causal-delivery buffers built from exactly the version vectors of Chapter 10. The engineering is respectable and well understood. But when someone tells you their op-based system is "conflict-free by construction", ask them what happens when a message is retransmitted, and watch for the words "well, the delivery layer".

11.5 Exercises

Exercise 11.1 — The op-based PN-Counter

★ Define `opPNCounter R : OpCRDT (Bool × Fin R) (PNCounter R)`: an operation is a signed increment — `(true, i)` increments and `(false, i)` decrements at replica `i`. Prove the commutativity law. (Exercise 3.1 does most of the work.)

Exercise 11.2 — At-least-once overcounts

★★ Model an at-least-once network as: every op log `l` may be replaced by any log `l'` such that every operation of `l` occurs in `l'` at least as often. Show, with a concrete `decide`-checked instance, that the op-based counter's value is not an invariant of this relation, and prove that the state-based G-Counter's value is (for the corresponding state deliveries). Which lemma of Chapter 8 carries the second half?

Exercise 11.3 — Delta-CRDTs

★★★ Between "ship the whole state" and "ship one operation" lies a midpoint: ship a small state — a *delta* — that joins like a state. Read Almeida, Shoker & Baquero, *Delta State Replicated Data Types* (2018). Then prove the delta-interval lemma for the G-Counter: for any `g ⊑ h`, the "delta" `d := h` itself satisfies `g ⊔ d = h`, and find the *smallest* delta (pointwise) with that property. Hints in Appendix B; finish after Chapter 10 if you want dotted deltas.

Chapter 12

Capstone: A Replicated Store in Lean

"The test of a theory is what it makes routine."

The predecessor's capstones earned their keep by assembling everything: the Sudoku solver was cells, lattices, and propagators with no new theory. This book's capstone holds itself to the same standard, with one sharpening: the payoff is not just that the pieces *compose*, but that the composition needs *no new proofs at all*.

We build a collaborative shopping list, replicated across three simulated replicas — call them Alice, Bob, and Carol — under an adversarial delivery schedule. The list is an OR-Set of items (Chapter 7: adds must win, re-adds must work). Each item carries a quantity, and quantities can go up and down, so each is a PN-Counter (Chapter 4) — one per item, which is a pointwise function lattice (Chapter 6).

12.1 The Store

```
/-- The store. Every part is a semilattice we already built, so the
    whole is one, with no new proofs: product of OR-Set and a pointwise
    map of PN-Counters. -/
abbrev Store (R : Nat) := ORSet R × (Nat → PNCounter R)
```

That single line is the entire data-type design. Look at what Lean does with it: asked for `BoundedJoinSemilattice (Store R)`, instance search composes the OR-Set instance (Chapter 7), the PN-Counter instance (a product of G-Counters, Chapter 4), the pointwise function-space instance (Chapter 6), and the product instance (Chapter 2) — and every ACI law, every bound, every monotonicity fact composes with them. We wrote no `sorry` and also no proof.

The operations are the pieces' operations, lifted:

```
def addItem (i : Fin R) (item : Nat) (s : Store R) : Store R :=
  (ORSet.add i item s.1, s.2)

def removeItem (item : Nat) (s : Store R) : Store R :=
  (ORSet.remove item s.1, s.2)

def incrByAt (i : Fin R) : Nat → PNCounter R → PNCounter R
```

```
  | 0, p => p
  | k + 1, p => PNCounter.incr i (incrByAt i k p)

def addQty (i : Fin R) (item qty : Nat) (s : Store R) : Store R :=
  (s.1, fun e => if e = item then incrByAt i qty (s.2 e) else s.2 e)

def hasItem (item : Nat) (s : Store R) : Prop := ORSet.mem item s.1

def itemQty (item : Nat) (s : Store R) : Int := PNCounter.value (s.2 item)

def render (probe : List Nat) (s : Store R) : List (Nat × Bool × Int) :=
  probe.map (fun e => (e, decide (hasItem e s), itemQty e s))
```

12.2 The Theorem, in One Line

Chapter 8 proved strong eventual consistency once, for every bounded join-semilattice. The store is a bounded join-semilattice. Therefore:

```
/-- **The capstone theorem, in one line.** Strong eventual consistency
    for the whole composed store is `sec_state_based` instantiated at
    `Store` — the entire book is in how little there is to do here. -/
theorem sec_store {r₁ r₂ : SEC.Replica (Store R)}
    (h : SEC.SameDeliveredSet r₁ r₂) : r₁.state = r₂.state :=
  SEC.sec_state_based (σ := Store R) h
```

Say it loudly, because it is the entire book compressed to a type-ascription: *the convergence proof for the composed, application-shaped, three-features-deep replicated store is the general theorem applied to the store's type.* No per-feature argument. No "and by a similar case analysis". The work was done once, in Chapter 8, for every lattice at once; Part II's job was only ever to manufacture lattices.

12.3 Replicas as Cells with a Network Attached

The predecessor gave us `Cell`: an `IO.Ref` whose `write` is a join. A replica node is the same object with a peer-facing method. (The store contains function types, so there is no decidable equality to short-circuit unchanged writes with; a node simply merges.)

```
structure Node (R : Nat) where
  id : Fin R
  ref : IO.Ref (Store R)

def Node.new (i : Fin R) : IO (Node R) := do
  return ⟨i, ← IO.mkRef (⊥ : Store R)⟩

```

```
def Node.edit (n : Node R) (f : Store R → Store R) : IO Unit :=
  n.ref.modify f

/-- Anti-entropy: `dst` merges `src`'s state. Just a join. -/
def Node.pull (dst src : Node R) : IO Unit := do
  let s ← src.ref.get
  dst.ref.modify (· ⊔ s)
```

The demonstration fixes the *edits* and varies the *network*. Alice puts bread on the list, twice over (item 1, quantity 2). Bob adds eggs, thinks better of it and removes them, then puts milk on the list (item 2). Carol, concurrently with Alice and without having seen her, also adds bread and bumps its quantity once more. Note the two deliberately spiky spots: bread is added *concurrently at two replicas* (the OR-Set must not double it), and eggs are added and removed (the tombstone must hold everywhere).

```
def localEdits (alice bob carol : Node 3) : IO Unit := do
  alice.edit (fun s => addQty 0 1 2 (addItem 0 1 s))
  bob.edit (addItem 1 3)      -- Bob adds eggs ...
  bob.edit (removeItem 3)     -- ... and thinks better of it
  bob.edit (fun s => addQty 1 2 1 (addItem 1 2 s))
  carol.edit (fun s => addQty 2 1 1 (addItem 2 1 s))
```

A *schedule* is a list of directed merges. We run each schedule's merges — duplicated, reversed, self-directed, whatever the schedule says — and finish with a full anti-entropy sweep so that every update reaches every replica (the hypothesis of `sec_store`, made true by brute force; Chapter 9's driver did it with a termination proof instead).

```
def runSchedule (name : String) (sched : List (Nat × Nat)) : IO Unit := do
  let alice ← Node.new 0
  let bob   ← Node.new 1
  let carol ← Node.new 2
  let node (k : Nat) : Node 3 :=
    if k % 3 = 0 then alice else if k % 3 = 1 then bob else carol
  localEdits alice bob carol
  for (src, dst) in sched do
    (node dst).pull (node src)
  -- the sweep: two full rounds of everyone-pulls-everyone
  for _ in [0, 1] do
    for src in [0, 1, 2] do
      for dst in [0, 1, 2] do
        (node dst).pull (node src)
  let sA ← alice.ref.get
  let sB ← bob.ref.get
  let sC ← carol.ref.get
  IO.println s!"{name}:"
  IO.println s!"  alice: {render [1, 2, 3] sA}"
```

```
  IO.println s!"  bob:   {render [1, 2, 3] sB}"
  IO.println s!"  carol: {render [1, 2, 3] sC}"

/-- Three adversarial schedules, identical final states. -/
def demoConvergence : IO Unit := do
  runSchedule "schedule A (orderly)" [(0, 1), (1, 2), (2, 0)]
  runSchedule "schedule B (reversed, duplicated)"
    [(2, 1), (2, 1), (1, 0), (1, 0), (0, 2)]
  runSchedule "schedule C (chaotic, self-merges)"
    [(1, 1), (2, 0), (0, 0), (1, 2), (2, 1), (2, 1), (0, 2)]
```

12.4 Computation Interlude: Three Networks, One Answer

`#eval demoConvergence` prints, in full:

```
schedule A (orderly):
  alice: [(1, (true, 3)), (2, (true, 1)), (3, (false, 0))]
  bob:   [(1, (true, 3)), (2, (true, 1)), (3, (false, 0))]
  carol: [(1, (true, 3)), (2, (true, 1)), (3, (false, 0))]
schedule B (reversed, duplicated):
  alice: [(1, (true, 3)), (2, (true, 1)), (3, (false, 0))]
  bob:   [(1, (true, 3)), (2, (true, 1)), (3, (false, 0))]
  carol: [(1, (true, 3)), (2, (true, 1)), (3, (false, 0))]
schedule C (chaotic, self-merges):
  alice: [(1, (true, 3)), (2, (true, 1)), (3, (false, 0))]
  bob:   [(1, (true, 3)), (2, (true, 1)), (3, (false, 0))]
  carol: [(1, (true, 3)), (2, (true, 1)), (3, (false, 0))]
```

Nine renders, one answer. Bread is present with quantity 3 — Alice's two plus Carol's one, concurrent adds merged without double-counting the item and without losing a single increment. Milk is present with quantity 1. Eggs are absent everywhere, Bob's tombstone having travelled with his state. The three schedules delivered in different orders, delivered twice, and merged replicas into themselves; the states cannot tell.

That is what `sec_store` promised, and this is the appropriate moment to re-read what it did not promise: nothing about *when* agreement happens (only at quiescence), and nothing about the agreed state being *desirable*. Chapter 13 takes up that second gap.

12.5 Exercises

Exercise 12.1 — Grow the store

★ Add a third component to `Store`: an LWW-Register (Chapter 6) holding the list's title. How many new proofs does convergence of the extended store require? Verify your answer by extending `sec_store`.

Exercise 12.2 — Clear the list

★★ Implement `clearList : Fin R → Store R → Store R`, which removes every item currently on the list. Run it against a concurrent `addItem` from another replica and inspect the merge. Which semantics did you accidentally choose — does the concurrent add survive? Relate the answer to `addWins` (Chapter 7), and state the semantics you *wish* you had chosen precisely enough that a teammate could implement it.

Exercise 12.3 — The wrong set, on purpose

★★★ Swap the OR-Set for a 2P-Set (Chapter 5) in `Store`. Everything still compiles and still converges — convergence was never the issue. Re-run the capstone demo with an item that is removed and re-added, and write the bug report your users would file, in their words. Then write the one-paragraph design-review comment that would have caught it before launch.

Chapter 13

What CRDTs Cannot Do

> *"A theorem is also a boundary: everything it does not say, it quietly hands back to you."*

This book has spent twelve chapters building machinery that makes a strong promise — replicas that have heard the same news hold the same state — and proving it with no coordination, no consensus, no waiting. The honest way to end is to walk the boundary of that promise, because production systems fail at the boundary, not in the interior.

13.1 Convergence Is Not Correctness

Strong eventual consistency says the replicas *agree*. It says nothing about the state they agree *on*. Chapter 6's LWW-Register converges beautifully while discarding one of two concurrent writes — silently, deterministically, and possibly the one your user cared about. Converging to a state nobody wanted counts as convergence.

We met a small version of this in Exercise 4.2: a query can hide what the state knows. The PN-Counter itself is more honest — and what it honestly reports can be negative, with no one replica to blame:

```
/-- The PN-Counter happily goes negative: convergence is not
    correctness. (The Chapter 4 `clampedValue` exercise hid the
    problem; it did not solve it.) -/
theorem pn_goes_negative :
    PNCounter.value (PNCounter.decr 0 (⊥ : PNCounter 1)) < 0 := by decide
```

For a counter of likes, a negative blip is a shrug. For an inventory, it is an oversold product. For an account balance, it is the next section.

13.2 Cross-Replica Invariants: the Double-Spend

Suppose two replicas hold an account with balance 100, a partition separates them, and each — being *available*, as advertised — accepts a withdrawal of 80. Each replica, locally impeccable, now shows 20. The partition heals. What should the merge say?

Any merge worthy of a CRDT must be idempotent — Chapter 8 proved duplicated

delivery forces that, and here both replicas hold the *same* value 20, so idempotence pins the merge of 20 and 20 to 20. But honoring both withdrawals demands $100-80-80 = -60$. One function cannot say both.

Refuted 13.1 — There is a merge for non-negative bank accounts

Formally: no `merge : Int → Int → Int` is both idempotent and honors every pair of withdrawals from a shared initial balance. The proof instantiates the two requirements at the double-spend and watches them collide.

```
theorem no_bank_merge :
    ¬ ∃ merge : Int → Int → Int,
        (∀ a, merge a a = a) ∧
        (∀ x y : Nat, x ≤ 100 → y ≤ 100 →
          merge (100 - (x : Int)) (100 - (y : Int))
            = 100 - (x : Int) - (y : Int)) := by
  intro ⟨merge, hidem, hboth⟩
  have h₂ := hboth 80 80 (by omega) (by omega)
  have he : (100 : Int) - (80 : Nat) = 20 := by decide
  rw [he, hidem 20] at h₂
  exact absurd h₂ (by decide)
```

Note what is refuted: not a particular data structure, but the *existence* of any merge function meeting the specification. "Balance never goes negative" is a *cross-replica* invariant: its truth depends on the sum of actions taken at different replicas, and no amount of per-replica discipline can protect it while every replica stays available under partition. This is the CAP theorem (Chapter 1) arriving at street level.

The general shape is worth internalizing. CRDTs preserve invariants that are *closed under join*: "the set only grows", "every tally is at least its last shipped value", "the register holds the lexicographically largest write". They cannot preserve invariants that relate *quantities spent* across replicas — budgets, stock levels, uniqueness of usernames — because a partition lets both sides spend the same headroom.

13.3 When You Must Coordinate

The impossibility is not the end of engineering; it is a pricing signal. Three standard responses, in increasing order of coordination:

Escrow. Split the headroom *before* the partition: give each replica a budget it may spend unilaterally ($e_1 + e_2 \le 100$), and require coordination only to *rebalance* budgets. Withdrawals within budget commute and merge safely — the budget is a per-replica invariant again, and Exercise 13.2 makes this precise for the PN-Counter, finally discharging the marked Exercise 4.3.

Reservations. A hybrid: optimistic operations execute immediately against a reserved allocation; operations that exceed it queue until a coordinator grants more.

The available path stays available; the dangerous path waits.

Consensus. For invariants that admit no slack — uniqueness, exactly-once side effects in the world — you need agreement before action: Paxos, Raft, or a database that runs one for you. That is a different book, but now you can say precisely what you are buying: the right to enforce predicates that are not closed under join, at the price of unavailability under partition.

The professional failure mode is not choosing coordination or choosing CRDTs; it is choosing one *per system* instead of *per invariant*. A shopping cart's contents and its checkout are different invariants. Replicate the first. Coordinate the second.

13.4 The Garbage That Cannot Be Collected (Alone)

One more debt from Part II comes due. The 2P-Set's tombstones (Chapter 5) and the OR-Set's (Chapter 7, Exercise 7.2) only ever grow — `tombs_monotone` is a theorem, and it is the *good* news, because monotonicity is what lets removal survive merges. The bad news is its contrapositive in operations: you cannot delete a tombstone unilaterally. If one replica discards a tombstone while a peer still carries the matching add, their next merge resurrects the element — exactly the failure of Refutation 5.1, re-enacted by the garbage collector.

Discarding a tombstone is safe only when it is *causally stable*: every replica has seen it, so no surviving state can contradict it. And "every replica has seen it" is knowledge about the global configuration — the version vectors of Chapter 10 can witness it, but gathering the witness is a round of coordination. Hence the closing irony, which deserves its own line:

The data type that exists to avoid coordination needs coordination to take out its trash.

Production systems live with this honestly: they run anti-entropy anyway, so they piggyback stability tracking on it, and they garbage collect in the calm between partitions. The theory did not fail; it told you the price in advance.

13.5 Closing

Here is the whole book in four sentences. A replica is a propagator cell; merge is join; updates climb; and the three network sins are absorbed by the three semilattice laws — that is Chapters 1 through 7. Convergence is then one theorem about folds over multisets, proved once for every lattice — that is Chapter 8, wearing operational clothes in Chapters 9 and 10 and shipping smaller messages in Chapter 11. Composition is free — that is Chapter 12, in one line. And the theorem's boundary is real: agreement is not correctness, join-closed invariants are the only free ones, and the trash needs a committee — that is this chapter.

The predecessor ended with a solver and a type-inferencer running to a fixpoint inside one process. This book ends with the same algebra holding a small distributed system

together while a hostile network does its worst. The mathematics did not change. The postal service did.

13.6 Exercises

Exercise 13.1 — CRDT-able or not

★ Classify each feature as replicable-without-coordination or not, with the invariant that decides it: (a) a document's set of tags; (b) a "seen by" read-receipt list; (c) an auction's current highest bid; (d) a username registry; (e) a like counter; (f) a bank ledger's *history* (as opposed to its balance).

Exercise 13.2 — Escrow, precisely

★★ Bound the PN-Counter: fix per-replica budgets `e : Fin R → Nat` and say a state `p` is *within escrow* when each replica's decrement tally obeys `p.2 i ≤ e i` and the budgets obey `(budget sum) ≤ (initial credit)`. Prove: (a) within-escrow states are closed under merge; (b) a within-escrow state's value is at least the initial credit minus the budget sum, hence non-negative. This discharges Exercise 4.3 and makes the "escrow" response of this chapter a theorem rather than a slogan.

Appendix A

Lean 4 Quick Reference

This appendix carries the predecessor's reference tables forward and adds the machinery this book introduced: quotients, well-founded recursion, function extensionality, and `Fin` idioms.

A.1 Basic Types and Typeclasses

```
-- Function types, implicit arguments, instance arguments
def f (a : Nat) {b : Nat} [DecidableEq α] : Nat := ...

-- Declaring and instantiating a typeclass
class MyClass (α : Type) where
  someMethod : α → Nat

instance : MyClass Nat where
  someMethod := id

-- Classes can extend classes; omitted parent fields are
-- synthesized from registered instances:
class BoundedJoinSemilattice (α : Type) extends LE α, PartialOrder α where
  sup : α → α → α
  ...
```

A.2 Quotient Types

The five moves, in the order Chapter 8 teaches them:

```
-- 1. A Setoid packages a relation with its equivalence proof.
instance permSetoid (α : Type) : Setoid (List α) where
  r := Perm
  iseqv := ⟨Perm.refl, Perm.symm, Perm.trans⟩

-- 2. Quotient: the type of equivalence classes.
def MSet (α : Type) : Type := Quotient (permSetoid α)

```

```
-- 3. Quotient.mk: inject a representative.
def ofList (l : List σ) : MSet σ := Quotient.mk (permSetoid σ) l

-- 4. Quotient.sound: related representatives give EQUAL classes.
example : ofList [1, 2] = ofList [2, 1] := Quotient.sound (.swap 1 2 [])

-- 5. Quotient.lift: define functions out of the quotient by
--    providing the respect-the-relation proof (the "toll");
--    Quotient.ind: prove properties of all classes via representatives.
def foldJoin : MSet σ → σ :=
  Quotient.lift SEC.foldJoin (fun _ _ h => SEC.foldJoin_perm h)
```

`#print axioms` on anything built this way reports `Quot.sound`: quotient soundness is an axiom of Lean's kernel, like `propext`. The audit block at the end of `Crdt.lean` confirms these two are the only axioms in the book, with `Classical.choice` appearing nowhere.

A.3 Well-Founded Recursion

```
-- When recursion is not structural, name a measure that strictly
-- decreases; `decreasing_by` discharges the obligation.
def gossipUntilQuiescent (n : Net R σ) : Net R σ :=
  match _h : findPair n with
  | none => n
  | some pq => gossipUntilQuiescent (exchange pq.1 pq.2 n)
termination_by cost n
decreasing_by
  exact cost_exchange_lt (findPairAux_spec _h)
```

Idioms worth remembering from Chapter 9: name the scrutinee (`match _h : findPair n`) so the decrease proof can use the equation; keep the measure a plain `Nat`; prove the strict-decrease lemma *before* the definition so `decreasing_by` is one line.

A.4 funext and Friends

```
-- Two functions are equal if they agree at every point.
theorem le_antisymm : g ⊑ h → h ⊑ g → g = h :=
  fun h₁ h₂ => funext fun i => Nat.le_antisymm (h₁ i) (h₂ i)

-- propext: equal propositions from a biconditional (used by ext-style
-- set reasoning); proof irrelevance is definitional for Prop fields,
-- which is why ⟨v, p₁⟩ = ⟨v, p₂⟩ closes by rfl after `cases`.
```

A.5 Fin Idioms

```
#eval List.finRange 4          -- [0, 1, 2, 3] : List (Fin 4)
-- Every inhabitant is a member:
#check @List.mem_finRange      -- ∀ (x : Fin n), x ∈ List.finRange n
-- Induction over the index space via the cons form:
#check @List.finRange_succ     -- finRange (n+1) = 0 :: (finRange n).map
 ↪  Fin.succ
-- Case-split a Fin (n+1) into zero and successor:
--   cases i using Fin.cases with
--   | zero => ...
--   | succ i' => ...
-- Empty index space: exact i.elim0
-- Decidability of bounded quantifiers is instance-resolved:
#check (inferInstance : Decidable (∀ i : Fin 3, i.val < 5))
```

A.6 Permutations

Our hand-rolled `Perm` (Chapter 8) has four constructors — `nil`, `cons`, `swap`, `trans` — and proofs by `induction h` get one case per constructor, with the accumulator generalized when folding. The standard library's `List.Perm` is the same design; after this book, read its source and feel at home.

A.7 Tactics Reference

Tactic	Usage
`rfl`, `exact h`, `apply f`	as in the predecessor
`refine f ?_ h ?_`	apply, keeping argument order for goals
`intro h` / `cases h` / `induction h`	introduction, case split, induction
`cases i using Fin.cases`	zero/succ split on `Fin (n+1)`
`induction q using Quotient.ind`	representative for a quotient value
`by_cases h : p`	decidable case split
`subst h`	eliminate an equation `h : a = b`
`simp` / `simp_all` / `simp at h ⊢`	simplification
`omega`	linear arithmetic over `Nat`/`Int`
`decide`	kernel-evaluate a decidable proposition
`funext i`	pointwise function equality
`constructor` / `obtain ⟨x, h⟩ := ...`	build / destruct
`calc`	chained (in)equalities

Table A.1: Tactics used in this book, beyond the predecessor's table.

A.8 Unicode Input

Symbol	Input	Meaning
⊑	`\sqsubseteq`	information order on states
⊔	`\sqcup`	join (merge)
⊥	`\bot`	bottom (empty state)
≼	`\preceq`	element total order (Ch. 5)
≺	`\prec`	strict element order
≈	`\approx`	setoid relation
⟦l⟧	`\[[ l \]]`	quotient class of `l`
σ	`\s`	state type
→ ← ∀ ∃ ∈ ∉ ×	as in the predecessor	

Table A.2: Unicode used in this book, beyond the predecessor's table.

Appendix B

Selected Solutions

Full solutions are given for exercises with a single definitive Lean answer; every solution below compiles, verbatim, in the `Crdt.Solutions` namespace of `Crdt.lean` (or is a named theorem of the main text, cited by name). Long-fuse (★★★) exercises get hints, not spoilers. Open-ended design exercises get a paragraph of discussion.

Chapter 1

Exercise 1.1. There are $\binom{2}{1} = 2$ interleavings of one increment per replica if syncs happen only at the end, but the interesting count is of *sync placements*: for each of the six orderings of $\{\text{incr}_A, \text{incr}_B, \text{sync}_{A\to B}, \text{sync}_{B\to A}\}$ consistent with program order, tabulate the final pair of values. Four of them lose an update under overwrite-merge.

Exercise 1.3. As in the main text:

```
theorem overwrite_not_comm :
    ¬ (∀ a b : NaiveCounter, overwrite a b = overwrite b a) :=
  fun h => absurd (h 1 2) (by decide)
```

Chapter 2

Exercise 2.1 and **Exercise 2.2** are `Order.sup_idem` and `Order.le_iff_sup_eq` in the main text; try them from the six axioms alone before peeking. **Exercise 2.3** is the `instance : BoundedJoinSemilattice (α × β)` of Chapter 2 — each field is the corresponding field of the components, applied coordinatewise.

Chapter 3

Exercise 3.1. Increments commute at *any* indices, equal or not. The direct proof is `funext j` followed by a four-way `by_cases` on `j = i` and `j = j'` (substitute where positive, `simp [GCounter.increment]` everywhere) — write that version first. Once you reach Chapter 11, the op-based counter's commutativity law *is* this statement, so the solution collapses to one line:

```
theorem increment_comm {R : Nat} (i j : Fin R) (g : GCounter R) :
    GCounter.increment i (GCounter.increment j g)
      = GCounter.increment j (GCounter.increment i g) :=
  (OpBased.opCounter R).comm i j g
```

Exercise 3.2. Rendering the merge is zipping the renders; the lemma is a two-line induction once stated over an arbitrary index list:

```
theorem zipWith_map_map {α β : Type} (f : β → β → β) (g h : α → β) :
    ∀ (l : List α),
      List.zipWith f (l.map g) (l.map h) = l.map (fun i => f (g i) (h i))
  | [] => rfl
  | x :: t => by
      simp only [List.map_cons, List.zipWith_cons_cons]
      rw [zipWith_map_map f g h t]

theorem toList_sup {R : Nat} (g h : GCounter R) :
    GCounter.toList (g ⊔ h)
      = List.zipWith Nat.max (GCounter.toList g) (GCounter.toList h) :=
  (zipWith_map_map Nat.max g h (List.finRange R)).symm
```

Exercise 3.3 (★★★, hints). Pointwise, $\max(g_i, h_i) \le g_i + h_i$; the work is turning a pointwise bound into a bound on sums. Two ingredients: the seed-generalized comparison `GCounter.foldl_add_le`, and a "sum splits over pointwise addition" lemma proved with *two* generalized seeds. For strictness, one shared increment suffices — `decide` on `GCounter 1`. (A complete development compiles in `Crdt.Solutions`; earn it first.)

Chapter 4

Exercise 4.1 is `PNCounter.value_bot` in the main text; the only content is that a fold of zeros is its seed.

Exercise 4.2.

```
def clampedValue {R : Nat} (p : PNCounter R) : Nat :=
  (PNCounter.value p).toNat

example : clampedValue (PNCounter.decr 0 (⊥ : PNCounter 1)) = 0 := by decide
example : PNCounter.value (PNCounter.decr 0 (⊥ : PNCounter 1)) < 0 := by
  ↪ decide
```

The pair of **`example`**s is the point: the clamp changes the report, not the state. Chapter 13 returns to this.

Exercise 4.3 is discharged by Exercise 13.2; see the Chapter 13 solutions below.

Chapter 5

Exercise 5.1 is the `Decidable (x ∈ s)` instance of the main text — delegate to the list.

Exercise 5.2. The lemma is a pigeonhole for sorted duplicate-free lists: a subset relation between them bounds the lengths. Induct on the *larger* list; when the heads agree, strip both (membership in the tail is clean because strict sortedness forbids the head from recurring); when they differ, the smaller list lives entirely in the larger tail:

```
theorem size_monotone {s t : GSet} (h : s ⊑ t) :
    SList.size s ≤ SList.size t :=
  sorted_subset_length s.sorted t.sorted h
```

(`sorted_subset_length` is proved in full in `Crdt.Solutions`; its skeleton is the same double-head analysis as `sorted_ext`.)

Exercise 5.3 is `TwoPSet.remove_inflationary` in the main text.

Exercise 5.4 (★★★, hints). Intersection is the greatest lower bound of the subset order; prove the three `IsInfOf`-shaped laws through `SList.mem_filter`. For the second half, exhibit a state above another whose meet-merge falls strictly *below* one argument, and conclude a meet-merging replica can unlearn — violating `Inflationary`, the defining discipline of Chapter 2.

Chapter 6

Exercise 6.1.

```
theorem lww_merge_bot {R : Nat} (x : LWW.LWWReg R Nat) : ⊥ ⊔ x = x :=
  Order.sup_bot_left x
```

Exercise 6.2. With bare timestamps the grouping decides the winner:

```
example :
    LWW.naiveMerge (LWW.naiveMerge (3, 1) (3, 2)) (3, 3)
      ≠ LWW.naiveMerge (LWW.naiveMerge (3, 2) (3, 1)) (3, 3) := by decide
```

Exercise 6.3 (★★★, hints). Flip the order on the Boolean payload (declare a second `TotalOrder Bool` with `false` on top, wrapped in a newtype to avoid instance clashes). Every theorem of Chapter 6 goes through verbatim, because `maxo`'s ACI never asked which order it was given — that is the punchline: *convergence is indifferent to your tie-break; only your users notice.* Convergence ≠ intent.

Chapter 7

Exercise 7.1 is the `Decidable (mem e s)` instance of the main text, via `mem_iff_bex` and `decidable_of_iff`.

Exercise 7.2. Compose two facts you already own — `tombs_monotone` and the Chapter 5 solution:

```
theorem orset_tombs_size_monotone {R : Nat} {s t : ORSet R} (h : s ⊑ t) :
    SList.size s.tombs ≤ SList.size t.tombs :=
  sorted_subset_length s.tombs.sorted t.tombs.sorted h.2.1
```

Exercise 7.3 (★★★, hints). Keep, per element and per replica, only the *largest* add-tag and a per-replica "removed below" watermark — a version vector (Chapter 10) doing the tombstones' job in bounded space. The invariant connecting the compact and verbose representations is the exercise; `sorted_ext` is again the way canonical forms are shown equal.

Chapter 8

Exercise 8.1 is `SEC.foldJoin_append` in the main text; the intended proof composes `List.foldl_append` with `foldJoinFrom_eq_sup`.

Exercise 8.2. `Quotient.lift₂` charges two tolls; both are append-congruences of `Perm`, each a four-line induction:

```
theorem perm_append_right {α : Type} (m : List α) {l₁ l₂ : List α}
    (h : Perm l₁ l₂) : Perm (l₁ ++ m) (l₂ ++ m) := by
  induction h with
  | nil => exact Perm.refl m
  | cons x _ ih => exact .cons x ih
  | swap x y l => exact .swap x y (l ++ m)
  | trans _ _ ih₁ ih₂ => exact .trans ih₁ ih₂

theorem perm_append_left {α : Type} (m : List α) {l₁ l₂ : List α}
    (h : Perm l₁ l₂) : Perm (m ++ l₁) (m ++ l₂) := by
  induction m with
  | nil => exact h
  | cons x t ih => exact .cons x ih

def msetUnion {α : Type} : MSet α → MSet α → MSet α :=
  Quotient.lift₂ (fun l₁ l₂ => MSet.ofList (l₁ ++ l₂))
    (fun _ _ _ _ h₁ h₂ => Quotient.sound
      (.trans (perm_append_right _ h₁) (perm_append_left _ h₂)))
```

Exercise 8.3 (★★★, hints). Suppose folding is insensitive to delivery order and

multiplicity for *all* lists. Idempotence: compare `[a]` with `[a, a]`. Commutativity: compare `[a, b]` with `[b, a]` — but folds start at ⊥, so first show the fold of `[a]` is recoverable (use insensitivity with the empty prefix), then peel. Associativity comes free once the operation is recovered as the fold of two-element lists. The converse direction is where you discover which hypotheses were actually load-bearing.

Chapter 9

Exercise 9.1. One step suffices: `Step.update` at replica 0 with any non-⊥ update produces a configuration where replica 0's state strictly dominates replica 1's. Convergence claims nothing before quiescence — and this trace is the witness.

Exercise 9.2. Induction along `Reachable`, one case per step; only `update` and `deliver` touch `seen`, and both extend it:

```
theorem seen_monotone {R : Nat} {σ : Type} [BoundedJoinSemilattice σ]
    {c₁ c₂ : Delivery.Config R σ} (h : Delivery.Reachable c₁ c₂)
    (r : Fin R) : ∀ x ∈ c₁.seen r, x ∈ c₂.seen r := by
  induction h with
  | refl => intro _ hx; exact hx
  | tail _ hs ih =>
      intro x hx
      have hx₂ := ih x hx
      cases hs with
      | update r' v =>
          show x ∈ (if r = r' then v :: _ else _)
          by_cases hr : r = r'
          · subst hr; simp; exact Or.inr hx₂
          · simp [hr]; exact hx₂
      | send r' r'' => exact hx₂
      | deliver r' p hp =>
          show x ∈ (if r = r' then p.2 ++ _ else _)
          by_cases hr : r = r'
          · subst hr; simp; exact Or.inr hx₂
          · simp [hr]; exact hx₂
```

Exercise 9.3 (★★★, hints). State fairness as: for every reachable configuration and every update seen by some replica, there is a further reachable configuration in which a message carrying it is delivered everywhere it is missing. Then convergence is Chapter 8's theorem applied at the limit of a fair schedule. The subtlety worth savoring: fairness quantifies over *schedules*, not steps — you are proving something about all sufficiently patient adversaries, which is why the relational semantics (not the deterministic driver) is the right arena.

Chapter 10

Exercise 10.1 is `VV.tick_inflationary` — definitionally `GCounter.increment_inflationary`, which is the exercise's point.

Exercise 10.2. The merge dominates both histories (`le_sup_left`/`right`) and is the least such:

```
theorem vv_merge_lub {R : Nat} {v w u : VV R} (hv : v ⊑ u) (hw : w ⊑ u) :
    v ⊔ w ⊑ u :=
  sup_le v w u hv hw
```

Exercise 10.3 (★★★, hints). A dotted version vector is a version vector plus one loose "dot" (i, k) with k *above* the contiguous prefix — exactly the shape of an OR-Set tag whose predecessor tags may be missing. Model it, define the order (contiguous part pointwise, dot by coverage), and check which of the Chapter 10 theorems survive. The literature entry point is Preguiça et al., *Dotted Version Vectors* (2010).

Chapter 11

Exercise 11.1.

```
def opPNCounter (R : Nat) : OpBased.OpCRDT (Bool × Fin R) (PNCounter R) where
  init := ⊥
  apply o p :=
    if o.1 then (GCounter.increment o.2 p.1, p.2)
    else (p.1, GCounter.increment o.2 p.2)
  comm o₁ o₂ s := by
    by_cases h₁ : o₁.1 <;> by_cases h₂ : o₂.1 <;> simp [h₁, h₂]
    · rw [increment_comm]
    · rw [increment_comm]
```

Same-signed operations commute by Exercise 3.1; opposite-signed ones act on different components and commute syntactically.

Exercise 11.2. The counterexample is the main text's `op_dup_overcounts`. For the state-based half, the invariant is Chapter 8's keystone: replacing a delivery list by any list with the same *members* (multiplicity be damned) fixes the fold — `foldJoin_eq_of_same_mems` is exactly the at-least-once tolerance statement.

Exercise 11.3 (★★★, hints). For the interval lemma: `g ⊔ h = h` is `le_iff_sup_eq`; the smallest delta sends slot i to `h i` when it exceeds `g i` and to 0 otherwise — smallest because any delta must dominate the changed slots (argue pointwise), and 0 is $\bot$ elsewhere.

Chapter 12

Exercise 12.1. Zero new proofs: extend the product by one more component whose instance exists (`LWWReg R Nat` works out of the box) and re-instantiate `sec_state_based` at the new type. If you wrote anything resembling a convergence argument, you missed the chapter's sentence about where the work lives.

Exercise 12.2. A `clearList` built from `ORSet.remove` on every current member tombstones only *observed* tags: a concurrent add survives the clear. You chose add-wins semantics — the same choice `addWins` made for single removes, inherited wholesale. If the product manager wanted "clear beats everything in flight", that is an arbitration by time, which is Chapter 6's register wrapped around a set — with Chapter 6's data-loss trade rejoining the party.

Exercise 12.3. The bug report writes itself — "I deleted bread by accident, and now I can never add bread again" — and the review comment is Refutation 5.2 cited by number.

Chapter 13

Exercise 13.1. (a) Tags: a set that grows, or an OR-Set if untagging matters — replicable. (b) Read receipts: grow-only per reader — replicable, it is a G-Set family. (c) Highest bid: the *value* joins as max, but "a bid is accepted only if higher than all previous" is cross-replica — the auction's *outcome* needs coordination even though its *ticker* does not. (d) Usernames: uniqueness is the canonical not-join-closed invariant — coordinate. (e) Likes: a PN-Counter, accepting the negative-blip caveat of Chapter 13. (f) The ledger history: an append-only set of signed entries is replicable; it is the *balance predicate* over it that is not — store the history as a CRDT, enforce overdraft rules with escrow or consensus.

Exercise 13.2. Hints, since the pleasure is real: within-escrow is a conjunction of pointwise bounds, and pointwise bounds survive pointwise max — `Nat.max_le` is the whole of part (a). For part (b), `value = P - N`, bound `N`'s total by the budget sum using the same seed-generalized sum lemmas as Chapter 3, and finish with `omega`. Note which direction of the escrow inequality made merge-closure work; that asymmetry *is* the design.

Further Reading

- **Shapiro, Preguiça, Baquero & Zawirski**, *Conflict-Free Replicated Data Types*, SSS 2011 (and the companion INRIA report *A Comprehensive Study of Convergent and Commutative Replicated Data Types*). The papers that named the field, defined strong eventual consistency, and catalogued the zoo of Part II. Read them after Chapter 8 and notice how much of their Section 3 you can now reprove.
- **Gilbert & Lynch**, *Brewer's Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services*, SIGACT News 2002. The formalization behind Chapter 1's one honest page on CAP.
- **Lamport**, *Time, Clocks, and the Ordering of Events in a Distributed System*, CACM 1978. Happens-before, logical clocks, and the sentence quoted at the head of Chapter 10. Still the best-written paper in the field.
- **Radul & Sussman**, *The Art of the Propagator*, MIT CSAIL Technical Report, 2009. Where the predecessor book began, and where this one's reading of "a replica is a cell" comes from.
- **Almeida, Shoker & Baquero**, *Delta State Replicated Data Types*, JPDC 2018. The midpoint of Exercise 11.3: ship joinable deltas, keep the state-based guarantees.
- **Preguiça, Baquero, Almeida, Fonte & Gonçalves**, *Dotted Version Vectors*, 2010. The sequel to Chapter 10 that Exercise 10.3 sketches.
- **Kleppmann**, *Designing Data-Intensive Applications*, O'Reilly 2017, Chapters 5 and 9. The systems context — replication, linearizability, consensus — that Chapter 13 points at when it says "coordinate".
- **The Lean 4 Documentation**, `https://lean-lang.org`, and **Mathlib4**, `https://leanprover-community.github.io/mathlib4_docs/`. `Mathlib.Order` scales the hand-rolled hierarchy of these two books to a large verified library; `List.Perm` and `Multiset` there are the industrial versions of Chapter 8's constructions.